Universidade Federal de Minas Gerais
Escola de Engenharia
Curso de Graduação em Engenharia de Controle e Automação

Instrumentação Eletrônica de um Automodelo Radiocontrolado para Controle Dinâmico em Malha Fechada

Karine Giselle de Souza Silva

Orientador: Prof. Armando Alves Neto, Dr.

Belo Horizonte, Novembro de 2026

Monografia

Instrumentação Eletrônica de um Automodelo Radiocontrolado para Controle Dinâmico em Malha Fechada

Monografia submetida à banca examinadora designada pelo Colegiado Didático do Curso de Graduação em Engenharia de Controle e Automação da Universidade Federal de Minas Gerais, como parte dos requisitos para aprovação na atividade Projeto Final de Curso II.

Belo Horizonte, Novembro de 2026

Resumo

Este trabalho tem como objetivo caracterizar experimentalmente o sistema de propulsão do automodelo radiocontrolado Kyosho Mad Force Kruiser 2.0 VE, equipado com motor brushless e Controlador Eletrônico de Velocidade (*Electronic Speed Controller* – ESC). Busca-se desenvolver e validar um método de baixo custo para a estimação indireta do torque produzido pelo conjunto motor–ESC–transmissão, de modo a fornecer subsídios para o desenvolvimento de estratégias de controle do propulsor em malha fechada.

A metodologia emprega um sensor de corrente ACS758, conectado a um microcontrolador Arduino Due para a aquisição dos sinais elétricos, e uma balança digital para a medição da força de tração durante ensaios com o automodelo imobilizado e as rodas em contato com o solo. A partir da força medida e do raio efetivo da roda motriz, calcula-se o torque desenvolvido pelo sistema de propulsão, permitindo estabelecer sua relação com a corrente elétrica consumida. Os dados obtidos são analisados quanto à concordância com o modelo teórico do *powertrain* e submetidos a um tratamento das incertezas de medição, de forma a determinar a faixa de validade e a confiabilidade do método de estimação proposto. Os resultados indicam uma relação aproximadamente linear entre torque e corrente na faixa de operação avaliada, além de evidenciarem limitações associadas à região próxima ao neutro do ESC e às condições de alimentação da bateria.

Palavras-chave: Automodelo radiocontrolado. Sensor de corrente. Estimação de torque. Controle em malha fechada. Instrumentação eletrônica. Incerteza de medição.

Abstract

This work aims to experimentally characterize the propulsion system of the Kyosho Mad Force Kruiser 2.0 VE radio-controlled model car, equipped with a brushless motor and an Electronic Speed Controller (ESC). It seeks to develop and validate a low-cost method for the indirect estimation of the torque produced by the motor–ESC–transmission assembly, in order to provide support for the development of closed-loop torque control strategies.

The methodology employs an ACS758 current sensor, connected to an Arduino Due microcontroller for the acquisition of the electrical signals, and a digital scale to measure the traction force during tests with the model car immobilized and its wheels in contact with the ground. Based on the measured force and on the effective radius of the driving wheel, the torque developed by the propulsion system is calculated, allowing its relationship with the consumed electric current to be established. The obtained data are analyzed for agreement with the theoretical powertrain model and subjected to a measurement uncertainty treatment, in order to determine the validity range and reliability of the proposed estimation method. The results indicate an approximately linear relationship between torque and current within the evaluated operating range, in addition to revealing limitations associated with the region near the ESC neutral point and with the battery supply conditions.

Keywords: *Closed-loop control. Current sensor. Electronic instrumentation. Measurement uncertainty. Radio-controlled model car. Torque estimation.*

Sumário

Resumo i

Abstract ii

Lista de Figuras vii

Lista de Tabelas viii

1 Introdução 1

1.1 Motivação e Justificativa . . . 1
1.2 Objetivos do Projeto . . . 2
1.3 Local de Realização . . . 2
1.4 Estrutura da Monografia . . . 2

2 Descrição do Processo 4

2.1 O Sistema de Propulsão . . . 4
2.2 Situação Atual e Estado da Arte . . . 6
2.2.1 Sensor Inercial (IMU) – MPU-6050 . . . 6
2.2.2 Sensor Magnético de Efeito Hall SS41F . . . 8
2.2.3 Shunt Resistivo + Amplificador de Instrumentação (INA240) . . . 10
2.2.4 Sensor de Efeito Hall Linear (ACS758) . . . 12
2.2.5 Síntese Comparativa . . . 14
2.3 Proposta do Projeto . . . 15
2.3.1 O seletor de mínimo . . . 16
2.3.2 O *feedforward* . . . 16
2.3.3 A cascata . . . 17
2.3.4 O controle inferencial . . . 18
2.3.5 As proteções do comando . . . 19
2.3.6 A classificação das variáveis . . . 19
2.3.7 A implementação digital . . . 19

2.4 Resumo do Capítulo 22

3 Metodologia **23**
3.1 O Processamento dos Dados e Ajuste das Retas 23
3.2 Aquisição e registro de dados 24
3.3 Teste em Bancada 25
3.4 Ensaio em Vazio 28
3.5 Ensaio de Tração de Corrente versus Força 30
3.5.1 A Montagem do Ensaio 30
3.5.2 Ancoragem e Geometria do Cabo 31
3.5.3 Massa Adicional 31
3.5.4 Raio Efetivo da Roda 32
3.5.5 O Procedimento do Ensaio 34
3.5.6 O Tratamento dos Dados 35
3.6 A Malha de Controle no Código da Simulação 36
3.6.1 O Estimador de Torque 36
3.6.2 O Seletor e a Rampa da Referência 36
3.6.3 O Controlador Mestre 37
3.6.4 O Feedforward e o Caminho Direto 37
3.6.5 O Controlador Escravo e o Clamping 38
3.7 Incertezas de Medição 38
3.8 Resumo do Capítulo 41

4 Resultados **45**
4.1 Testes em Bancada 45
4.2 Regiões de operação 46
4.3 Ensaio em Vazio 47
4.4 A Curva Característica Estática 49
4.5 A Dinâmica entre Comando e Corrente 51
4.6 O Efeito da Bateria nos Ensaios 54
4.7 A Sintonia dos Controladores 55
4.8 O Desempenho da Malha em Simulação 57
4.9 Resumo do Capítulo 58

5 Conclusões **60**
5.1 Considerações Finais 60
5.2 Propostas de Continuidade 62

Referências Bibliográficas **63**

Appendices **65**

A Scripts de processamento dos dados **66**

A.1 Caracterização estática 66

A.2 Dinâmica entre comando e corrente 73

A.3 Simulação da malha de controle 79

A.4 Automação do ensaio 90

A.5 Efeito da bateria nos ensaios 96

Lista de Figuras

2.1 Visão explodida de um motor brushless – o campo magnético produzido pelo estator interage com os ímãs permanentes do rotor, gerando torque. 5

2.2 Conexão do módulo MPU-6050 a um microcontrolador Arduino por meio do barramento I^2C, com alimentação e sinais SDA e SCL. 8

2.3 Comportamento do SS41F diante do polo norte de um ímã: a saída digital do sensor, indicada pelo LED, assume nível alto na presença do polo norte e nível baixo na do polo sul. 9

2.4 Amplificador de instrumentação INA240 para leitura de corrente por shunt resistivo. 12

2.5 Sensor de corrente de efeito Hall linear ACS758. 13

2.6 Malha de controle proposta: seletor de mínimo, pré-alimentação C_{ff}, cascata mestre–escravo C_1–C_2 e estimador $\hat{G}$. 15

2.7 Ordem de execução dentro de cada amostra de 10 ms: a corrente lida alimenta o estimador, a referência é cortada no seletor e rampada no caminho do mestre, e mestre, *feedforward* e escravo se sucedem antes de o comando seguir à planta. 21

3.1 Teste em bancada da cadeia de corrente, com a fonte ajustável, o multímetro de referência e o sensor ACS758. 26

3.2 Sensor de corrente ACS758 instalado no ramo de alimentação, entre a bateria e o ESC, por adaptação com conector Deans-T. 27

3.3 Sensor de corrente instalado no automodelo durante a conferência da cadeia, com o fio vermelho do ramo de potência interrompido para a inserção do multímetro em série. 28

3.4 Ensaio em vazio, rodas motrizes suspensas em caixa, sem contato com a mesa. 29

3.5 Montagem do ensaio de tração, automodelo ancorado pela balança de gancho suspensa entre o engate traseiro e a estrutura fixa. 32

3.6 Halteres fixados na dianteira do veículo para aumentar a carga normal sobre os pneus. 33
3.7 Fluxograma do estimador de torque a cada amostra: cálculo da estimativa bruta, limite à faixa validada e filtro de primeira ordem. 37
3.8 Fluxograma do seletor de mínimo e da rampa da referência, com a bifurcação para o *feedforward* (antes da rampa) e para o mestre (depois). 38
3.9 Fluxograma do controlador mestre: erro contra o torque estimado, lei PI com ganho programado e integrador condicionado à não saturação do escravo. 42
3.10 Fluxograma do *feedforward* e do caminho direto: corrente esperada, comando equivalente rampado e soma com a correção do mestre. 43
3.11 Fluxograma do controlador escravo com anti-*windup* por *clamping*: saturação nos limites da região validada e condicionamento do integrador. 44

4.1 Verificação da cadeia de corrente: ajuste $y = 1{,}0742\,x$ sobre a resposta ideal $y = x$. 46
4.2 Corrente de perdas I_0 em função do comando normalizado u, com a zona morta destacada e o ajuste linear. 48
4.3 Força de tração mediana em função do comando normalizado, para os dois sentidos da escada, com as regiões de operação delimitadas. 50
4.4 Janelas brutas de dois comandos do ensaio de tração de corrente, com corrente bruta (cinza) e média móvel (vermelho): o patamar inicial é o neutro que precede o comando, e a subida é rápida uma vez iniciada. . 52
4.5 Janela bruta do degrau de comando 103°, com corrente bruta a 100 Hz e média móvel: o repouso que precede o comando e o platô de regime. . 52
4.6 Janela bruta do degrau de comando 109°, com corrente bruta a 100 Hz e média móvel: o mesmo padrão no fim da região de aderência. 53
4.7 Respostas ao degrau alinhadas no início elétrico, normalizadas, com a média das dez bordas e o ajuste de primeira ordem. 54
4.8 Mediana da corrente por degrau de comando: sessão de referência, com a bateria em bom estado, e sessão antiga, com a bateria degradada, destacando o colapso das retomadas de 114°. 55
4.9 Desempenho da malha em simulação: resposta ao degrau de referência (PSR), rejeição da perturbação de afundamento de tensão e deslocamento das perdas (PRP), retorno de referência e atuação do seletor de aderência. 58

Lista de Tabelas

2.1 Implementação da malha: onde cada elemento vive no código (script A.3). 20

4.1 Verificação da cadeia de corrente: pares de leituras do multímetro de referência (x) e do sistema (y), em amperes. 46
4.2 Faixas de comando das regiões de operação, por ensaio (graus). 47
4.3 Ensaio em vazio: corrente de perdas média por comando. 48
4.4 Caracterização estática: medianas das quatro repetições por comando, com desvio-padrão entre parênteses. Correntes em A; forças em N. . . . 49
4.5 As duas malhas da cascata e seus elementos de sintonia. 57

Capítulo 1

Introdução

Este trabalho apresenta a instrumentação eletrônica de um automodelo radiocontrolado, desenvolvida com o propósito de caracterizar a dinâmica do seu sistema de propulsão e, a partir dela, fornecer ferramentas experimentais para implementar o controle em malha fechada.

Na configuração original do veículo, o automodelo opera em malha aberta: o comando de aceleração é aplicado sem qualquer conhecimento sobre o resultado efetivamente alcançado pelo sistema. Um controlador em malha fechada, por sua vez, é capaz de ajustar continuamente esse comando a partir de medições obtidas do próprio sistema, de modo a aproximar o comportamento real do veículo daquele desejado.

Essa capacidade de ajuste contínuo está diretamente relacionada à solução de dois problemas centrais da teoria de controle: o Problema de Seguimento de Referência (PSR), que consiste em fazer com que a velocidade do veículo acompanhe fielmente uma referência especificada ao longo do tempo, e o Problema de Rejeição de Perturbações (PRP), que consiste em manter esse comportamento mesmo diante de perturbações externas, como variações de carga, do terreno ou da alimentação elétrica do sistema.

1.1 Motivação e Justificativa

A Universidade Federal de Minas Gerais (UFMG), por meio do Departamento de Engenharia Eletrônica (DELT), oferece a disciplina de Veículos Autônomos, que proporciona aos estudantes de engenharia a oportunidade de desenvolver e experimentar soluções de controle, percepção e robótica em veículos reais. Este trabalho utiliza como objeto experimental o automodelo elétrico radiocontrolado Kyosho Mad Force Kruiser 2.0 VE, empregado nas atividades da disciplina.

Por operar em malha aberta, o automodelo responde de forma imprevisível a fatores externos ao comando aplicado, como a variação do coeficiente de atrito do solo

em diferentes trechos da pista ou a queda de tensão da bateria ao longo do uso. Essas variações constituem perturbações que o PRP deveria rejeitar e que, quando não compensadas, comprometem também o PSR, impedindo que a velocidade real do veículo acompanhe a referência desejada.

Além disso, variações abruptas de aceleração geram picos de esforço mecânico que aceleram o desgaste do sistema de propulsão, reduzindo sua vida útil e a disponibilidade do automodelo para as aulas práticas da disciplina. A instrumentação eletrônica proposta neste trabalho fornece os dados necessários para caracterizar a dinâmica do sistema de propulsão, fundamentando o desenvolvimento futuro de um controle em malha fechada capaz de monitorar o esforço exigido do automodelo e atenuar picos de aceleração, evitando comandos excessivos que desgastem precocemente o sistema.

1.2 Objetivos do Projeto

Tendo em vista o exposto, o objetivo geral é instrumentar o automodelo para caracterizar a dinâmica do seu sistema de propulsão, de modo a viabilizar o desenvolvimento de um controle do torque do propulsor em malha fechada. Como objetivos específicos, busca-se: (i) selecionar e instalar a instrumentação eletrônica necessária à aquisição de dados do sistema; (ii) desenvolver um método experimental para caracterizar a dinâmica do sistema de propulsão; (iii) validar os dados obtidos por comparação com um modelo teórico do sistema; e (iv) quantificar os erros e as incertezas associados ao método de medição proposto.

1.3 Local de Realização

Este projeto é desenvolvido no Laboratório de Sistemas Inteligentes (LSI), localizado na Sala 2515 do Bloco 1 da Escola de Engenharia da Universidade Federal de Minas Gerais. O LSI é um laboratório pluridisciplinar e compartilhado, com infraestrutura voltada à montagem e ao teste de circuitos eletrônicos, no qual pesquisadores do Departamento de Engenharia Eletrônica atuam no desenvolvimento de sistemas embarcados, de automação, robótica e controle.

1.4 Estrutura da Monografia

O restante deste trabalho está organizado em quatro capítulos. O Capítulo 2 descreve o sistema de propulsão do automodelo e a instrumentação eletrônica escolhida para a aquisição de dados. O Capítulo 3 apresenta a metodologia experimental adotada

para a caracterização da dinâmica do sistema de propulsão, incluindo a montagem do ensaio. O Capítulo 4 expõe os resultados obtidos, o tratamento das incertezas de medição e o desempenho da malha de controle projetada sobre os modelos identificados. O Capítulo 5 fecha o trabalho com as conclusões e as propostas de continuação.

Capítulo 2

Descrição do Processo

Este capítulo apresenta o sistema de propulsão do automodelo utilizado como objeto de estudo e discute as abordagens de instrumentação consideradas para a caracterização de sua dinâmica. Para cada abordagem, são apresentados o princípio de funcionamento, a proposta de controle em malha fechada, as vantagens e as limitações, de modo a fundamentar as decisões do projeto.

2.1 O Sistema de Propulsão

O automodelo Kyosho Mad Force Kruiser 2.0 VE, de escala 1/8, é equipado com um motor brushless Orion NEON 8BL (Ori28188), cuja constante de velocidade é de 2100 rpm/V – rotações por minuto por volt aplicado, em condições próximas à operação sem carga mecânica significativa. É constituído por um estator, componente fixo com núcleo ferromagnético envolvido por enrolamentos de fio de cobre, e por um rotor, componente móvel com ímãs permanentes fixados na carcaça. No rotor, os ímãs permanentes produzem campo magnético próprio em razão das propriedades do material, enquanto no estator, a corrente elétrica que percorre os enrolamentos gera um campo eletromagnético. A interação entre esses dois campos produz o torque responsável pela rotação do eixo, como ilustrado na Figura 2.1.

A continuidade desse movimento depende da comutação eletrônica, processo pelo qual a alimentação das três fases do motor é alternada em uma sequência que modifica continuamente a orientação do campo magnético resultante do estator, produzindo um campo girante. Essa função é realizada pelo Controlador Eletrônico de Velocidade (*Electronic Speed Controller* – ESC) Orion Vortex R8, que recebe o comando de aceleração e controla a energia elétrica transferida da bateria para os enrolamentos. Como o motor é do tipo *sensorless*, o ESC estima sua posição a partir da força contraeletromotriz (*back-EMF*) – a tensão induzida nos enrolamentos pela rotação dos

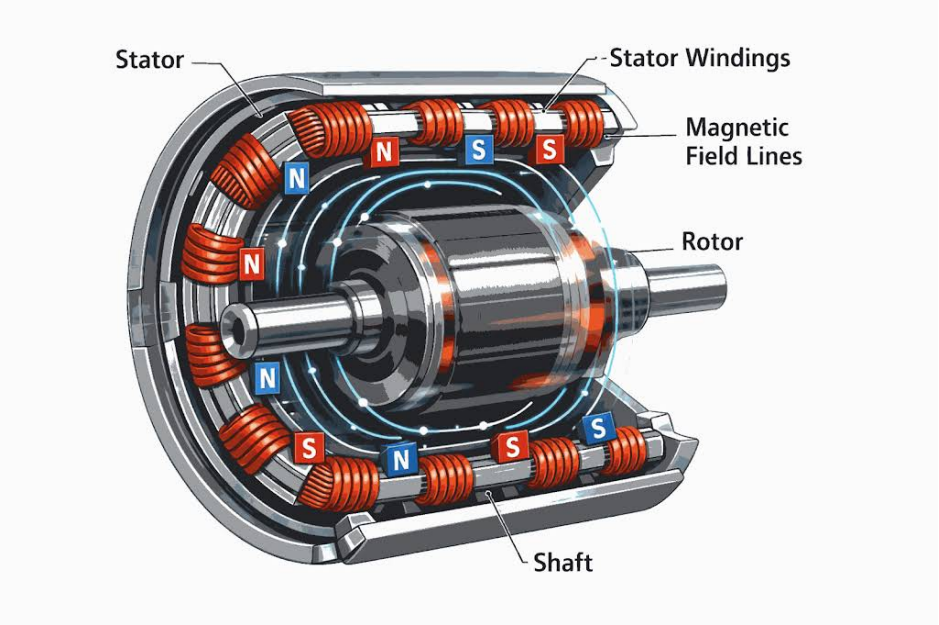


Figura 2.1: Visão explodida de um motor brushless – o campo magnético produzido pelo estator interage com os ímãs permanentes do rotor, gerando torque.
Fonte: [1].

ímãs permanentes do rotor. Para isso, uma das fases permanece temporariamente sem alimentação durante a comutação, permitindo ao controlador estimar os instantes adequados para a próxima etapa. Dessa forma, o ESC mantém o campo do estator sincronizado com o movimento do rotor, possibilitando a conversão contínua da energia elétrica em energia mecânica.

Segundo o manual do fabricante, o ESC suporta corrente contínua de até 120 A e corrente de pico de até 760 A [2] (p. 28). A alimentação do sistema é realizada por uma bateria de polímero de lítio (LiPo) de duas células, com tensão nominal de 7,4 V, capacidade de 5200 mAh e taxa de descarga de 60C. Ao longo do trabalho, também são empregadas baterias preexistentes, com anos de uso, cujo efeito sobre os ensaios será analisado.

O comando de aceleração aplicado ao ESC pode ter duas origens, selecionadas por uma chave de comutação, sendo a primeira o sinal proveniente do controle remoto, por meio do receptor de rádio, e a segunda o sinal gerado por uma Raspberry Pi embarcada no veículo, com bateria própria e conversor *step-down* para alimentação em 5 V. Ambos geram um pulso de modulação por largura de pulso (*Pulse Width Modulation* — PWM), no qual a duração de cada pulso codifica o valor do comando.

2.2 Situação Atual e Estado da Arte

Para atender os objetivos, podem ser consideradas diferentes alternativas de medição, sendo que a escolha da mais adequada entre elas depende de fatores como precisão, complexidade de implementação, custo e adequação às condições experimentais. Nesse contexto, as subseções a seguir apresentam o princípio de funcionamento de cada técnica, discutem sua aplicabilidade na caracterização do sistema e fundamentam a opção adotada.

2.2.1 Sensor Inercial (IMU) – MPU-6050

A Unidade de Medição Inercial (Inertial Measurement Unit – IMU) MPU-6050 reúne um acelerômetro e um giroscópio triaxiais, que medem, respectivamente, a aceleração a e a velocidade angular ω ao longo dos eixos x, y e z. Para caracterizar o movimento do automodelo, a IMU deve ser fixada rigidamente em uma região próxima ao centro geométrico do chassi, com seus eixos alinhados às direções longitudinal, lateral e vertical do veículo, e de preferência afastada de fontes intensas de vibração e protegida contra impactos.

O torque produzido pelo motor T_{motor} é transmitido às rodas pelo sistema de transmissão, resultando em um torque nas rodas T_{roda} que se relaciona com a redução da transmissão i e o rendimento mecânico η,

$$T_{\text{roda}} = i\,\eta\,T_{\text{motor}}.$$

Nas rodas, o torque é convertido em força de tração $F_{\text{tração}}$, que atua sobre o solo e impulsiona o veículo. Considerando o raio efetivo r_{ef} da roda e desprezando perdas no contato pneu-solo,

$$F_{\text{tração}} = \frac{T_{\text{roda}}}{r}.$$

A aceleração longitudinal do automodelo depende da força resultante, obtida pela diferença entre a força de tração e as forças resistivas ao movimento. De acordo com a segunda lei de Newton, essa relação é descrita por

$$m\,a_x = F_{\text{tração}} - F_{\text{resistivos}}, \tag{2.1}$$

em que m é a massa total do veículo, a_x é a aceleração longitudinal e $F_{\text{resistivos}}$ representa a resultante das forças resistivas, como o atrito de rolamento e o arrasto aerodinâmico.

No sistema de controle, emprega-se a estrutura básica de malha fechada com realimentação da variável de processo, que é a velocidade longitudinal do automodelo, $v(t)$, estimada como $\hat{v}(t)$ pela integração numérica da aceleração longitudinal medida pela IMU. A variável manipulada é o comando de aceleração u, enviado ao ESC,

$$\hat{v}(t) = \hat{v}(0) + \int_0^t a_x(\xi)\, d\xi,$$

em que $\hat{v}(0)$ é a condição inicial da estimativa.

A estimativa por integração introduz o erro de medição do acelerômetro, que continua sendo acumulado na integral, mesmo para aceleração igual à zero. Assim, um desvio constante produz na velocidade estimada um erro que cresce linearmente com o tempo, e a estimativa é confiável em trajetos curtos, degradando-se progressivamente em operação prolongada e exigindo filtragem e correção contínuas.

A velocidade desejada é representada pela referência $v_{\text{ref}}(t)$, que define o erro de controle.

$$e_v(t) = v_{\text{ref}}(t) - \hat{v}(t).$$

A partir do erro de velocidade $e_v(t)$, o controlador calcula o comando $u(t)$, transmitido ao ESC para controlar a energia fornecida pela bateria aos enrolamentos do motor, produzindo o torque transmitido às rodas. As perturbações do processo são as variações das forças que agem sobre o veículo sem depender do comando, como o atrito de rolamento, a inclinação do solo e a perda de aderência, que alteram a aceleração resultante para um mesmo valor de u. A IMU mede a nova aceleração do chassi, a estimativa $\hat{v}(t)$ é atualizada por integração e realimentada ao controlador, que fecha o ciclo a cada período de amostragem.

Quanto à proteção do sistema, o microcontrolador pode detectar picos de aceleração e limitar a taxa de variação do comando u, suavizando a aceleração e reduzindo os picos de torque na transmissão. Essa ação é apenas parcialmente preventiva, pois atua depois que o pico já começou a ser medido. Desse modo, o sensor cumpre o objetivo de realimentação de velocidade, mas não o de proteção elétrica direta dos componentes.

A Figura 2.2 ilustra a conexão de um módulo MPU-6050 a um microcontrolador Arduino por meio do barramento I^2C: a alimentação (V_{CC} e terra, GND) e dois sinais de comunicação, SDA (Serial Data), responsável pela transmissão dos dados, e SCL (Serial Clock), que fornece o sinal de sincronismo.

Em síntese, a IMU oferece uma cadeia de medição simples e de baixo custo, cuja limitação decisiva é a necessidade de identificar os parâmetros dinâmicos do veículo antes de qualquer estimativa de torque. Ela permite medir a aceleração do chassi

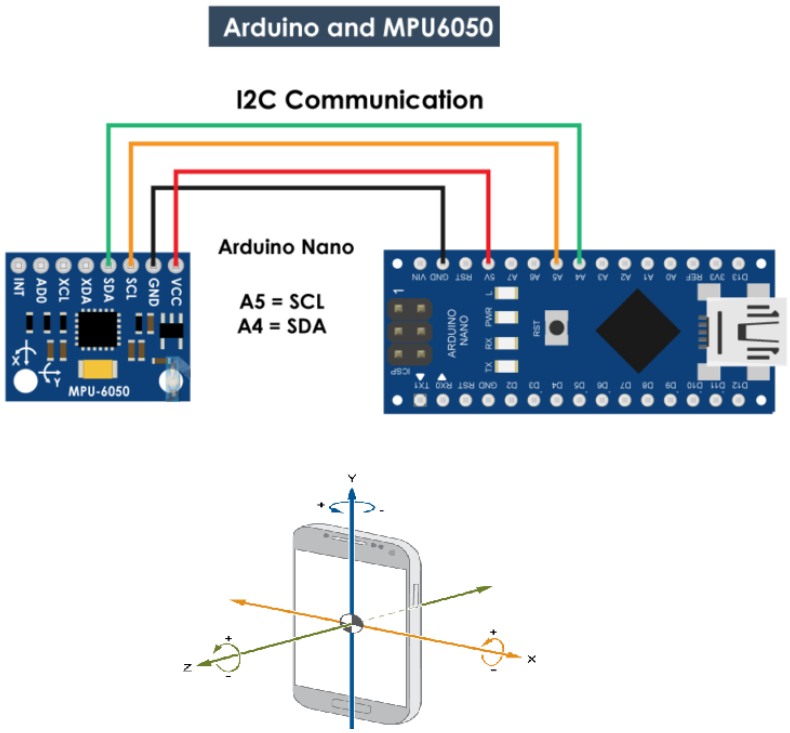


Figura 2.2: Conexão do módulo MPU-6050 a um microcontrolador Arduino por meio do barramento I^2C, com alimentação e sinais SDA e SCL.
Fonte: [3].

sem depender de um modelo mecânico, apresenta dimensões reduzidas, baixo custo e interface digital simples, e fornece medições de velocidade angular que podem ser úteis em análises posteriores da dinâmica do veículo. Em contrapartida, a estimativa por integração acumula erros e pode exigir filtragem e correção de deriva, as medições são sensíveis às vibrações e à projeção da gravidade nos eixos do sensor, e a aceleração não permite determinar diretamente a corrente elétrica para garantir a proteção dos componentes contra sobrecorrente.

2.2.2 Sensor Magnético de Efeito Hall SS41F

O SS41F é um sensor digital de efeito Hall do tipo bipolar, cuja saída assume nível alto na presença do polo norte de um ímã e nível baixo na presença do polo sul. Quando um anel magnetizado – um ímã de ferrita com polos alternados ao longo da circunferência – gira rigidamente acoplado à roda ou ao eixo, o sensor produz N transições digitais por revolução, sendo N o número de alternâncias de polos. Esse é o princípio dos encoders de roda usualmente empregados em robótica, nos quais o microcontrolador conta as transições em uma janela de tempo para estimar a rotação.

Com c pulsos contados em uma janela de duração T_j, a velocidade angular do eixo

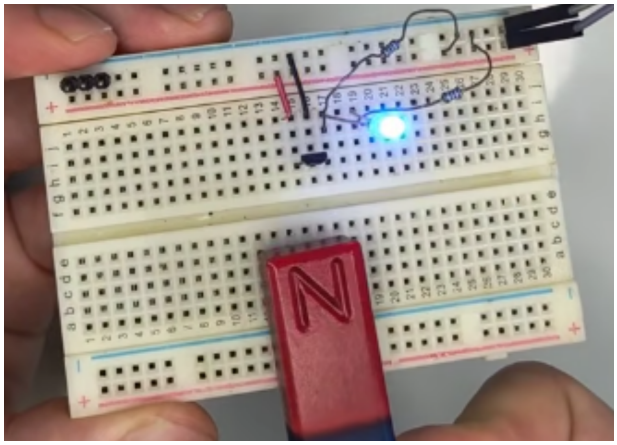


Figura 2.3: Comportamento do SS41F diante do polo norte de um ímã: a saída digital do sensor, indicada pelo LED, assume nível alto na presença do polo norte e nível baixo na do polo sul.

Fonte: elaborada pela autora (2026).

que carrega o anel é $\omega = 2\pi c/(NT_j)$, sendo que a contagem fornece diretamente a velocidade ω_m quando o anel está no eixo do motor e fornece ω_{roda}, que se relaciona com ω_m pela redução i da transmissão, quando está na roda.

Na fixação no veículo, o anel magnetizado é acoplado rigidamente à roda ou ao eixo cuja rotação se deseja medir, e o sensor é montado no chassi, fixo, próximo ao anel. Entre o elemento Hall e os polos mantém-se uma entreferência – a distância entre o sensor e o anel – pequena e constante. O campo magnético decai rapidamente com a distância, razão pela qual uma folga excessiva enfraquece o sinal, enquanto uma folga mínima demais arrisca o contato mecânico. A conexão com o microcontrolador é direta, apresentando uma saída digital que pode ser lida por interrupção, dispensando circuitos de condicionamento adicionais, uma vez que a saída do sensor já é um nível lógico compatível com a entrada do microcontrolador.

A estimativa de torque a partir da velocidade, contudo, requer a dinâmica rotacional do conjunto. Com momento de inércia J refletido no eixo do motor, coeficiente de atrito viscoso B e torque de carga T_L, a Segunda Lei de Newton para a rotação, aplicada a motores elétricos, é dada por:

$$T_{\text{motor}} = J\,\frac{d\omega_m}{dt} + B\,\omega_m + T_L. \tag{2.2}$$

O torque depende, nesse caso, da derivada da velocidade medida e de três parâmetros, J, B e T_L, que precisam ser identificados separadamente, pois são propriedades do conjunto veículo–carga. O J depende da massa e da redução, B do atrito interno da transmissão e T_L da carga e da inclinação, e todos estão sujeitos a mudança com

a configuração do veículo, de forma que a leitura do sensor, sozinha, não revela essas mudanças, o que compromete a confiabilidade da estimativa. Há, ainda, um limite de resolução inerente ao método, pois, em baixas rotações, poucos pulsos são contados e a estimativa se torna progressivamente mais grosseira. A derivada exigida pela Equação (2.2) amplifica esse ruído de quantização, tornando a estimativa de torque sensível à resolução do sensor.

Em malha fechada, a velocidade medida pode realimentar diretamente um controlador de velocidade, sem modelo de torque associado, da mesma forma que a velocidade estimada pela IMU, com a variável de processo trocada pela velocidade angular obtida da contagem dos pulsos e a variável manipulada pelo comando de aceleração u. A referência de velocidade é comparada à medida, ajustando a resposta de comando até eliminar o erro. Os motores brushless com sensores Hall integrados já são controlados dessa maneira, com os mesmos sinais servindo à comutação e à realimentação, conforme documentado pela Honeywell para a própria família SS41x [4].

Quanto à proteção do sistema, o SS41F não fornece nenhuma grandeza elétrica, e a proteção contra sobrecorrente por limiar não pode ser implementada com esse sensor. A única ação preventiva disponível é a limitação da taxa de variação do comando, que suaviza a excitação do motor, sem garantia efetiva contra sobrecorrente.

No conjunto, o SS41F mede a velocidade com robustez e simplicidade de interface, porém só se aproxima do torque depois de identificadas as variáveis do conjunto. Ele tem baixo custo, robustez mecânica e velocidade de comutação superior a 100 kHz, com interface digital simples, compatível com a leitura por interrupção em microcontroladores comuns, e fornece a velocidade por contagem direta dos pulsos. Em contrapartida, a estimativa de torque só é possível após a determinação de J, B e T_L, conforme a Equação (2.2), e a instalação demanda alinhamento mecânico preciso entre o sensor e o anel, pois desalinhamentos produzem pulsos assimétricos que distorcem a medida.

2.2.3 Shunt Resistivo + Amplificador de Instrumentação (INA240)

No arranjo com shunt resistivo, um resistor de baixa resistência R_s é inserido em série no ramo de alimentação, entre a bateria e o ESC, e pela Lei de Ohm a queda de tensão sobre ele é

$$V_s = R_s\, I.$$

Essa tensão, tipicamente da ordem de milivolts, é amplificada pelo circuito integrado

INA240, cuja saída, referenciada a uma tensão de polarização V_{bias}, é

$$V_{\text{out}} = G\,R_s\,I + V_{\text{bias}},$$

em que G é o ganho fixo do amplificador.

A dissipação de potência no resistor cresce com o quadrado da corrente. Considerando as correntes do ESC, um shunt de 1 mΩ dissipa cerca de 14 W em regime e mais de 500 W nos picos, o que faz o resistor aquecer e consumir parte da energia destinada ao motor. Observa-se, ainda, que o shunt instalado na alimentação do ESC mede a corrente média de entrada, e não a corrente de fase do motor, que é chaveada pelo PWM. Na aproximação usual, a corrente de entrada se relaciona à de fase pela razão cíclica D do chaveamento, a fração do período em que o PWM permanece ligado, o que resulta em:

$$\bar{I}_{\text{bat}} \approx D\,I_{\text{fase}}. \tag{2.3}$$

No motor, a corrente de fase se relaciona ao torque pela constante de torque K_t do propulsor, $T_{\text{motor}} = K_t\,I_{\text{fase}}$, o que permite estimar o esforço mecânico produzido a partir da corrente medida. Como a razão cíclica do chaveamento acompanha o comando u aplicado ao ESC, a cadeia fecha na variável manipulada da malha, pois u define a corrente de fase por meio da Equação (2.3), e a corrente de fase define o torque pelo produto K_t.

O INA240 é montado no circuito de medição, lendo a queda de tensão entre os terminais do shunt, e sua saída, um sinal analógico, é ligada à entrada do microcontrolador, conforme a Figura 2.4. O componente possui largura de banda de 400 kHz e possui rejeição reforçada a transientes de modo comum — a capacidade de rejeitar variações rápidas e simultâneas da tensão nos dois terminais de entrada, produzidas pelo chaveamento do ESC —, que, sem essa rejeição, se somariam ao sinal útil e corromperiam a leitura. Em altas frequências, a indutância parasita do próprio resistor, proveniente de seus terminais e de sua construção, soma-se à queda ômica uma parcela proporcional à variação da corrente, e o resistor deixa de se comportar como uma resistência pura, de forma que a tensão deixa de ser proporcional à corrente instantânea, distorcendo a leitura durante as bordas de chaveamento.

A resolução da medida, por fim, é limitada pelo conversor analógico-digital de n bits que recebe o sinal, pois a sua tensão de referência V_{ref} define o fundo de escala dividido em 2^n degraus, e o menor degrau distinguível corresponde, na corrente, a

$$\Delta I = \frac{V_{\text{ref}}}{2^n\,S}, \tag{2.4}$$

em que S é a sensibilidade do arranjo, $G\,R_s$ no caso do shunt com INA240.

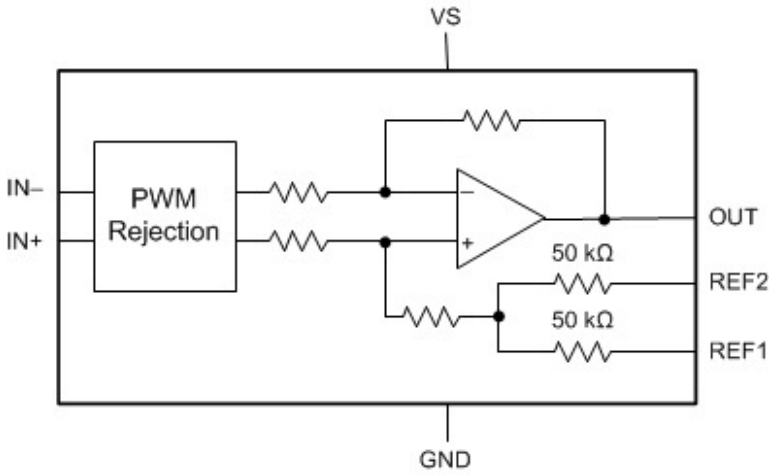


Figura 2.4: Amplificador de instrumentação INA240 para leitura de corrente por shunt resistivo.

Fonte: elaborada pela autora (2026).

Em malha fechada, a corrente medida permite agir diretamente sobre o torque do propulsor e implementar a proteção contra sobrecorrente por limiar. No sistema de controle, a variável de processo é a corrente do propulsor, relacionada ao torque pela constante K_t do motor, e a variável manipulada é o comando de aceleração u, aplicado ao ESC. A cadeia de malha fechada que leva a corrente ao torque é a mesma do sensor de efeito Hall da Seção 2.2.4, onde se fecha a instrumentação do processo. As perturbações são as variações da carga mecânica que exigem mais corrente para um mesmo torque na roda, como a inclinação do terreno e a perda de aderência, e medir a corrente diretamente viabiliza a proteção do sistema contra sobrecorrente por limiar.

Dessa forma, o arranjo com shunt e INA240 privilegia a precisão e o baixo custo em troca da simplicidade de instalação, o que o torna adequado como referência de bancada, mas não como sensor embarcado. Ele proporciona uma medida direta da corrente, com boa linearidade e ampla largura de banda a baixo custo, e a rejeição a transientes de modo comum preserva o sinal sob o chaveamento do ESC. Em contrapartida, a dissipação de potência sob as correntes do ESC é alta e aquece o resistor, cujas dimensões necessárias não são comercialmente disponíveis, e a leitura se distorce em alta frequência pela indutância parasita do resistor.

2.2.4 Sensor de Efeito Hall Linear (ACS758)

O sensor de efeito Hall ACS758 mede a corrente elétrica indiretamente, proporcionando isolamento galvânico, ou seja, não há conexão elétrica direta entre o circuito de

potência e o circuito de medição. O funcionamento baseia-se no efeito Hall, no qual a corrente que percorre um condutor interno gera um campo magnético perpendicular e esse campo, ao atravessar o elemento sensor, desenvolve uma tensão proporcional, e a tensão de saída reproduz a corrente medida:

$$V_{\text{out}} = V_{\text{offset}} + S\,I, \qquad I = \frac{V_{\text{out}} - V_{\text{offset}}}{S},$$

em que S é a sensibilidade, em V/A, e $V_{\text{offset}} = V_{CC}/2$ é o *offset* do sensor, isto é, a tensão de saída quando a corrente é nula ($I = 0$).

O sensor ACS758ECB-200B possui sensibilidade típica de 10 mV/A com alimentação de 5 V, reduzindo-se proporcionalmente em alimentações menores, a 6,6 mV/A com os 3,3 V do Arduino Due, uma vez que a saída é ratiométrica. O condutor interno de corrente apresenta resistência da ordem de 100 µΩ, de modo que a queda de tensão e a dissipação introduzidas pelo sensor são pequenas. Assim como no arranjo com shunt, a resolução da medida é limitada pelo conversor analógico-digital, conforme a Equação (2.4), e, com a sensibilidade reduzida a 6,6 mV/A, o menor degrau distinguível corresponde a 122 mA, valor que entra no orçamento de incertezas da metodologia.

Na fixação, o sensor é instalado em série no ramo de alimentação, entre a bateria e o ESC, e a saída analógica é ligada ao conversor analógico–digital (ADC) do microcontrolador, que converte a tensão de saída no número digital lido pelo programa, conforme a Figura 2.5.

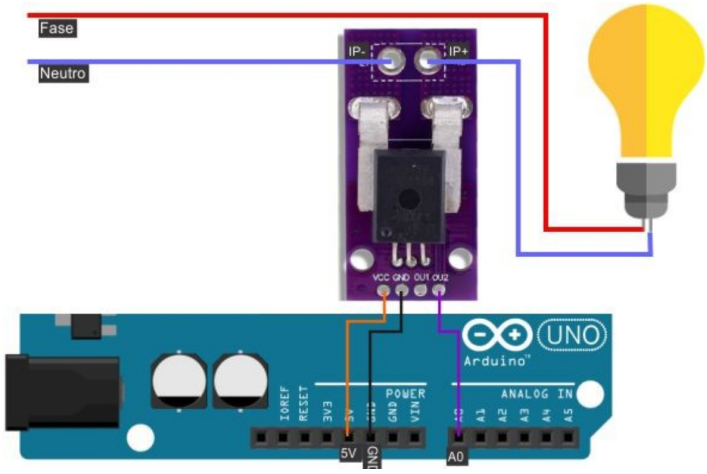


Figura 2.5: Sensor de corrente de efeito Hall linear ACS758.
Fonte: elaborada pela autora (2026).

Em malha fechada, o sensor se comporta como o arranjo com shunt. A variável de processo é o torque estimado a partir da corrente, a variável manipulada é o comando de acionamento u, e a referência de torque desejado é convertida na corrente corres-

pondente e comparada à medida, com o controlador ajustando o comando até anular o erro, na estrutura de malha fechada com realimentação, aqui organizada em cascata, com malha interna de corrente e malha externa de torque. A cadeia que relaciona as grandezas parte do comando e termina no esforço mecânico: o valor u aplicado ao ESC produz a corrente de bateria pela dinâmica comando–corrente, e, com as perdas descontadas e o resultado dividido pelo próprio comando, essa corrente recupera a corrente de fase equivalente, que a curva característica relaciona à força e o raio efetivo ao torque na roda,

$$F_{\text{tração}} = G_F \, \frac{I_{\text{bat}} - I_0(u)}{u} \qquad \text{e} \qquad \hat{T} = F_{\text{tração}} \, r_{\text{ef}} = K_T \, \frac{I_{\text{bat}} - I_0(u)}{u},$$

em que $I_0(u)$ é a corrente de perdas do conjunto, G_F é a força por unidade de corrente, identificada no ensaio de tração, e $K_T = G_F \, r_{\text{ef}}$ é o torque por unidade da mesma corrente, coeficiente que o controlador emprega. Por medir a corrente diretamente, o sensor viabiliza, ainda, a proteção contra sobrecorrente por limiar, com o controlador reduzindo a excitação sempre que a leitura ultrapassa o valor máximo admitido.

Como no arranjo com shunt, o sensor instalado no ramo de alimentação mede a corrente média de entrada, e não a corrente de fase do motor, que é chaveada pelo PWM, mas a conversão entre as duas, dada pela razão cíclica do comando e apresentada com a Equação (2.3), preserva a informação de esforço mecânico do propulsor.

Em síntese, o ACS758 isola eletricamente a medição e não dissipa potência no circuito de potência. O isolamento galvânico protege o circuito de medição das altas correntes, e a banda do sensor acompanha o chaveamento do ESC sem dissipação adicional significativa. Em contrapartida, a resolução fica abaixo da do shunt para a mesma faixa de corrente, o nível de repouso em $V_{CC}/2$ exige compensação, pois qualquer deriva se confunde com corrente contínua e transfere erro para a estimativa de torque, e campos eletromagnéticos externos se somam ao campo da corrente medida.

2.2.5 Síntese Comparativa

A diferença entre as abordagens está, sobretudo, na cadeia de dependências até o torque: a IMU e o SS41F exigem a identificação prévia de parâmetros mecânicos (m, $F_{\text{resistivos}}$, J, B e T_L), que são propriedades do conjunto veículo–carga e mudam com a configuração, enquanto o conjunto Shunt + INA240 e o ACS758 se relacionam ao torque por constantes do próprio propulsor, identificáveis em bancada. Com isso, a corrente elétrica é a grandeza mais direta até o torque, sendo que o sensor Hall linear representa o melhor equilíbrio entre as possibilidades, já que possui isolamento galvânico para proteger o circuito das altas correntes e não dissipa potência.

2.3 Proposta do Projeto

Conforme visto anteriormente, é necessário um instrumento capaz de medir a força de tração na escala do veículo para relacionar a corrente medida ao torque. A escolha desse instrumento foi precedida de um estudo prévio que comparou as principais soluções encontradas na literatura. A balança de gancho mede diretamente a força, dispensa o condicionamento de sinal e a adaptação mecânica, servindo como padrão de bancada para o ajuste e a validação do coeficiente K_T. O protocolo do experimento é apresentado na Metodologia, Capítulo 3.

A Figura 2.6 apresenta o diagrama de blocos da estrutura de controle proposta.

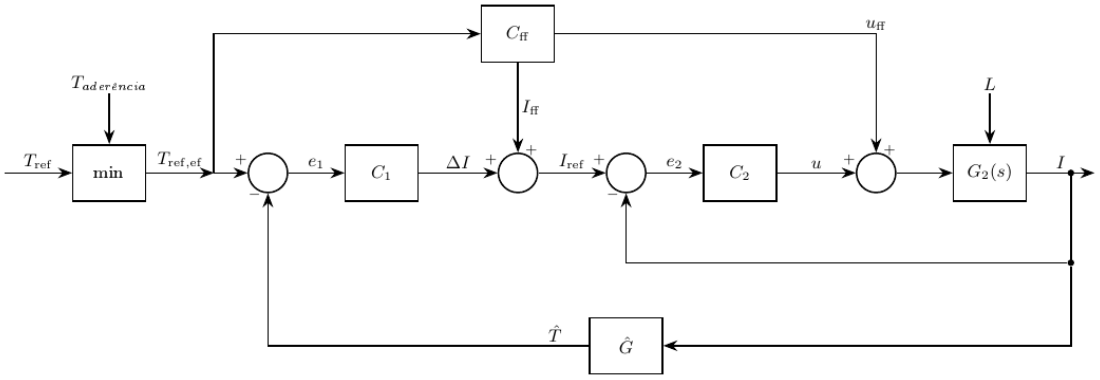


Figura 2.6: Malha de controle proposta: seletor de mínimo, pré-alimentação C_{ff}, cascata mestre–escravo C_1–C_2 e estimador $\hat{G}$.
Fonte: elaborada pela autora (2026).

A corrente é medida no ramo de alimentação pelo sensor de efeito Hall linear ACS758 e convertida em torque por um estimador que não exige o conhecimento das variáveis do sistema, como a redução da transmissão e a eficiência do caminho até a roda, pois todos esses fatores são absorvidos em único coeficiente identificado experimentalmente.

$$I_{\text{fase,eq}} = (I_{\text{bat}} - I_0(u))/u, \tag{2.5}$$

aplica a curva característica, com o G_F identificado no ensaio de tração,

$$F_{\text{tração}} = G_F \, I_{\text{fase,eq}}, \tag{2.6}$$

e fecha na estimativa de torque, com o K_T da mesma cadeia,

$$\hat{T} = K_T \, I_{\text{fase,eq}}, \tag{2.7}$$

em que $K_T = G_F \, r_{\text{ef}}$ relaciona o coeficiente da curva característica ao raio efetivo

medido, convertendo a força do ensaio no torque que a malha controla, e alimenta a malha externa. O estimador, portanto, traduz a corrente medida no esforço mecânico do propulsor por dois elos estáticos, e a dinâmica do caminho pertence apenas à malha interna

2.3.1 O seletor de mínimo

O controle seletivo, também chamado de controle *override*, é a estrutura na qual duas ou mais referências candidatas disputam o comando do processo e um seletor escolhe, a cada instante, aquela que governa o controlador [5]. No seletor de mínimo, empregado neste trabalho, a escolha se dá pela comparação do torque de referência com o limiar de aderência.

$$T_{\text{ref,ef}} = \min\left(T_{\text{ref}},\, T_{\text{aderência}}\right), \tag{2.8}$$

em que T_{ref} é a referência, $T_{\text{aderência}}$ o limiar de aderência e $T_{\text{ref,ef}}$ a referência efetiva, que segue adiante na malha.

Quando a referência pedida excede o que o processo admite, segui-la pode causar danos ao equipamento ou tornar o modelo de controle inválido. Assim, o seletor corta a referência no limiar, e o controlador nunca recebe um pedido inexecutável. Em compensação, enquanto o limite atua, o torque entregue difere do torque pedido, e essa divergência é deliberada. A malha passa a regular o processo contra o limiar transferível, não contra a referência, e o erro persistente entre ambos não é falha de controle, mas a proteção em exercício. Para que o controlador não persiga a referência inalcançável, é a referência efetiva, já cortada, a que alimenta os dois caminhos da estrutura.

No projeto, o limiar é o torque máximo transferível ao solo $T_{\text{aderência}}$, e o ponto de inserção é a entrada da malha, antes do *feedforward* e da realimentação, de modo que a referência efetiva, já cortada, alimenta os dois caminhos da estrutura, sendo convertida na corrente esperada no *feedforward* e servindo de termo de comparação do torque estimado na realimentação. Com isso, a roda não patina, pois o torque comandado nunca excede o transferível, e o modelo do propulsor permanece válido, pois só descreve o processo abaixo do limite de aderência, de forma que o seletor protege, logo na entrada da estrutura, tanto a aderência quanto a validade do modelo.

2.3.2 O *feedforward*

A pré-alimentação da referência, ou *feedforward*, é o bloco que converte a referência diretamente em uma parcela da ação de controle, sem esperar que o erro se manifeste

[6]. O fundamento é a divisão de esforço entre os dois caminhos da estrutura, assim, o bloco de pré-alimentação antecipa a resposta à referência e o bloco de realimentação corrige o que a pré-alimentação não previu. Para que a referência se converta em ação de controle sem passar pelo erro, o caminho que ela atravessa precisa ser o inverso do modelo da malha

$$C_p(s) = \frac{1}{H_1(s)}, \tag{2.9}$$

em que $H_1(s)$ é o modelo que a referência atravessa. No projeto, o controlador C_{ff} recebe a referência efetiva e fornece a corrente esperada I_{ff}, que se soma à correção ΔI do mestre para formar a referência da malha interna I_{ref}. A realimentação, por sua vez, atua depois que o desvio aparece.

Sem a pré-alimentação, o controlador dependeria apenas da realimentação, e o termo integrador carregaria sozinho todo o esforço de regime, deteriorando o transiente. A limitação da estrutura é a dependência do modelo. Quanto mais o processo real se afasta de $H_1(s)$, maior o resíduo que recai sobre a realimentação. A pré-alimentação da perturbação, possível apenas quando a perturbação é medida ou estimada, não é empregada, pois nem o afundamento da tensão da bateria sob carga nem a variação térmica de I_0 são mensuráveis no veículo.

No projeto, o *feedforward* é a inversa da curva característica. Aplica-se a Equação (2.7) no sentido da referência, o que fornece a corrente esperada

$$I_{\text{ff}} = I_0(u) + \frac{u\,T_{\text{ref,ef}}}{K_T}, \tag{2.10}$$

e o comando equivalente u_{ff} é o valor que resolve essa equação. A saída do controlador mestre, a correção de corrente ΔI, responde pelo que a pré-alimentação não previu, e o somador forma a referência da malha interna $I_{\text{ref}} = I_{\text{ff}} + \Delta I$. Dessa forma, a pré-alimentação age antes do erro e a realimentação corrige o que resta.

2.3.3 A cascata

A estrutura em cascata organiza duas malhas aninhadas. O controlador mestre compara a referência com a variável controlada primária, e sua saída é a referência do controlador escravo, que compara essa referência com a variável secundária [5]. Cada malha obedece à lei de controle proporcional-integral, em que o erro $e = R - Y$ alimenta

$$u(t) = K_c \left(e(t) + \frac{1}{T_i} \int_0^t e(\tau)\, d\tau \right), \tag{2.11}$$

em que K_c é o ganho e T_i o tempo integral do controlador [7]. A justificativa da

cascata está na rejeição da perturbação interna. Na estrutura, a perturbação L_2 atua dentro da malha escrava, e a função de transferência entre ela e a variável secundária é

$$\frac{I}{L_2} = \frac{G_{L2}}{1 + C_2\, G_2\, G_{m2}}, \tag{2.12}$$

em que G_{L2} é o caminho da perturbação até a corrente, C_2 o controlador escravo, G_2 a planta interna comando–corrente e G_{m2} o sensor de corrente que realimenta a malha. Com o ganho de malha grande na banda de frequência da perturbação, o denominador reduz o efeito de L_2 antes que ela se transforme em erro na malha externa. Para que as duas malhas não disputem o mesmo atuador, a regra de separação de velocidades exige que a interna seja de três a dez vezes mais rápida que a externa.

No projeto, o mestre é o controlador de torque, e o escravo, o controlador de corrente, mais rápido e sensível às perturbações elétricas. A planta interna $G_2(s)$ é a dinâmica comando–corrente, e a malha interna rejeita, pela Equação (2.12), as perturbações elétricas antes que se tornem erro de torque, enquanto a malha externa cuida do seguimento da referência.

2.3.4 O controle inferencial

No controle inferencial, a variável controlada não pode ser medida on-line e é reconstruída por um modelo do processo a partir de uma variável secundária, medida e sensível a ela [5]. No projeto, o comando u alimenta a planta $G_2(s)$, cuja saída primária, o torque T, não é mensurável embarcada. A corrente I, variável secundária medida, e o próprio comando alimentam o modelo $\hat{G}$, que reconstrói o torque $\hat{T}$. O fundamento da decisão é a indisponibilidade de sensor para a variável primária. Sem instrumentação para ela, o modelo é o único caminho de realimentação disponível, e o desempenho de toda a estratégia depende da precisão desse modelo.

No projeto, o torque na roda é a saída primária, e a corrente do ramo de alimentação é a medida secundária. O modelo é a curva característica validada no ensaio de tração de corrente versus força, sintetizada pela Equação (2.7). O estimador de torque fecha o laço externo: limita a estimativa à faixa validada e filtra-a antes de alimentar o mestre, pois é quase-estático e não deve reagir a transientes em que corrente e comando se desbalanceiam. O estimador, portanto, concentra em si a precisão do controle de torque.

2.3.5 As proteções do comando

O comando do ESC satura em dois pontos do seu curso, sendo um a zona morta, abaixo da qual o motor não acelera, e o outro a fronteira da região validada. Quando o atuador satura, o erro continua alimentando o integrador do controlador, que acumula até a saída ficar presa no limite mesmo após a reversão do erro. O tratamento clássico dessa condição é o condicionamento da ação integral, também chamado de anti-*windup* [7]. No condicionamento, a saída linear u_{lin} passa pela saturação σ, e o comando efetivo u retorna ao controlador, e o integrador não acumula o erro em excesso enquanto a saturação persiste.

No projeto emprega-se o *clamping* no integrador do controlador escravo, ajustado para que a saída fique exatamente no limite, em vez de simplesmente congelar. Com isso, qualquer reversão do erro tira a saída da saturação imediatamente. Duas proteções completam a estrutura sobre o comando, sendo a primeira a limitação à região de aderência validada, fronteira que mantém o modelo, o estimador e o controlador válidos. A segunda é a proteção contra sobrecorrente por limiar, que reduz a excitação sempre que a leitura ultrapassa o valor máximo admitido, o que só é possível porque a corrente é medida.

2.3.6 A classificação das variáveis

Na classificação das variáveis de processo, o controle de torque deste propulsor segue o perfil do controle de vazão, de dinâmica rápida com tempo morto desprezível, perturbações frequentes porém pequenas e ruído de alta frequência na medição [8]. A variável controlada primária é o torque na roda, que não é mensurável embarcada, e a variável controlada secundária é a corrente de bateria, lida pelo sensor. A variável manipulada é o comando u, e as perturbações são o afundamento de tensão da bateria sob carga e a variação da corrente de perdas com a temperatura, ambas sem sensor no veículo e ambas grandezas de influência clássicas, que alteram a medição e o processo sem serem o objeto do controle [9].

2.3.7 A implementação digital

A teoria do projeto é contínua, ao passo que a implementação é necessariamente digital, pois o laço passa integralmente por computadores. A corrente é amostrada pelo conversor do Arduino, o controlador executa como programa no computador de bordo e o comando segue por pulso digital ao ESC. O controlador contínuo é, assim, traduzido em equações de diferenças executadas a cada período de amostragem T_s, e o projeto permanece válido se T_s for pequeno frente às dinâmicas controladas, conforme

as regras clássicas de discretização [10]. Adota-se $T_s = 10$ ms, o regime do *firmware* rápido já gravado no Arduino, menor que um décimo dos tempos integrais de ambas as malhas.

As duas malhas se discretizam da mesma forma. A escrava executa, a cada amostra,

$$u[k] = u_{\text{ff}}[k] + K_{p2}\left(e_2[k] + \frac{T_s}{T_{i2}}\sum_j e_2[j]\right),$$

e o mestre,

$$I_1[k] = K_{p1}(u[k])\left(e_1[k] + \frac{T_s}{T_{i1}}\sum_j e_1[j]\right),$$

com as somas dos erros substituídas, na prática, por acumuladores atualizados a cada amostra. O filtro do estimador e as rampas da pré-alimentação e da referência são recursões de primeira ordem idênticas, todas no mesmo relógio de 10 ms. Cada elemento da malha corresponde a um bloco do script da simulação (Apêndice A.3), que é a versão de referência da implementação, e o mapa entre elementos e linhas de código consolida-se na Tabela 2.1. A Figura 2.7 resume a ordem de execução dentro de cada amostra, na qual o estimador, o seletor, o mestre, o *feedforward* e o escravo se sucedem antes de o comando seguir à planta.

Tabela 2.1: Implementação da malha: onde cada elemento vive no código (script A.3).

Elemento	Localização no script
Constantes do projeto (λ, T_s, limites, rampas, filtro)	linhas 49–62
Corrente de perdas $I_0(u)$ e comando equivalente	linhas 108–113 e 150–157
Ganhos IMC (K_{p2} e K_{p1} programado)	linhas 175–177
Estimador de torque, com teto e filtro	linhas 213–217
Seletor de aderência e rampa da referência	linhas 220–223
Feedforward de corrente e caminho direto	linhas 230–236
PI escravo, com anti-windup por clamping	linhas 238–248
Limite do comando à região de aderência	linhas 241 e 56–57

Duas escolhas de implementação condicionam o desempenho, e a primeira delas é o *feedforward* duplo, no qual, além da corrente esperada, a inversa da curva fornece o comando equivalente u_{ff}, e o PI escravo atua apenas sobre o desvio. Sem esse pré-posicionamento, o PI precisaria acumular integral para segurar qualquer comando diferente de zero, e o transiente destruiria o estimador, como revelou a simulação. A segunda é o anti-*windup* por *clamping* (linhas 243–245 do script A.3), o condicionamento da ação integral descrito na Seção 2.3.5, cuja alternativa mais simples, a integração condicional, travou o sistema em saturação nas simulações iniciais.

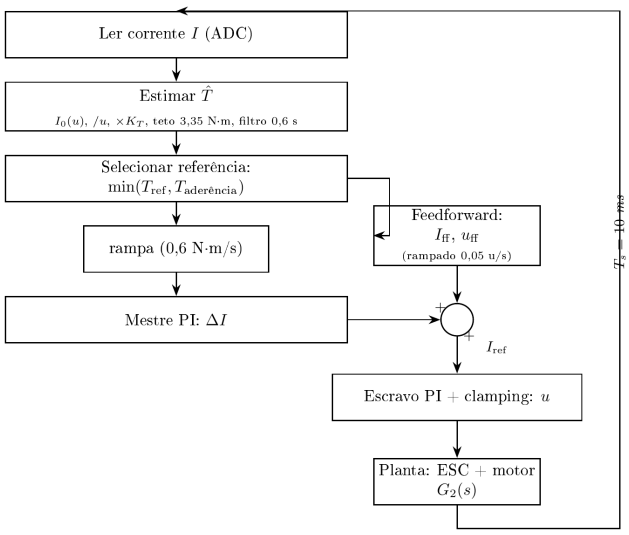


Figura 2.7: Ordem de execução dentro de cada amostra de 10 ms: a corrente lida alimenta o estimador, a referência é cortada no seletor e rampada no caminho do mestre, e mestre, *feedforward* e escravo se sucedem antes de o comando seguir à planta.
Fonte: elaborada pela autora (2026).

Sobre a malha externa, duas salvaguardas se aplicam, pois o estimador é quase-estático e só é válido onde foi caracterizado. Em transientes em que corrente e comando desbalanceiam, ele produz valores fisicamente impossíveis, e sua saída é limitada à faixa validada. O teto, 3,35 N·m, é o torque que o próprio estimador atinge no topo da região de aderência, e a estimativa é, ainda, filtrada por 0,6 s, constante de tempo escolhida em simulação para rejeitar o ruído da divisão sem atrasar a malha. A referência que entra no mestre é rampada a 0,6 N·m/s. O filtro da especificação de malha fechada no método de modelo interno já filtra a referência [11], e a rampa é sua versão prática, que evita que degraus grandes de referência carreguem o integrador do mestre. A rampa está no caminho da referência, não da realimentação, e a rejeição de perturbação não fica mais lenta por causa dela. Por fim, o próprio comando é limitado à região de aderência validada, $u \in [0{,}045;\ 0{,}21]$, que corresponde, pela normalização da metodologia, aproximadamente aos comandos de 99° a 113°. O limite inferior tira a malha da zona morta, e o superior a mantém na fronteira da aderência. A simulação mostrou esse limite ser necessário, pois a curva de torque do propulsor não é monotônica além dessa região e comandar nela inverte o sinal da planta e desestabiliza a malha externa. O limite não é apenas proteção, mas a fronteira que mantém o modelo, o estimador e o controlador válidos.

O conjunto dessas decisões atende diretamente aos objetivos do trabalho. A estrutura controla o torque em malha fechada, com seguimento de referência e rejeição de perturbações, sobre um propulsor caracterizado experimentalmente, operando dentro

dos limites em que o modelo é válido e protegido contra sobrecorrente e patinagem. Com a estrutura, o projeto dos controladores e a implementação digital, a proposta do sistema de controle fica completamente especificada, e a sua materialização experimental é descrita na Metodologia, Capítulo 3.

2.4 Resumo do Capítulo

Este capítulo apresentou o sistema de propulsão do automodelo e comparou as principais grandezas mensuráveis para sua caracterização, a saber, aceleração, velocidade angular, corrente elétrica e força de tração. Cada uma se associa ao torque por uma cadeia própria, descrita pelas Equações (2.1) e (2.2). A comparação evidenciou a corrente elétrica como a grandeza de cadeia mais curta até o torque e o sensor de efeito Hall linear ACS758 como a melhor opção entre os sensores avaliados.

Sobre essa escolha, definiu-se a proposta do trabalho. Ela é um estimador de torque a partir da corrente medida, validado contra um padrão de tração obtido com uma balança de gancho. A malha de controle em cascata, com *feedforward*, anti-windup e as proteções de aderência e de sobrecorrente, estrutura-se para atender aos objetivos de controle em malha fechada do propulsor. Com a classificação das variáveis, o projeto dos controladores por modelo interno e a implementação digital, a proposta fica completamente especificada. A materialização experimental dessa proposta – a montagem, os ensaios e o orçamento de incertezas – é descrita na Metodologia, Capítulo 3.

Capítulo 3

Metodologia

Este capítulo descreve os métodos experimentais empregados na caracterização do sistema de propulsão do automodelo. O capítulo inicia com o método de ajuste das retas comum a todos os ensaios, segue pela cadeia de aquisição, pelo teste em bancada e pelos ensaios com o veículo e finaliza com o orçamento de incertezas.

3.1 O Processamento dos Dados e Ajuste das Retas

Todos os ajustes dos ensaios são retas obtidas pelo método dos mínimos quadrados, o mesmo procedimento empregado na calibração de instrumentos [12]. O método escolhe os coeficientes que minimizam a soma dos quadrados dos resíduos, isto é, das diferenças entre cada valor medido e o valor que a reta prevê para ele. Para a reta $y = a + b\,x$, a soma $S = \sum_i (y_i - a - b\,x_i)^2$ é mínima quando suas derivadas em relação a a e a b se anulam, condição que fornece a solução fechada

$$b = \frac{\sum_i (x_i - \bar{x})(y_i - \bar{y})}{\sum_i (x_i - \bar{x})^2}, \qquad a = \bar{y} - b\,\bar{x}, \tag{3.1}$$

em que $\bar{x}$ e $\bar{y}$ são as médias dos dados.

A função `minimos_quadrados` do script `analise_estatica.py` (linhas 53–60), implementa a Equação (3.1) retornando os dois coeficientes e o índice de determinação. Para a reta forçada pela origem, $y = g\,x$, minimiza-se a mesma soma apenas em g, o que resulta em

$$g = \frac{\sum_i x_i\,y_i}{\sum_i x_i^2}. \tag{3.2}$$

A Equação (3.2) é implementada pela função `minimos_quadrados_origem` (linhas 63–68 do mesmo script), e a curva característica a emprega por meio da função `ajuste_gf` (linhas 104–114), que subtrai a corrente de perdas e divide pelo comando antes do

ajuste, linearizando a característica de modo que a reta forçada pela origem tenha sentido físico.

A qualidade de cada ajuste é quantificada pelo coeficiente de determinação,

$$r^2 = 1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \bar{y})^2}, \tag{3.3}$$

fração da variância dos dados, medida em torno de sua média $\bar{y}$, que a reta explica, e é calculado nas duas funções de ajuste (linhas 59 e 67).

Os tratamentos foram implementados em scripts dedicados, que leem os arquivos de dados brutos e geram as figuras e tabelas do Capítulo 4, e a divisão entre eles segue os dois modelos que o controlador consome. O objetivo do primeiro é o modelo estático, e o `analise_estatica.py` (Apêndice A.1) o constrói a partir da planilha de anotações do ensaio de tração de corrente versus força e do registro do ensaio em vazio, ajustando a corrente de perdas $I_0(u)$ e o coeficiente G_F da curva característica, e produz as figuras correspondentes. O objetivo do segundo é o modelo dinâmico, a planta $G_2(s)$, e o `analise_dinamica.py` (Apêndice A.2) o identifica a partir do ensaio de tração de corrente, seguindo o tratamento da Seção 3.5.6, com a constante de tempo ajustada pelos dois métodos e o ganho de cada borda computado como o quociente do platô pelo comando, e gera as duas figuras da dinâmica, as janelas brutas com a média móvel e as respostas alinhadas com o ajuste. O `simulacao_cascata.py` (Apêndice A.3) executa a malha de controle sobre os dois modelos e produz a figura de desempenho.

3.2 Aquisição e registro de dados

Descrita uma única vez, a cadeia de aquisição serve a todos os ensaios de degraus. A corrente é amostrada por um programa dedicado gravado no Arduino, o *firmware* do ensaio. A cada 10 ms, ele lê o conversor e transmite a amostra pela porta serial, o que resulta em uma taxa de aproximadamente 100 Hz. A escolha do período segue a regra usual de discretização, de amostrar dez a trinta vezes mais rápido que a dinâmica de interesse [10]. A resposta da corrente ao comando é governada pela constante de tempo do caminho comando–corrente, cuja expectativa a priori era a faixa de décimos de segundo. A identificação conduzida no ensaio de tração de corrente confirmou $\tau = 0{,}18$ s (Seção 4.5 do Capítulo 4), de modo que o período de 10 ms amostra essa dinâmica quase vinte vezes mais rápido que ela, acompanhando a subida com folga e registrando sua forma completa, o que viabiliza a identificação da dinâmica entre comando e corrente.

O acionamento dos degraus emprega o *script* `teste_throttle.py`, disponibilizado

pelo orientador, que executa no Raspberry Pi embarcado no veículo e é comandado remotamente via SSH pelo terminal do notebook, por `ssh alunos@raspberrypi.local "python3 teste_throttle.py <ângulo> <duração>"`. O *script* coloca o ESC em neutro, aplica o degrau pela duração programada e retorna ao neutro ao final. Adota-se aqui a palavra *degrau* no vocabulário de controle, no qual ela designa a função de entrada que salta de imediato para um valor constante, mantido por toda a duração programada, sendo a mesma entrada empregada na identificação clássica de modelos dinâmicos. O comando constante é um ângulo α, na convenção de rádio-controle, na faixa de 90° a 180°. Nela, 90° é o limite inferior de armamento do ESC, 180° é a aceleração máxima e 95° é o neutro, o comando para o qual o ESC mantém o motor parado. Embora a convenção do sinal varra 0° a 180°, os valores abaixo de 90° correspondem à região de ré do ESC e não são empregados no ensaio, que percorre somente a escala de tração adiante. Ao longo do trabalho, o comando é usado na forma normalizada,

$$u = \frac{\alpha - 95}{85}, \tag{3.4}$$

de forma que o neutro corresponda a $u = 0$ e a aceleração máxima corresponda a $u = 1$.

A automação do ensaio foi desenvolvida no âmbito deste trabalho (Apêndice A.4) e percorre a escala de degraus em um laço, substituindo a cada iteração o ângulo e a duração e disparando o comando SSH correspondente, enquanto lê a porta serial do Arduino e grava, em um arquivo CSV, o tempo de ensaio, o tempo do degrau, o ângulo e a corrente de cada amostra. No terminal, imprime-se somente a média das amostras de cada segundo, o tempo $t = 1, 2, \ldots, 15$ e a corrente média associada, linha que aparece no vídeo do ensaio ao lado do display da balança. As amostras completas são gravadas por experimento, em `ensaio_vazio_corrente.csv` para o ensaio em vazio e em `ensaio_tracao_corrente_versus_forca.xlsx` e `ensaio_tracao_corrente.csv` para os ensaios de tração das Seções 3.4 e 3.5, respectivamente.

3.3 Teste em Bancada

Antes de qualquer ensaio com o veículo, os instrumentos são validados em bancada, para que suas incertezas sejam conhecidas antes de entrarem na curva característica, já que um desvio sistemático de qualquer uma das grandezas se propagaria diretamente para o modelo do sistema usado na sintonia dos controladores.

O circuito de medição é isolado galvanicamente do circuito de potência. O polo positivo da fonte ajustável, capaz de fornecer de 0 a 6 A, alimenta o pino IP+ do ACS758, e o pino IP− retorna ao polo negativo da fonte, fechando o laço, com o

multímetro de referência, configurado como amperímetro, em série entre a fonte e o sensor. A alimentação do sensor provém dos pinos de 3,3 V e GND do Arduino, e sua saída analógica passa por um filtro RC, com $R = 1$ kΩ e $C = 100$ nF, cuja frequência de corte é de aproximadamente 1,6 kHz, até o pino A0. Nesse teste, o ESC e o motor não participam, já que o intuito é realizar os testes de funcionamento do sensor e caracterizar os seus desvios.

Atravessa o laço apenas o condutor interno do sensor. Antes de energizar, o checklist compreende o aperto dos bornes, a polaridade do multímetro, a faixa de corrente selecionada e a conexão da porta serial do Arduino ao notebook.

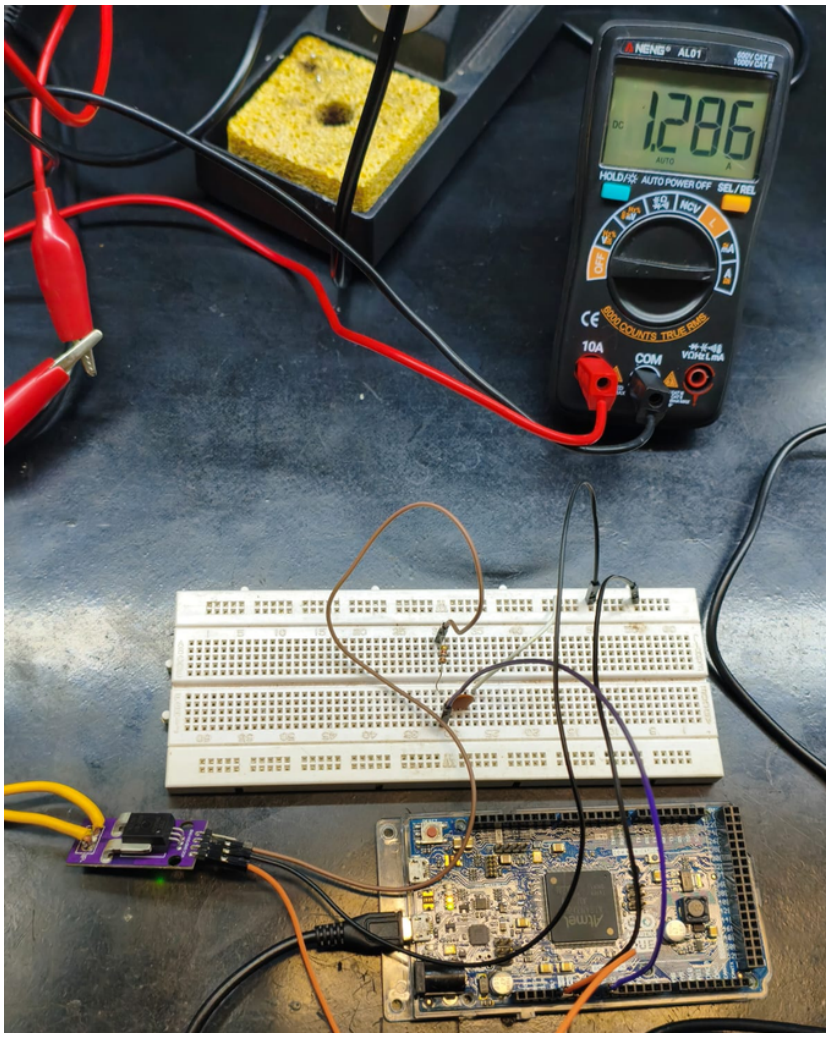


Figura 3.1: Teste em bancada da cadeia de corrente, com a fonte ajustável, o multímetro de referência e o sensor ACS758.
Fonte: elaborada pela autora (2026).

O procedimento de verificação segue os passos abaixo.

1. Ajustar um valor de corrente na fonte, distribuído pela faixa de 0 a 6 A.

2. Aguardar a estabilização da leitura do multímetro, tomada como referência.

3. Registrar o par de leituras, a do multímetro e a do sistema formado pelo sensor e pelo conversor, exibida no terminal do notebook.

4. Repetir os passos anteriores ao longo de toda a faixa, sem desmontar o laço.

Posteriormente, a instalação em série entre a bateria e o ESC resolve-se com um conector Deans-T em Y, usualmente empregado para conectar duas baterias. O lado vermelho do conector foi interrompido, e cada uma de suas extremidades foi soldada a um terminal de corrente do sensor, com termoretrátil envolto, conforme a Figura 3.2. A adaptação dispensa qualquer corte nos cabos originais do automodelo.

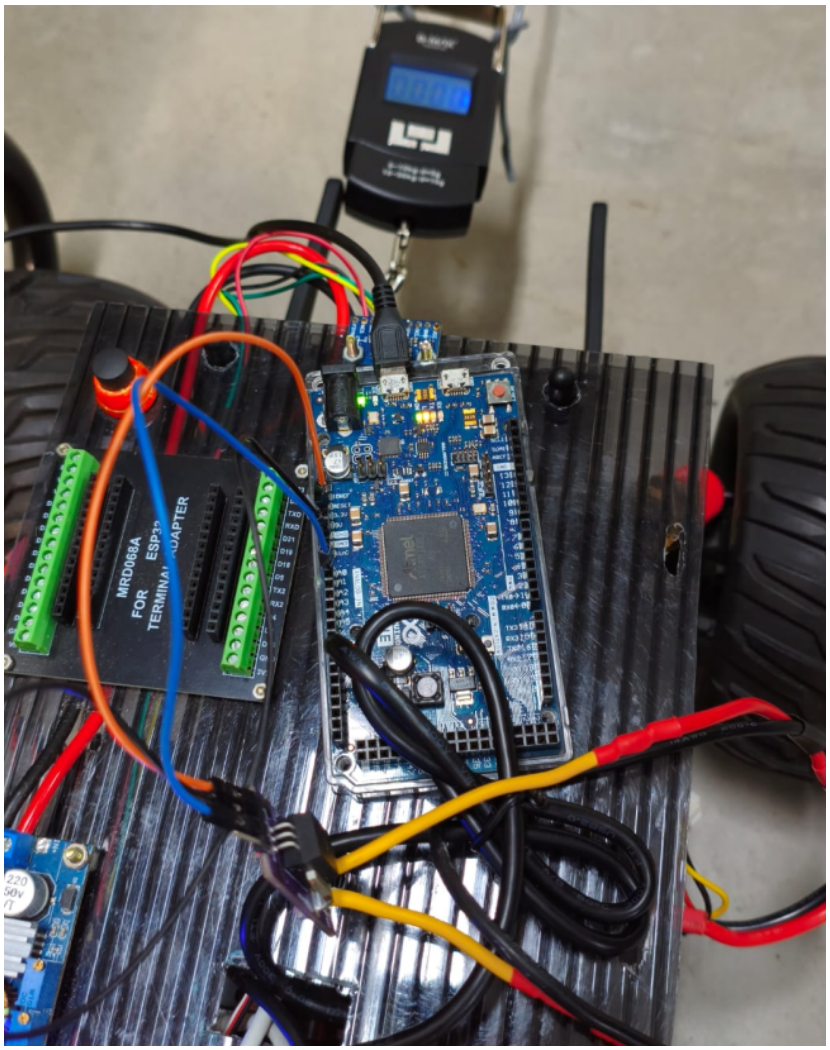


Figura 3.2: Sensor de corrente ACS758 instalado no ramo de alimentação, entre a bateria e o ESC, por adaptação com conector Deans-T.
Fonte: elaborada pela autora (2026).

Concluída a verificação em bancada, a mesma cadeia foi conferida na própria montagem do veículo, validando a adaptação do conector instalado. Com o veículo em bancada, o fio vermelho do ramo de potência foi interrompido e um multímetro configurado como amperímetro foi inserido em série, de forma que a linearidade do sensor pôde ser verificada na configuração definitiva. Encerrada a conferência, o fio foi soldado novamente. Como o automodelo já abriga diversos circuitos embarcados e o espaço de trabalho é limitado, empregam-se apenas o Arduino e o sensor, alimentados pelos 3,3 V e pelo terra do próprio Arduino.

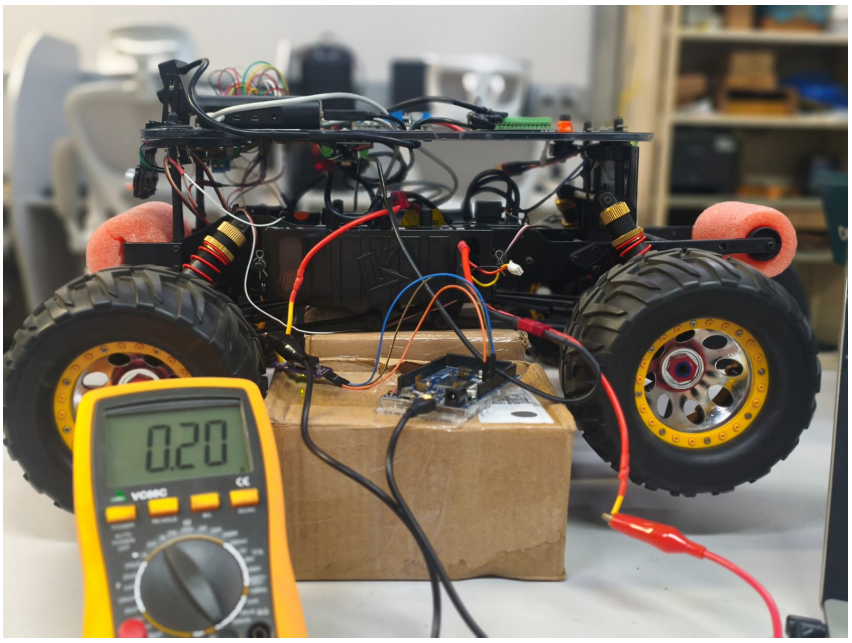


Figura 3.3: Sensor de corrente instalado no automodelo durante a conferência da cadeia, com o fio vermelho do ramo de potência interrompido para a inserção do multímetro em série.

Fonte: elaborada pela autora (2026).

O confronto entre as medidas de corrente segue o procedimento clássico de calibração estática, no qual a reta de ajuste entre os valores de referência e os indicados é obtida pelo método dos mínimos quadrados da Seção 3.1 [12]. Os resultados da verificação, a regressão da cadeia de corrente e o tratamento do desvio são apresentados na Seção 4.1.

3.4 Ensaio em Vazio

O primeiro ensaio com o veículo é o ensaio em vazio, que mede a corrente consumida pelo conjunto propulsor quando ele não entrega tração alguma, com as rodas suspensas. Sua função decorre da forma como a corrente entra na malha de controle,

pois o estimador desconta as perdas da corrente medida, conforme a Equação (2.7), e o *feedforward* converte o torque pedido na corrente esperada, conforme a Equação (2.10). Sem o ensaio em vazio, $I_0(u)$ seria desconhecido, o estimador absorveria as perdas como se fossem tração e o modelo ficaria enviesado principalmente nos degraus baixos, nos quais a fração de perdas é maior.

O estimador representa a corrente de fase pelo quociente I/u, o que só é legítimo se o *duty cycle* do ESC acompanhar o comando de forma linear. O ensaio em vazio valida essa hipótese, que está por trás da divisão por u da Equação (2.5). Com as rodas suspensas, a corrente medida em cada degrau é a própria corrente de perdas $I_0(u)$, ou seja, a Equação (2.5) com $I_{\text{bat}} = I_0$. Se o *duty cycle* for de fato proporcional ao comando, essa corrente deve crescer linearmente com u, de modo que a linearidade de $I_0(u)$ observada nos resultados é a evidência da hipótese. O ajuste linear e o coeficiente de determinação que quantificam essa evidência são apresentados no Capítulo 4.

O veículo é apoiado sobre uma caixa, com as rodas motrizes suspensas, sem contato com a mesa, conforme a Figura 3.4, e a cadeia de aquisição é empregada sem alterações. Antes da sessão, verificam-se as condições comuns a todo ensaio com o veículo, isto é, bateria LiPo 2S carregada e em bom estado, bateria do Raspberry Pi carregada, com a tensão de saída do *step-down* conferida, cabo do Arduino conectado ao notebook para alimentação e registro, e notebook conectado à rede do Raspberry, com o acesso remoto por `ssh` funcionando.



Figura 3.4: Ensaio em vazio, rodas motrizes suspensas em caixa, sem contato com a mesa.

Fonte: elaborada pela autora (2026).

O procedimento da sessão é padronizado em quatro passos, do posicionamento do veículo à conferência do arquivo registrado.

1. Posicionar o veículo sobre a caixa e conferir o afastamento das rodas em relação à mesa.
2. Energizar o conjunto e a cadeia de aquisição, aguardando o armamento do ESC.
3. Disparar a varredura automatizada, que aplica a escala de degraus de 97° a 117° de grau em grau, com 15 s por degrau e 3 s de neutro entre degraus.
4. Ao final, encerrar o registro e conferir o arquivo `ensaio_vazio_corrente.csv`.

Para o tratamento dos dados de cada degrau, tomam-se as amostras em regime, isto é, após a subida transiente, e descartam-se as execuções com pico inferior a 1 A, nas quais o comando não foi efetivamente executado pelo ESC e que distorceriam a média para baixo. O script da caracterização estática (Apêndice A.1) agrupa então as amostras por comando, calcula a média de cada grupo e ajusta a reta $I_0(u)$ sobre essas médias por mínimos quadrados (Equação (3.1)). O resultado é a curva $I_0(u)$, parâmetro do estimador e do *feedforward* da malha, cujos valores são apresentados no Capítulo 4.

3.5 Ensaio de Tração de Corrente versus Força

Na estrutura de controle proposta, a corrente medida alimenta a malha interna e o torque estimado alimenta a malha externa, sendo o elo entre as duas a curva característica, que relaciona a corrente ao esforço mecânico do propulsor. O ensaio mede, portanto, em cada degrau, o par corrente–força com que essa curva é construída ponto a ponto.

3.5.1 A Montagem do Ensaio

Com o automodelo ancorado, o pneu é impedido de rolar, e a força que o propulsor trocaria com o solo na direção do movimento desvia-se integralmente para a estrutura de retenção. O cabo que une essa estrutura à balança converte o esforço em tensão mensurável.

Com a roda travada, o motor opera na condição de rotor bloqueado, na qual a exigência de torque é máxima, pois é a sua condição de partida.

O instrumento utilizado para medir a força de tração é uma balança de gancho, cuja indicação é expressa em quilogramas. Quando submetido à tração, o instrumento

detecta a força aplicada e a apresenta como uma massa equivalente, correspondente à força-peso que produziria a mesma indicação sob a aceleração da gravidade de referência. Essa equivalência permite converter a leitura em força, expressa em newtons, por meio de

$$F\,[\mathrm{N}] = 9{,}81\, m_{\mathrm{lei}}\,[\mathrm{kg}],$$

em que m_{lei} representa o valor indicado pela balança.

A verificação da balança tem objetivo diferente, pois a balança é a própria referência da força e não dispõe de canal de correção, uma vez que seu display não tem saída elétrica. Ela é confrontada com pesos padrão de 0,500 kg, 1,000 kg e 1,500 kg aplicados sucessivamente ao gancho, e o desvio verificado é declarado como componente de incerteza, conforme a Seção 3.7.

3.5.2 Ancoragem e Geometria do Cabo

A montagem experimental utiliza um cabo de 126 cm, passado ao redor de uma gaveta pesada e fixa, que atua como estrutura de ancoragem, atando cada extremidade da alça da balança, conforme a Figura 3.5. O gancho é acoplado a uma estrutura na parte traseira do automodelo, posicionada acima do eixo de transmissão das rodas, buscando manter a linha de tração alinhada à direção longitudinal do veículo para minimizar as componentes transversais da força. Durante o ensaio, a balança permanece suspensa a 15 cm do solo e o ponto de ancoragem na gaveta está a 28 cm. Todas as dimensões foram medidas três vezes, adotando-se a média aritmética como valor representativo.

Como os engates se encontram a alturas diferentes, o cabo, a linha horizontal e a diferença de altura formam um triângulo retângulo cuja hipotenusa é o próprio cabo. O cateto vertical, oposto ao ângulo θ entre o cabo e a horizontal, vale $28 - 15 = 13$ cm, assim:

$$\operatorname{sen}\theta = \frac{13}{126} \qquad \Rightarrow \qquad \theta = 5{,}9^{\circ},$$

o que resulta em $\cos\theta = 0{,}995$. Tomar a leitura da balança diretamente como força de tração equivale a aproximar $\cos\theta$ por 1. O erro dessa aproximação é quantificado como componente de incerteza na Seção 3.7.

3.5.3 Massa Adicional

Nos ensaios de tração, fixam-se halteres na dianteira do veículo, conforme ilustrado na Figura 3.6, de modo a aumentar a carga normal sobre os pneus e, com ela, o limite

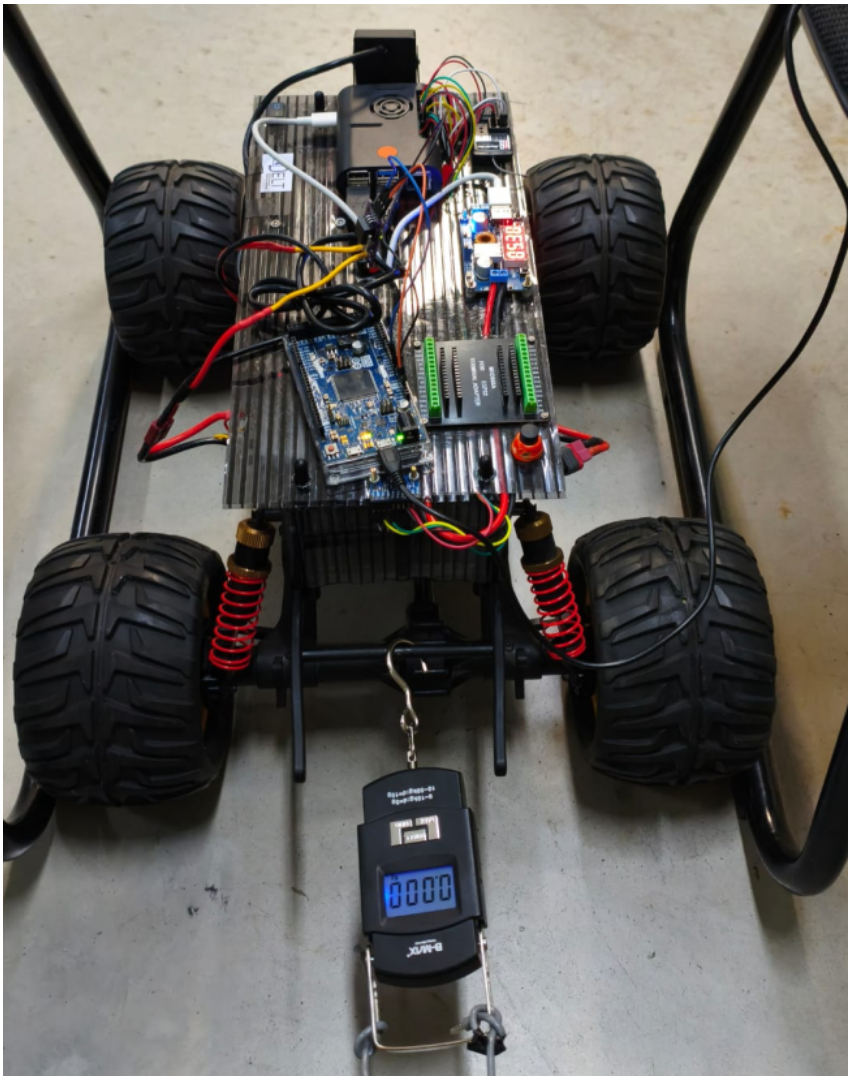


Figura 3.5: Montagem do ensaio de tração, automodelo ancorado pela balança de gancho suspensa entre o engate traseiro e a estrutura fixa.
Fonte: elaborada pela autora (2026).

de aderência antes da patinagem. O peso atua na vertical, ao passo que a balança mede na horizontal, assim, em regime estático, a leitura corresponde à tração desenvolvida pelo propulsor na região de aderência. A massa total do conjunto, veículo, baterias e halteres, foi de 6,640 kg, média de três pesagens, adotada como ponto de operação do projeto, pois variações de massa alteram o limite de aderência e comprometem a comparabilidade entre sessões.

3.5.4 Raio Efetivo da Roda

O torque da roda depende do produto da força pelo raio efetivo de rolamento sob carga, pois o pneu se deforma sob o peso do conjunto, apresentando um raio menor que o



Figura 3.6: Halteres fixados na dianteira do veículo para aumentar a carga normal sobre os pneus.

Fonte: elaborada pela autora (2026).

nominal. A medida segue a definição normativa do perímetro efetivo dos pneumáticos das rodas, a média das distâncias percorridas pelas rodas motrizes em uma rotação completa [13], obtida aqui pelo método de *roll-out*, com marcação de referenciais no pneu e no solo e contagem de voltas completas. Com o veículo encostado em uma parede, para garantir o deslocamento em linha reta, marca-se o pneu e o solo em um mesmo alinhamento e movimenta-se o automodelo até completar cinco voltas da roda. A distância percorrida entre as marcas, anotada em três medições independentes, é de 236 cm em média, e o raio efetivo segue da Equação (3.5):

$$r_{\mathrm{ef}} = \frac{L}{2\pi n_v} = \frac{236}{2\pi \times 5} = 7{,}51 \text{ cm} = 0{,}0751 \text{ m}, \tag{3.5}$$

em que $n_v = 5$ é o número de voltas completas.

A cadeia mecânica da medida fecha com a Equação (3.6):

$$T_{\mathrm{roda}} = F\, r_{\mathrm{ef}}. \tag{3.6}$$

A medida do raio existe porque o controlador trabalha com torque, e não com força, e a conversão da Equação (3.6) é o elo que leva o coeficiente da curva característica, medido em newtons, ao coeficiente de torque que a malha emprega, $K_T = G_F\, r_{\mathrm{ef}}$, aplicado no estimador e no *feedforward*.

3.5.5 O Procedimento do Ensaio

O experimento central do trabalho é o ensaio de tração de corrente versus força, no qual o veículo ancorado recebe degraus de comando que fornecem um par de valores, a corrente consumida e a força de tração, que compõem a curva característica ponto a ponto.

A configuração segue a montagem da Seção 3.5.2, com o veículo ancorado pela balança, os halteres fixados e a massa total conferida por pesagem (Seção 3.5.3). Uma câmera registra a sessão, com o display da balança e o terminal do notebook no mesmo plano, este exibindo o horário, o tempo dentro do degrau e a corrente média do segundo. Antes da sessão, repetem-se as verificações comuns do ensaio em vazio e acrescentam-se a taragem da balança em vazio, a conferência do cabo e dos engates e o posicionamento da câmera. A balança é tarada nos 3 s de neutro entre um degrau e o seguinte, pois, durante a aceleração, parte da força de tração não aparece como tração instantânea no cabo, uma vez que se acumula como energia de deformação elástica no conjunto cabo–engates–estrutura. Ao cessar o comando, essa tensão elástica se relaxa e deixa no display um valor residual de 0,1 a 0,3 kg que não corresponde a esforço algum do propulsor, o que torna necessária a taragem a cada neutro.

O protocolo de cada degrau segue os passos abaixo e é o mesmo em toda a escala.

1. Tarar a balança durante o neutro entre degraus.
2. Executar o script remoto do degrau, de duração de 15 s.
3. Decorridos 7 s, anotar a força exibida em cada um dos segundos 7, 8, 9 e 10, em regime estacionário.
4. Adotar como ponto da curva a média das quatro leituras.

A corrente, por sua vez, é registrada durante todo o ensaio, e empregam-se os pontos por segundo impressos no terminal, as médias de cada segundo, que representam o regime das medições, relacionados em vídeo, quadro a quadro, à força do mesmo degrau e do mesmo segundo. A leitura em vídeo padroniza o processo, elimina os erros de pressa e de paralaxe da leitura ao vivo e dá rastreabilidade a cada valor anotado. Como o display da balança não possui saída elétrica, conforme discutido na Seção 3.5.1, o vídeo é o canal de registro da força. O ensaio fornece a curva característica e todos os parâmetros que dela derivam, apresentados no Capítulo 4.

3.5.6 O Tratamento dos Dados

O tratamento consolida, para cada comando, as quatro leituras dos segundos 7 a 10 em duas estatísticas, a mediana e o desvio-padrão amostral (Equação (3.7)), e converte as forças de kgf para N com $g = 9{,}81$ m/s^2, consolidando a Tabela mestra dos resultados. Comandos não executados em uma sessão, como o de 98° da descida, são excluídos do consolidado e assinalados na tabela.

Cada força anotada precisa ser pareada à corrente do mesmo degrau e do mesmo segundo, e o pareamento se valida contra o ensaio de tração de corrente (Seção 3.5.6), conduzido no mesmo protocolo e na mesma escala. A corrente anotada por vídeo no ensaio com balança e a corrente registrada eletronicamente nesse segundo ensaio são a mesma variável medida por dois canais independentes, e a coincidência entre as medianas comando a comando, dentro da dispersão entre repetições, comprova que a leitura pareada a cada força é representativa da condição de regime.

O coeficiente G_F da curva característica ajusta-se sobre os pontos individuais das repetições, não sobre as medianas, pois cada leitura fornece um par cuja abscissa é a corrente de fase equivalente, $(I - I_0(u))/u$, com $I_0(u)$ dado pelo ajuste do ensaio em vazio e u pela normalização da Equação (3.4), e cuja ordenada é a força convertida. A reta é forçada pela origem e ajustada pela função `ajuste_gf` (Apêndice A.1), que primeiro subtrai a corrente de perdas e divide pelo comando, linearizando a característica antes de aplicar a Equação (3.2), conforme descrito na Seção 3.1.

Ensaio de Tração de Corrente

A identificação da dinâmica entre o comando e a corrente, que define a planta $G_2(s)$ sobre a qual a malha escrava é projetada, é conduzida no ensaio de tração de corrente. Emprega-se o mesmo protocolo de degraus e a mesma montagem do ensaio de tração de corrente versus força, dispensada a anotação de força na balança, com a corrente amostrada a 100 Hz e registrada por inteiro em cada degrau, amostra a amostra, no arquivo `ensaio_tracao_corrente.csv`. O registro completo de cada platô viabiliza o ajuste da resposta ao degrau apresentado no Capítulo 4.

O script da dinâmica (Apêndice A.2) isola a sessão de aquisição rápida do arquivo bruto e segmenta o registro em blocos, um por comando, e os blocos não começam no instante do comando, pois a janela de registro abre antes de o comando remoto produzir efeito no ESC. Os primeiros 1,5 a 2 s de cada bloco registram a corrente em nível de repouso, compatível com a corrente de perdas I_0 do ensaio em vazio, e somente após esse período a corrente sobe, de forma que tratar o bloco inteiro como resposta ao degrau conduziria a uma constante de tempo superestimada, mistura do

atraso do protocolo com a dinâmica da planta. O script define o nível de repouso como a média dos primeiros 1,5 s do bloco, o platô como a média dos últimos 3 s e o início elétrico como a primeira amostra acima de 20% da excursão entre ambos, e a partir dele guarda 6 s de resposta, normalizados entre repouso e platô. Sobre a média das dez bordas da região de aderência, os comandos de 100° a 109°, empregam-se dois métodos, o da 63,2%, em que o tempo que a resposta leva para atingir 63,2% do valor final é, pela definição de constante de tempo, o próprio τ, e o ajuste de mínimos quadrados ao modelo de primeira ordem, procurando o τ que minimiza a soma dos quadrados dos resíduos entre a média e a curva $1 - e^{-t/\tau}$, conforme a caracterização dinâmica clássica por resposta ao degrau [14]. Bordas acima de 110° não são empregadas, pois a resposta já nasce contaminada pela patinagem. O ganho de cada borda é o quociente do platô pela corrente pelo comando respectivo, e o ganho da planta é a média desses quocientes sobre as dez bordas.

3.6 A Malha de Controle no Código da Simulação

A malha projetada no Capítulo 2 é implementada como um programa que executa um passo por amostra de 10 ms, e o fundamento da discretização está na Seção 2.3.7. O script da simulação (Apêndice A.3) é a versão de referência dessa implementação, e cada elemento é apresentado a seguir na ordem em que executa, com o fluxograma do seu código e as particularidades da implementação digital, remetendo ao Capítulo 2 para a teoria de cada bloco.

3.6.1 O Estimador de Torque

O estimador implementa a Equação (2.7) com as salvaguardas projetadas na Seção 2.3.4, e o código registra dois cuidados que a descrição contínua não carrega, o comando efetivo protegido da zona morta, que guarda a divisão, e o deslocamento de perturbação descontado junto às perdas. A Figura 3.7 apresenta o fluxograma do bloco.

3.6.2 O Seletor e a Rampa da Referência

O seletor implementa a Equação (2.8), e o código faz a distinção que a descrição contínua não carrega, pois a referência efetiva alimenta o *feedforward* antes da rampa e o mestre depois dela, avançando um passo de 0,6 N·m/s por amostra, conforme a Seção 2.3.7. A Figura 3.8 apresenta o fluxograma do bloco, com a bifurcação.

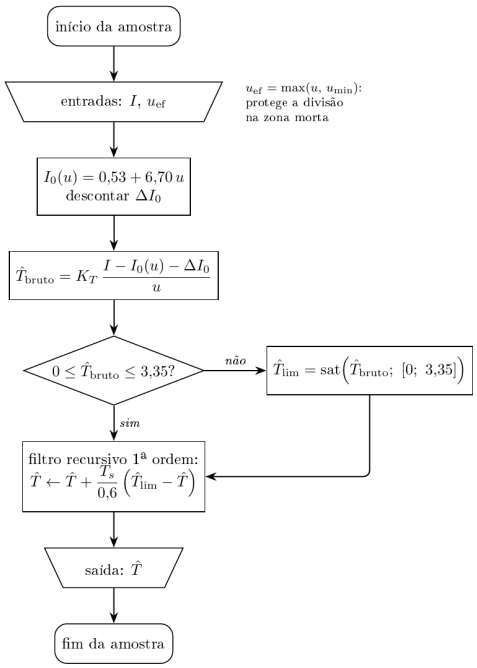


Figura 3.7: Fluxograma do estimador de torque a cada amostra: cálculo da estimativa bruta, limite à faixa validada e filtro de primeira ordem.
Fonte: elaborada pela autora (2026).

3.6.3 O Controlador Mestre

O mestre compara a referência rampada ao torque estimado e emprega a lei PI com ganho programado, $K_{p1}(u) = \lambda_2\, u/(K_T\, \lambda_1)$, que atualiza o ganho com o comando vigente e lineariza o caminho da malha externa. O integrador acumula o erro apenas quando o escravo não satura, pois, do contrário, a saturação do comando carregaria integral espúria para a malha de torque. A Figura 3.9 apresenta o fluxograma do bloco.

3.6.4 O Feedforward e o Caminho Direto

O *feedforward* converte a referência efetiva na corrente esperada pela Equação (2.10), $I_{ff} = I_0(u) + u\, T_{ref,ef}/K_T$, e o caminho direto obtém o comando equivalente u_{ff}, que é rampado com constante própria de 0,05 u/s. O somador forma a referência da malha interna, $I_{ref} = I_{ff} + \Delta I$, e o PI escravo atua apenas sobre o desvio, uma vez que o pré-posicionamento já entrega a maior parte do comando. A Figura 3.10 apresenta o fluxograma do bloco.

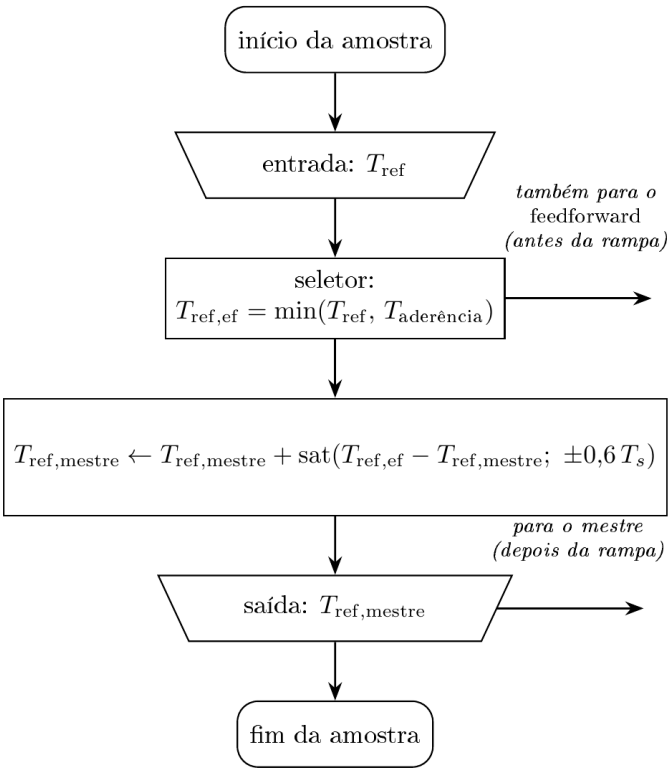


Figura 3.8: Fluxograma do seletor de mínimo e da rampa da referência, com a bifurcação para o *feedforward* (antes da rampa) e para o mestre (depois).
Fonte: elaborada pela autora (2026).

3.6.5 O Controlador Escravo e o Clamping

O escravo compara a referência de corrente à medida e emprega a lei PI, com o comando linear passando pela saturação nos limites $u_{\min}$ e $u_{\max}$ da região validada. Quando a saída satura, o *clamping* ajusta o integrador para que a saída fique exatamente no limite, e qualquer reversão do erro tira a saída da saturação imediatamente, tratamento descrito na Seção 2.3.5. Sem saturação, o integrador acumula normalmente e o mestre acumula junto. A Figura 3.11 apresenta o fluxograma do bloco.

3.7 Incertezas de Medição

O orçamento de incertezas segue o método das notas de aula de instrumentação [15]. Cada componente de incerteza é caracterizada como de Tipo A, quando avaliada estatisticamente a partir das repetições, ou de Tipo B, quando avaliada por propriedades do instrumento e do procedimento. As componentes, sendo independentes, se combinam em quadratura, isto é, pela raiz da soma dos quadrados de suas incertezas-padrão, resultando na incerteza combinada. A incerteza expandida é $U = k\,\sigma_c$, com

$k = 2$, que corresponde a aproximadamente 95% de confiança e delimita o intervalo dentro do qual o valor verdadeiro deve se situar. Para não confundir com o comando normalizado u, adota-se σ para as incertezas-padrão.

Tipo A. A repetibilidade do ensaio é quantificada pelo desvio-padrão amostral das quatro leituras de regime de cada degrau, anotadas nos segundos 7 a 10,

$$s = \sqrt{\frac{\sum_{i=1}^{n}(x_i - \bar{x})^2}{n-1}}, \qquad n = 4. \tag{3.7}$$

Emprega-se o denominador $n - 1$ porque a média $\bar{x}$ é estimada das próprias leituras, e essa correção torna o desvio-padrão amostral uma estimativa não enviesada da dispersão das medidas repetidas. São quatro leituras porque o protocolo adota os quatro segundos de regime como amostras da condição estacionária de cada degrau. O desvio é calculado sobre as forças de cada degrau, nas duas escadas, e, para caracterizar uma faixa de operação, adota-se a média dos desvios-padrão de seus degraus, o valor típico da repetibilidade na faixa. Os valores resultantes, por faixa, são apresentados com os resultados do orçamento no Capítulo 4.

Tipo B. Cada componente é modelada por uma distribuição retangular de meia-largura a, uma vez que o valor verdadeiro é conhecido apenas como pertencente a um intervalo, sem que posição alguma dentro dele seja mais provável que outra. O desvio-padrão dessa distribuição uniforme vale $\sigma = a/\sqrt{3}$, conforme as notas de aula, e é esse o valor empregado como incerteza-padrão. As componentes da força são as seguintes.

- **Resolução da balança,** a quantização imposta pelo display, que só distingue variações de 10 g, de modo que o valor verdadeiro pode estar até 5 g acima ou abaixo da indicação. A meia-largura é $a = 5$ g e

$$\sigma_{\text{res}} = \frac{5}{\sqrt{3}} = 2{,}9 \text{ g} = 0{,}028 \text{ N}.$$

- **Calibração da balança,** o desvio sistemático revelado pela verificação com pesos padrão. O desvio é a diferença entre a leitura e o peso padrão, relativa ao peso, o que resulta em

$$\frac{0{,}010 \text{ kg}}{0{,}500 \text{ kg}} = 0{,}02 = 2\%$$

da leitura na faixa do ensaio, sempre no mesmo sentido. Como não há canal de correção, o desvio não corrigido é tratado como retangular de meia-largura $a = 2\%$ de F

$$\sigma_{\text{cal}} = \frac{2\%}{\sqrt{3}} = 1{,}15\% \text{ de } F.$$

- **Leitura em vídeo,** a legibilidade quadro a quadro do display. Quadros sucessivos separam instantes discretos e o display pode mudar entre eles, de forma que, quando há dúvida de leitura, o valor é estimado em ± 1 dígito, $a = 10$ g,

$$\sigma_{\text{vid}} = \frac{10}{\sqrt{3}} = 5{,}8 \text{ g} = 0{,}057 \text{ N}.$$

- **Ângulo do cabo,** a aproximação da leitura direta, que toma $\cos\theta$ por 1 e cujo erro é multiplicativo sobre a força, pois a componente longitudinal da tensão é $F\cos\theta$. Da geometria da Seção 3.5.2, esse erro é

$$1 - \cos\theta = 1 - \sqrt{1 - (13/126)^2} = 0{,}53\%.$$

  Como não é corrigido, é tratado como retangular de meia-largura $a = 0{,}53\%$, e $\sigma_\theta = 0{,}53\%/\sqrt{3} = 0{,}31\%$ da força.

- **Raio efetivo,** a incerteza de marcação relativa à distância percorrida. Os 236 cm resultam de duas marcações posicionadas com fita, cuja resolução prática é de 1 cm, ou meia-largura relativa de $1/236 = 0{,}42\%$, tratada como retangular, $\sigma_r/r_{\text{ef}} = 0{,}42\%/\sqrt{3} = 0{,}24\%$.

As componentes se somam em quadratura porque são independentes e atuam de forma multiplicativa sobre o torque, de modo que seus efeitos não se acumulam linearmente. A Equação (3.8) segue diretamente de $T = F\,r_{\text{ef}}$ pela regra de propagação para produtos, pela qual as incertezas relativas das grandezas multiplicadas se combinam pela raiz da soma dos quadrados. A incerteza-padrão combinada do torque é, portanto,

$$\frac{\sigma_c(T)}{T} = \sqrt{\left(\frac{\sigma_A(F)}{F}\right)^2 + \left(\frac{\sigma_B(F)}{F}\right)^2 + \left(\frac{\sigma(r_{\text{ef}})}{r_{\text{ef}}}\right)^2 + \sigma_\theta^2}, \qquad \sigma_B(F) = \sqrt{\sigma_{\text{res}}^2 + \sigma_{\text{cal}}^2 + \sigma_{\text{vid}}^2}, \tag{3.8}$$

em que $\sigma_A(F)$ é o valor típico da repetibilidade da faixa e $T = F\,r_{\text{ef}}$.

O mesmo raciocínio envolve as variâncias dos ajustes. O coeficiente de determinação da Equação (3.3) mede a fração da variância dos dados explicada pela reta, e a variância residual que ele deixa visível é o retrato agregado da repetibilidade, pois os pontos de um comando carregam exatamente as flutuações que o Tipo A quantifica pela Equação (3.7). Quando a variância residual de um ajuste é da mesma ordem da dispersão entre repetições, o modelo explica o que era explicável, e o que resta é o acaso de medição, não falha de forma.

3.8 Resumo do Capítulo

Este capítulo construiu o ensaio da montagem ao orçamento de incertezas. A abertura fixou o método dos mínimos quadrados, comum a todos os ajustes do trabalho, e os scripts que o implementam. A balança de gancho foi adotada como padrão de força, e a montagem ancora o automodelo, convertendo a tração em tensão mensurável, com a geometria do cabo quantificada e o raio efetivo medido em 0,0751 m. A corrente é adquirida por um sensor de efeito Hall instalado por adaptação com conector Deans-T e registrada sob degraus gerados por *script* e automatizados por programa próprio. A verificação em bancada validou os instrumentos e quantificou seus desvios, distinguindo o que é corrigido em software do que é declarado como incerteza. O ensaio em vazio forneceu a corrente de perdas, parâmetro do estimador e do *feedforward*, e o ensaio de tração de corrente versus força, com a força lida em vídeo nos segundos de regime, construiu a curva característica da qual saem o coeficiente do estimador, a inversa do *feedforward* e o limiar do seletor de aderência, enquanto o ensaio de tração de corrente identificou a dinâmica da malha escrava, com o tratamento de cada conjunto de dados especificado junto ao seu ensaio. A malha de controle foi então dissecada bloco a bloco no código da simulação, com o fluxograma de cada elemento, e as incertezas foram caracterizadas componente a componente, no método do Tipo A e do Tipo B, e combinadas por propagação em soma quadrática. O Capítulo 4 apresenta os resultados obtidos com este procedimento.

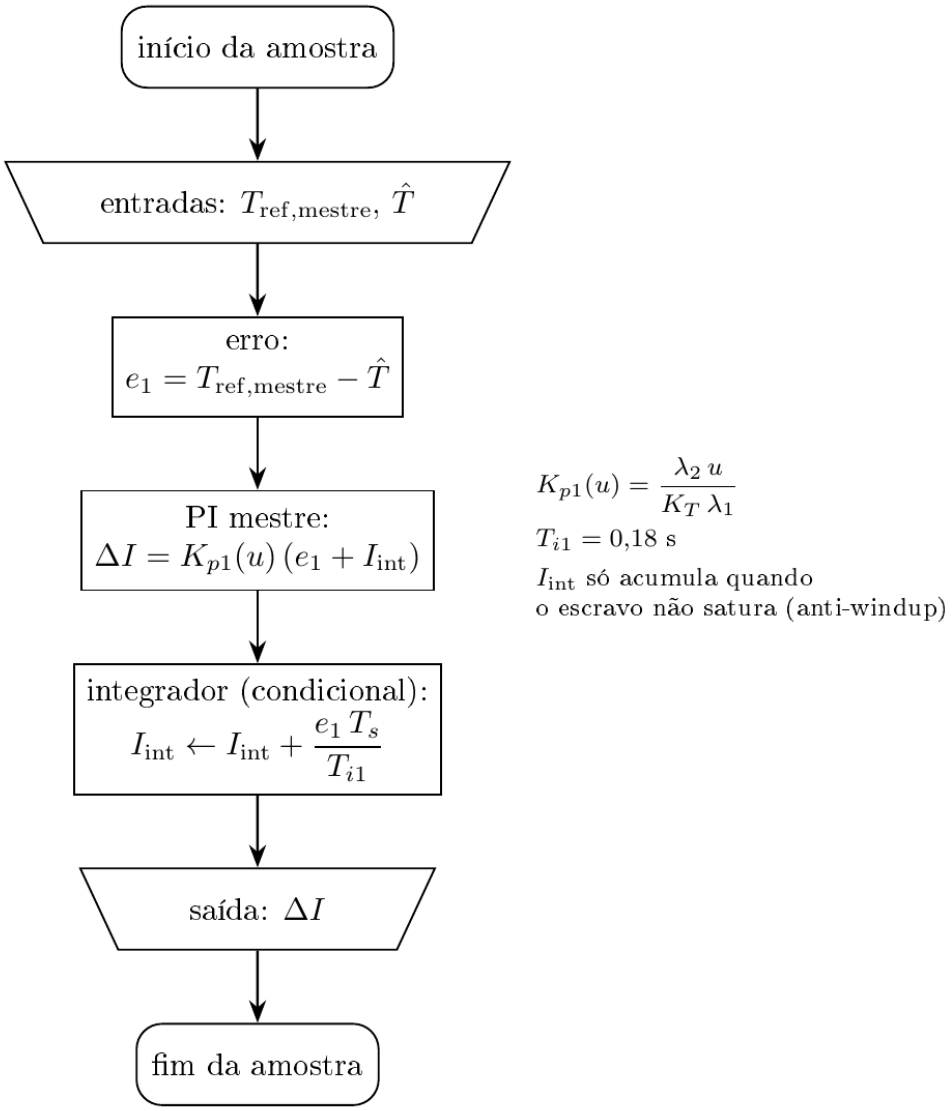


Figura 3.9: Fluxograma do controlador mestre: erro contra o torque estimado, lei PI com ganho programado e integrador condicionado à não saturação do escravo.
Fonte: elaborada pela autora (2026).

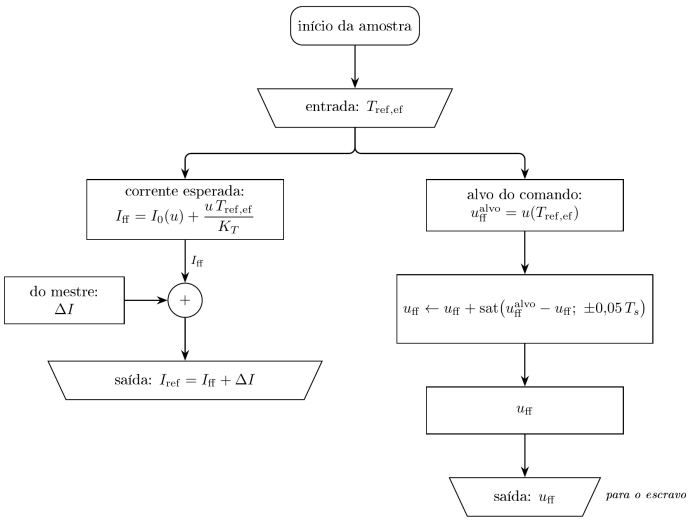


Figura 3.10: Fluxograma do *feedforward* e do caminho direto: corrente esperada, comando equivalente rampado e soma com a correção do mestre.
Fonte: elaborada pela autora (2026).

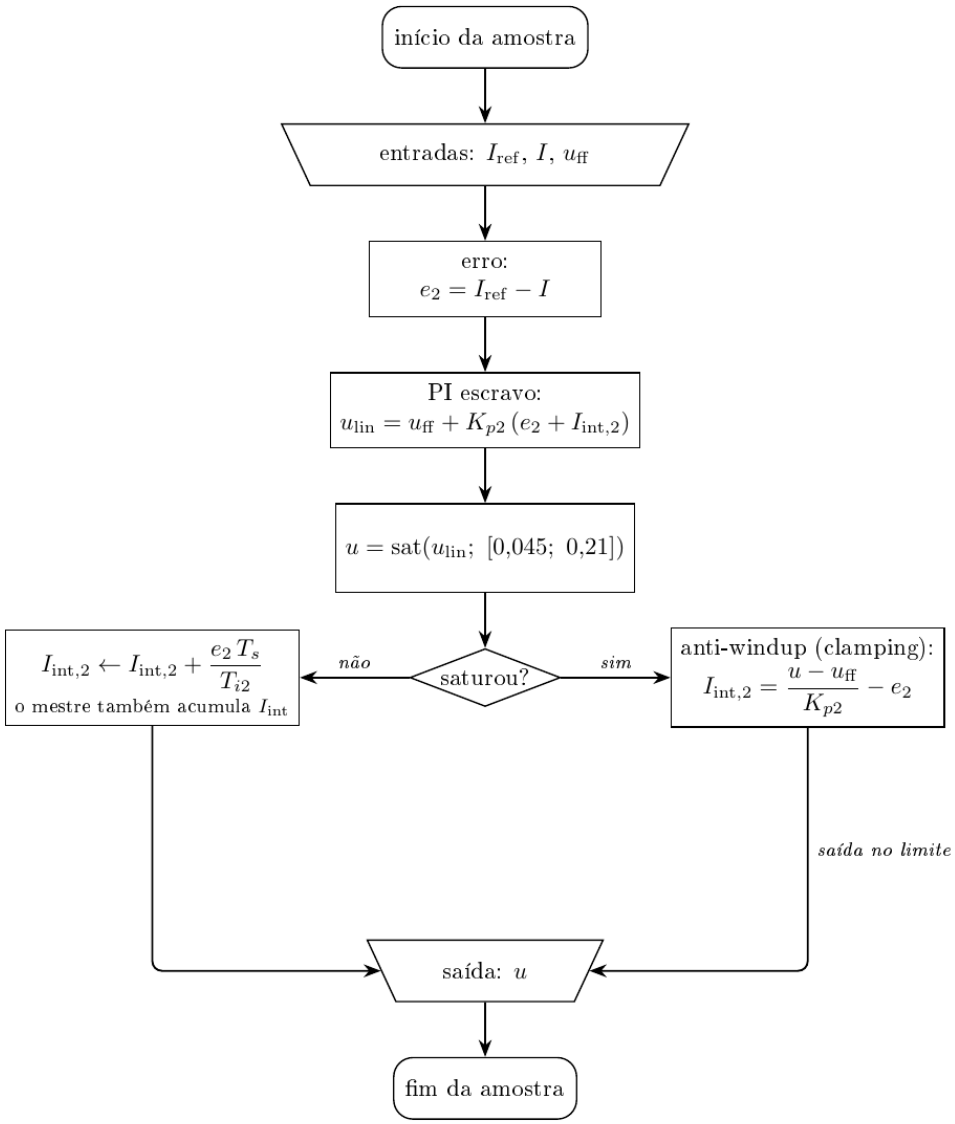


Figura 3.11: Fluxograma do controlador escravo com anti-*windup* por *clamping*: saturação nos limites da região validada e condicionamento do integrador.
Fonte: elaborada pela autora (2026).

Capítulo 4

Resultados

Este capítulo apresenta os resultados dos ensaios definidos na metodologia, na ordem em que foram conduzidos, e o desempenho da malha projetada sobre eles em simulação. As regiões de operação são definidas na Seção 4.2, já que o comportamento do propulsor não é uniforme ao longo da escala de comando. Cada ensaio é apresentado com as figuras e tabelas geradas e com a análise do que mostram, e o tratamento dos dados segue o capítulo anterior.

4.1 Testes em Bancada

A verificação em bancada fornece os desvios sistemáticos dos instrumentos que alimentam o orçamento de incertezas da Seção 3.7, e os valores obtidos são registrados a seguir. O teste foi conduzido em três configurações: o sensor em série com a fonte ajustável, a cadeia com o fio do conector interrompido para a inserção do multímetro e o sensor instalado no veículo, montagem empregada nos ensaios. Os três arranjos apresentaram resultados equivalentes, o que indica que a topologia da conexão não influencia a resposta da cadeia, e os dados consolidados estão na Tabela 4.1, com a reta ajustada forçada pela origem seguindo a Equação (3.2).

O resultado,

$$y = 1{,}0742\,x,$$

indica que o sistema lê 7,42% acima do multímetro, desvio quase constante em toda a faixa e sempre no mesmo sentido, podendo ser compensada via software sem distorcer a forma do sinal. A Figura 4.1 confronta o ajuste com a resposta ideal.

A balança foi verificada com pesos padrão de 0,500 kg, 1,000 kg e 1,500 kg, e as indicações foram 0,490 kg, 0,980 kg e 1,470 kg. O desvio é sistemático e proporcional, 10 g para cada 0,5 kg nos três pontos, característica de um desvio multiplicativo da

Tabela 4.1: Verificação da cadeia de corrente: pares de leituras do multímetro de referência (x) e do sistema (y), em amperes.

x (A)	y (A)
0,81	1,11
1,03	1,27
1,19	1,35
1,29	1,51
1,47	1,67
1,60	1,80
1,72	1,92
1,92	2,15

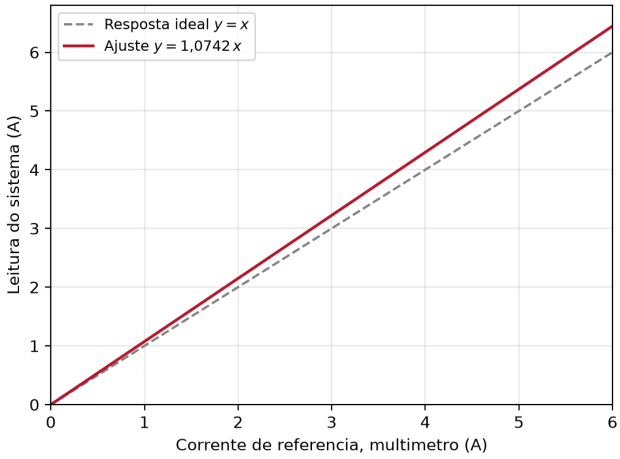


Figura 4.1: Verificação da cadeia de corrente: ajuste $y = 1{,}0742\,x$ sobre a resposta ideal $y = x$.

Fonte: elaborada pela autora (2026).

leitura, declarado como componente de incerteza sem correção, conforme a Seção 3.7.

4.2 Regiões de operação

O ensaio de tração de corrente versus força revelou que o propulsor opera de três maneiras distintas ao longo da escala de comando.

A **zona morta**, de 97° a 99°, é o intervalo em que o ESC está armado e consome corrente, mas a tração útil é residual, de modo que as referências dentro dessa faixa não produzem resposta mecânica e o controlador precisa atravessá-la sem acumular erro.

A **região de aderência**, de 100° a 113°, é aquela em que a roda permanece girando, sem patinagem, e toda a tração se transfere ao cabo. A força passa a depender apenas do torque do motor, que por sua vez decorre da corrente, sendo essa a única região em que o modelo é válido.

Acima de 113°–114° inicia-se a **região de saturação**, em que a aderência se esgota, ocorrendo patinagem das rodas ou o corte por subtensão do ESC, proteção de fábrica que interrompe o motor sob queda de tensão da bateria. A força deixa de ser determinada pela corrente, pois acima desse ponto o comando produz apenas deslizamento, delimitando a faixa útil do atuador.

Os limites exatos dessas regiões variam de ensaio para ensaio, e a Tabela 4.2 consolida as faixas observadas em cada um.

Tabela 4.2: Faixas de comando das regiões de operação, por ensaio (graus).

Ensaio	Zona morta	Região de aderência	Região de saturação
Ensaio em vazio	97–99	—[a]	não observada
Tração de corrente	97–99	100–109	a partir de 110[d]
Tração versus força (subida)	97–99	100–110[b]	não observada
Tração versus força (descida)	99	100–113	114–117[c]

[a] Rodas suspensas, sem tração.

[b] Força estaciona entre 105° e 110°.

[c] Força máxima em 114°; corte do ESC em 116–117°.

[d] Resposta contaminada pela patinagem.

Na subida, a força retoma o crescimento acima de 110° e atinge em 117° o mesmo máximo da descida, porém sem manifestar o limiar de aderência na escala percorrida, e, na descida, os dois últimos comandos são tomados pelo corte do ESC.

4.3 Ensaio em Vazio

O ensaio em vazio forneceu a corrente de perdas $I_0(u)$ que o estimador desconta de cada leitura e que a pré-alimentação soma a cada referência. O ensaio testa a hipótese estrutural do modelo, pois, com o veículo parado, a corrente de perdas é essencialmente a própria corrente de fase média, que se relaciona ao *duty cycle* conforme a Equação (2.3). O ajuste não linear de $I_0(u)$ invalidaria a representação da corrente de fase pelo quociente I/u. A Tabela 4.3 consolida os valores que alimentam o ajuste.

Na zona morta, de 97° a 99°, a corrente de perdas permanece abaixo de 1,2 A; na zona de aderência, de 100° a 110°, situa-se em torno de 1,3 A, com pouca variação em relação a faixa anterior; a partir de 111°, o crescimento se torna sustentado, e a corrente atinge 2,35 A no último comando. O ajuste linear sobre as médias da Tabela 4.3,

$$I_0(u) = 0{,}53 + 6{,}70\,u,$$

apresenta coeficiente de determinação $r^2 = 0{,}85$ (Equação (3.3)), representado na Fi-

Tabela 4.3: Ensaio em vazio: corrente de perdas média por comando.

α (graus)	u	I_0 (A)	α (graus)	u	I_0 (A)	α (graus)	u	I_0 (A)
97	0,024	0,48	104	0,106	1,29	111	0,188	1,99
98	0,035	0,77	105	0,118	1,33	112	0,200	2,01
99	0,047	1,14	106	0,129	1,48	113	0,212	2,10
100	0,059	0,92	107	0,141	1,14	114	0,224	2,17
101	0,071	0,98	108	0,153	1,71	115	0,235	1,58
102	0,082	1,03	109	0,165	1,78	116	0,247	2,28
103	0,094	1,15	110	0,176	1,32	117	0,259	2,35

gura 4.2. A inclinação da reta vem do crescimento concentrado nos comandos altos, pois até 110° a corrente de perdas quase não varia e só os últimos valores se elevam de fato, de forma que o coeficiente angular representa uma média do comportamento sobre toda a escala, superestimando as perdas nos comandos baixos e subestimando nos altos.

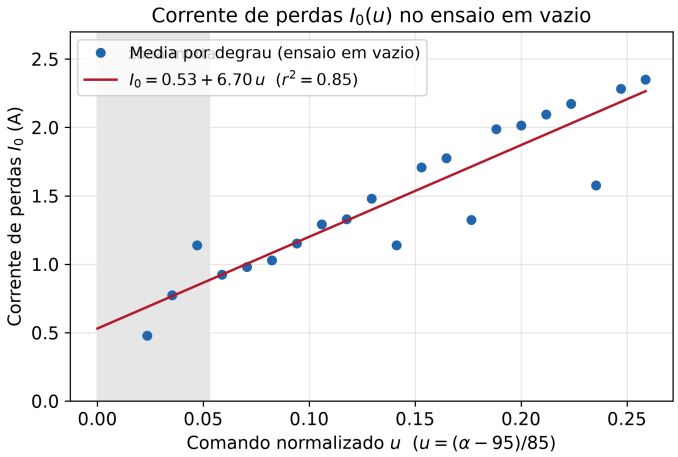


Figura 4.2: Corrente de perdas I_0 em função do comando normalizado u, com a zona morta destacada e o ajuste linear.
Fonte: elaborada pela autora (2026).

Os pontos se distribuem em torno da reta sem desvio sistemático aparente. A comparação com a corrente de carga da caracterização mostra a relevância relativa das perdas, pois no centro da região de aderência a corrente medida com carga é de 13 A no comando de 105° (Tabela 4.4), e as perdas descontadas, cerca de 1,3 A, são uma fração pequena, mas não desprezível, do sinal. A linearidade confirmada sustenta a cadeia do modelo, pois o desconto das perdas no estimador e a sua recomposição no *feedforward* passam a operar com um $I_0(u)$ bem comportado, sem o enviesamento que uma curva não linear introduziria nos degraus baixos.

4.4 A Curva Característica Estática

O protocolo fornece, em cada comando da escada, um par de valores em regime, a corrente consumida e a força de tração lidas nos segundos 7 a 10, e o tratamento, do pareamento corrente–força às medianas e ao ajuste do coeficiente, segue a Seção 3.5.6 da metodologia. A Tabela 4.4 consolida os dados, com o comando normalizado conforme a Equação (3.4) da metodologia, e os travessos marcam o comando de 98° da descida, não executado nessa sessão e excluído do consolidado.

Tabela 4.4: Caracterização estática: medianas das quatro repetições por comando, com desvio-padrão entre parênteses. Correntes em A; forças em N.

Comando	u	I_{sub}	F_{sub}	I_{desc}	F_{desc}
98	0,035	1,36 ± 0,26	4,44 ± 0,59	–	–
99	0,047	1,15 ± 0,81	5,17 ± 2,19	3,15 ± 0,35	10,69 ± 1,80
100	0,059	4,40 ± 0,19	14,22 ± 1,18	4,46 ± 0,28	14,37 ± 1,94
101	0,071	6,87 ± 0,29	23,23 ± 0,65	6,00 ± 1,04	21,26 ± 1,37
102	0,082	7,73 ± 0,23	25,29 ± 1,01	7,95 ± 0,29	23,89 ± 1,16
103	0,094	9,79 ± 0,24	30,02 ± 1,41	10,11 ± 0,28	27,93 ± 0,22
104	0,106	10,56 ± 0,73	32,08 ± 1,31	11,89 ± 0,09	34,89 ± 0,43
105	0,118	11,88 ± 0,86	31,54 ± 2,37	13,34 ± 0,03	36,89 ± 0,47
106	0,129	13,11 ± 1,27	31,76 ± 5,87	15,22 ± 0,32	40,25 ± 1,33
107	0,141	11,48 ± 1,15	32,01 ± 4,25	17,47 ± 0,42	43,24 ± 1,65
108	0,153	12,82 ± 0,76	34,51 ± 1,29	19,31 ± 0,67	41,55 ± 0,64
109	0,165	11,75 ± 2,16	32,47 ± 0,73	20,16 ± 0,63	46,17 ± 2,22
110	0,176	15,56 ± 1,07	32,52 ± 0,30	22,41 ± 0,27	47,18 ± 0,42
111	0,188	17,20 ± 0,85	35,24 ± 0,87	25,17 ± 2,39	46,08 ± 5,61
112	0,200	12,35 ± 1,00	37,87 ± 1,82	26,58 ± 1,02	50,89 ± 0,80
113	0,212	16,75 ± 1,76	37,67 ± 0,76	26,61 ± 0,36	50,79 ± 1,94
114	0,224	17,80 ± 2,30	40,66 ± 0,84	29,25 ± 0,87	52,16 ± 1,08
115	0,235	16,95 ± 3,72	46,65 ± 2,19	26,15 ± 0,50	49,76 ± 3,44
116	0,247	21,12 ± 2,25	48,15 ± 1,05	14,98 ± 1,00	38,48 ± 0,52
117	0,259	16,89 ± 0,23	52,04 ± 1,37	15,20 ± 0,35	44,15 ± 0,63

Na zona morta, a força permanece em torno de 4,4 N e a corrente em torno de 1,3 A, valores compatíveis com a corrente de perdas do ensaio em vazio, pois o ESC alimenta o motor, porém o torque entregue não vence as resistências internas do conjunto. O limite superior da região coincide com o início do crescimento sustentado da corrente de carga.

Na região de aderência, os dois sentidos divergem. Na descida, a força cresce de 14,4 N em 100° até 50,8 N em 113°, com o máximo de 52 N já na fronteira da saturação e dispersão típica de ±1 N entre repetições, comportamento esperado de um propulsor abaixo do limite de aderência, no qual a roda gira sem deslizar e a força acompanha a corrente. Essa é a base da validação do modelo. Na subida, entre 105° e 110°, a força

estaciona em torno de 31 N enquanto a corrente oscila entre as repetições, até retomar o crescimento acima de 110°.

A descida demanda, ainda, mais corrente para o mesmo comando ao longo de toda a escala, o que se percebe no ruído do motor e no tremor do conjunto, e registra-se como observação, sem medição dedicada, que a bateria se descarregava visivelmente mais rápido nesse sentido. A diferença entre os sentidos é sistemática e se quantifica pelo quociente das medianas da Tabela 4.4, comando a comando, de 104° a 115°, variando de +7%, em 115° (49,76/46,65 = 1,07), a +45%, em 110° (47,18/32,52 = 1,45).

Na região de saturação, a mudança de comportamento ocorre nos dois últimos comandos da descida: a corrente, que chegava a 26–29 A nos comandos vizinhos, cai para cerca de 15 A e a força recua de 50 N para 38–44 N. Foi observado um comportamento cíclico de aceleração e parada, compatível com o corte por subtensão do ESC, pois, em altas correntes, a tensão da bateria cai abaixo do limiar de proteção, o ESC interrompe o motor, a tensão se recupera e o ciclo recomeça. Esses pontos são registrados como observação e não integram a curva.

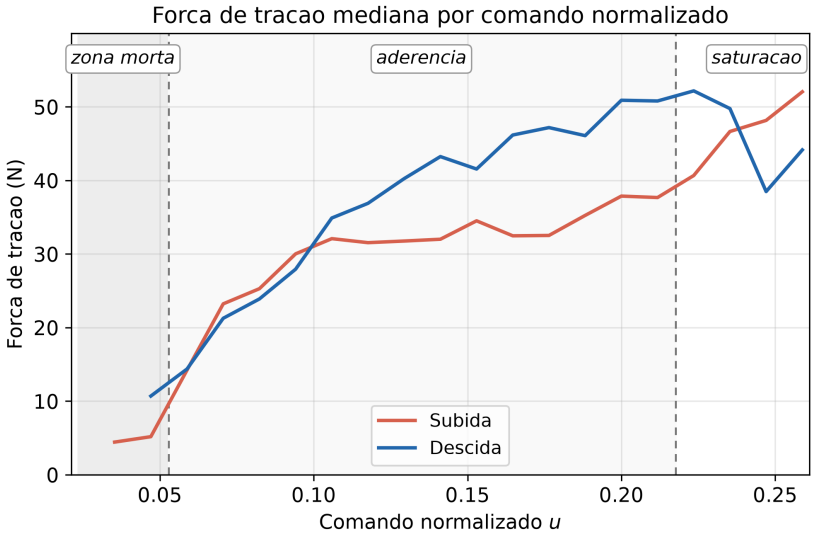


Figura 4.3: Força de tração mediana em função do comando normalizado, para os dois sentidos da escada, com as regiões de operação delimitadas.
Fonte: elaborada pela autora (2026).

A Figura 4.3 apresenta os mesmos dados em curva, na qual as regiões se distinguem de imediato e o espaço entre os dois sentidos registra visualmente a diferença sistemática quantificada acima. Os dois últimos pontos da descida ficam fora da curva pelo corte do ESC.

O resultado central é a validação do modelo de torque contra a balança. O modelo, definido na Seção 2.3 (Equação (2.6)), relaciona, na região de aderência, a força de

tração à corrente de fase equivalente por um único coeficiente, e, se é válido, os pontos medidos devem alinhar-se sobre uma reta que passa pela origem, com o coeficiente de determinação (Equação (3.3)) medindo o alinhamento e abrangendo, de uma vez, a linearidade do ESC, o desconto das perdas e a conversão pela razão cíclica. O valor de G_F é o coeficiente angular da reta da Equação (2.6), obtido por mínimos quadrados forçados pela origem sobre os pontos individuais das quatro repetições de cada comando da região de aderência, sendo cada ponto o par cuja abscissa é a corrente de fase equivalente $(I - I_0(u))/u$, com $I_0(u)$ vindo do ajuste do ensaio em vazio, e cuja ordenada é a força convertida de kgf para N, conforme a Seção 3.5.6. O cálculo é feito e impresso pelo script `analise_estatica.py` (Apêndice A.1), e o ajuste resulta, na descida, em $G_F = 0{,}372$ N/(A/u) com $r^2 = 0{,}79$ (56 pontos), e, na subida, $G_F = 0{,}368$ N/(A/u) com $r^2 = 0{,}11$ (44 pontos). Os dois coeficientes praticamente coincidem, o que confirma que a constante do conjunto não depende do sentido da escada. Na descida, a variância não explicada, cerca de 21%, é da mesma ordem da dispersão entre repetições da Tabela 4.4, e o modelo é validado. Na subida, o ajuste praticamente desaparece, e a razão é física, não estatística, pois a força estaciona enquanto a corrente oscila, espalhando os pontos na horizontal. A descida valida o modelo, e a subida delimita onde ele deixa de valer.

O estimador de torque usado no controlador converte o coeficiente de força em coeficiente de torque pelo raio efetivo, $\hat{T} = K_T\,(I_{\mathrm{bat}} - I_0(u))/u$, com $K_T = G_F\,r_{\mathrm{ef}} = 0{,}028$ N·m/(A/u), no qual o raio efetivo medido entra como fator de conversão entre a força da balança e o esforço na roda, ao passo que a constante de torque do motor, o rendimento e a redução ficam absorvidos no ajuste e não precisam ser conhecidos isoladamente. O modelo, em ambos os usos, estende-se apenas pela região de aderência, pois acima dela o quociente I/u deixa de representar a corrente de fase e a força deixa de ser determinada pela corrente, o que a própria regressão confirma ao produzir valores incompatíveis com a física fora da faixa. A malha externa herda essa limitação, que é a origem do seletor de aderência e da limitação do comando à faixa validada, projetados no Capítulo 2.

4.5 A Dinâmica entre Comando e Corrente

O segundo modelo vem do ensaio de tração de corrente (Seção 3.5.6), conduzido sem anotação de força, com a corrente amostrada a 100 Hz e registrada por inteiro em cada comando, e o tratamento das bordas, do nível de repouso ao início elétrico e aos dois métodos de ajuste, segue a metodologia. Os blocos não começam no comando, e os primeiros 1,5 a 2 s de cada bloco registram o repouso que o precede, como se vê na

Figura 4.4, de forma que a resposta é ancorada no seu início elétrico para não misturar o atraso do protocolo com a dinâmica da planta.

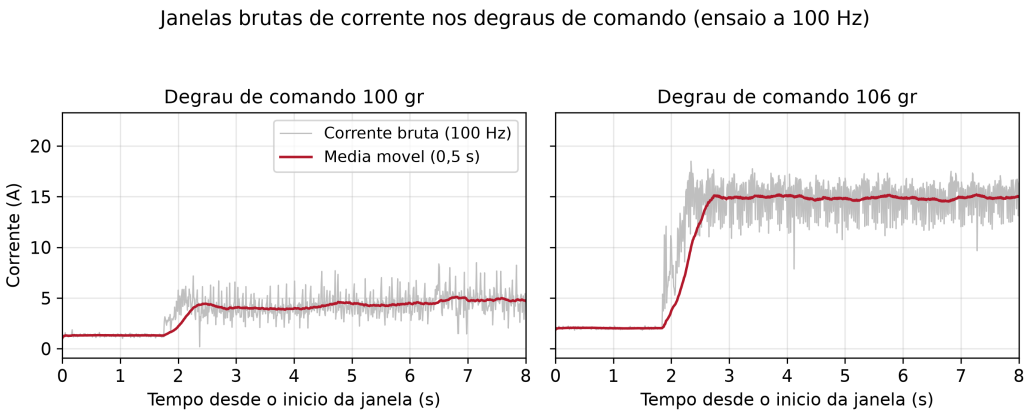


Figura 4.4: Janelas brutas de dois comandos do ensaio de tração de corrente, com corrente bruta (cinza) e média móvel (vermelho): o patamar inicial é o neutro que precede o comando, e a subida é rápida uma vez iniciada.
Fonte: elaborada pela autora (2026).

As Figuras 4.5 e 4.6 repetem a leitura para os degraus de 103° e 109°, no meio e no fim da região de aderência, e o mesmo padrão se confirma, repouso, subida rápida uma vez iniciada e platô estável.

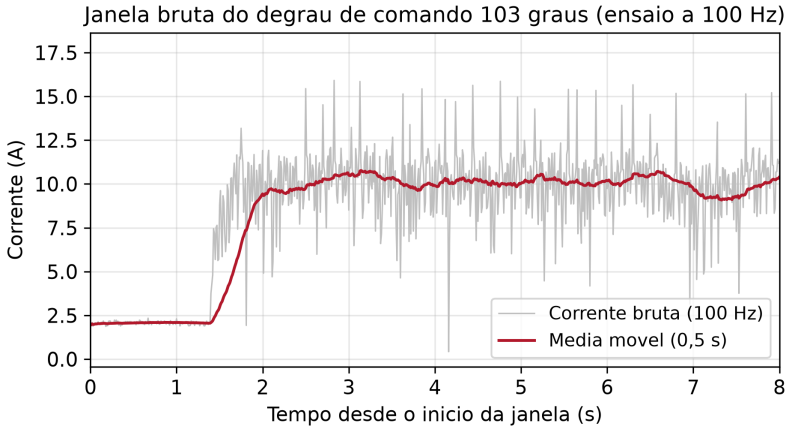


Figura 4.5: Janela bruta do degrau de comando 103°, com corrente bruta a 100 Hz e média móvel: o repouso que precede o comando e o platô de regime.
Fonte: elaborada pela autora (2026).

Cada resposta é normalizada entre o seu nível de repouso e o seu platô, $(I - I_r)/(I_p - I_r)$, de modo que todas partem de zero e se acomodam em um, o que permite sobrepor degraus de correntes diferentes. A Figura 4.7 apresenta as dez respostas alinhadas e normalizadas, com a média e o ajuste sobre ela, sobre os comandos de 100° a 109°, os únicos sem contaminação de patinagem, conforme o critério da Seção 3.5.6.

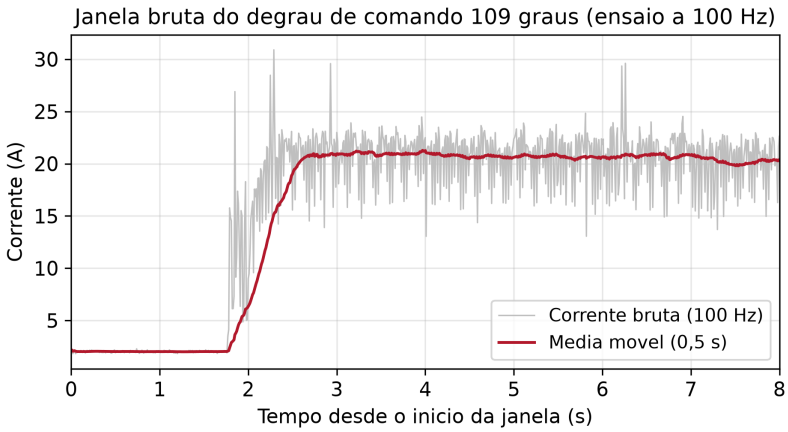


Figura 4.6: Janela bruta do degrau de comando 109°, com corrente bruta a 100 Hz e média móvel: o mesmo padrão no fim da região de aderência.
Fonte: elaborada pela autora (2026).

Os resultados são consistentes entre si. O atraso é inferior a 0,1 s, pois a corrente responde dentro da primeira janela de 100 ms após o início elétrico. A partida sem sensor de rotação, na qual o ESC estima a posição do rotor, não impõe atraso mensurável nesse caminho. A constante de tempo da subida situa-se em 0,18 s, valor que os dois métodos reproduzem sobre a média das dez bordas, e a curva do ajuste acompanha o feixe de respostas ao longo de toda a subida, sem desvio sistemático aparente. O ganho, tomado do platô, o quociente da corrente final pelo comando, cresce de 82 a 126 A/u ao longo da região, acompanhando a curva estática e cruzando as duas identificações, e a sobreposição das respostas normalizadas indica que a planta é a mesma para todos os comandos da região de aderência. A planta é, portanto,

$$G_2(s) = \frac{111}{0{,}18\,s + 1}, \tag{4.1}$$

sem atraso apreciável, em que o ganho 111 A/u é a média dos quocientes platô por comando das dez bordas, cada um como 13,31 A em 105° sobre $u = 0{,}118$, ou 113 A/u, na faixa de 82 a 126 A/u, conforme o tratamento da Seção 3.5.6, e a resposta se assenta em cerca de quatro vezes 0,18 s, pela regra usual de que uma primeira ordem se acomoda em quatro constantes de tempo. O valor é coerente com o esperado para a constante elétrica do caminho comando–corrente, pois, com a roda travada, a força contra-eletromotriz é baixa e a corrente acompanha o comando quase sem oposição, e a aparente lentidão observada nas janelas brutas pertence ao protocolo de disparo, não à planta, desaparecendo quando a resposta é ancorada no seu início elétrico.

Essa distinção tem consequência direta para a sintonia da próxima seção, pois a planta elétrica não limita a velocidade do controlador.

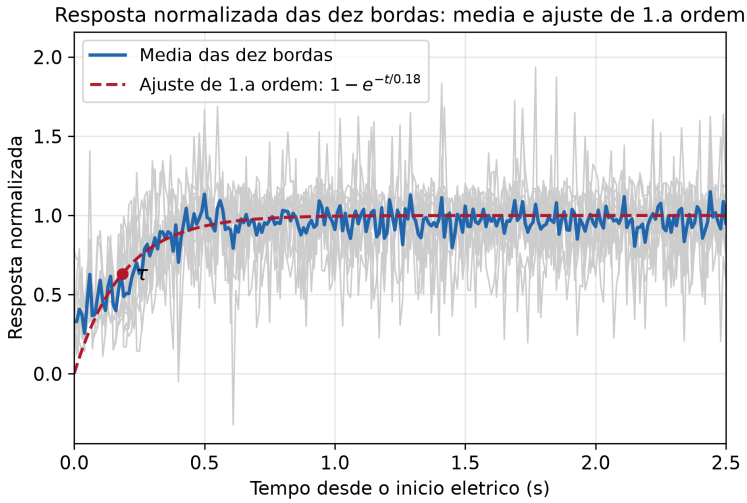


Figura 4.7: Respostas ao degrau alinhadas no início elétrico, normalizadas, com a média das dez bordas e o ajuste de primeira ordem.
Fonte: elaborada pela autora (2026).

4.6 O Efeito da Bateria nos Ensaios

O estado da bateria mostrou-se uma condição de ensaio tão decisiva quanto a massa e o piso, e os registros das sessões com as baterias antigas do laboratório, uma delas estufada, permitem quantificar o efeito. O estudo compara a sessão de referência, conduzida com a bateria em bom estado, a uma sessão antiga cuja primeira passagem ainda executava os degraus e às retomadas do comando de 114°, onde o colapso se manifestou. O processamento segue o mesmo protocolo do ensaio de tração de corrente e é reproduzido pelo script `analise_bateria.py` (Apêndice A.5).

Na sessão de referência, a mediana da corrente cresce de 4,3 A em 100° a 29,4 A em 114°, sem amostras nulas em nenhum degrau. Na primeira passagem da sessão antiga, os degraus ainda executam, com medianas de 19 a 28 A, porém com 10% a 22% de amostras nulas, sinal precoce da fraqueza da bateria. Nas retomadas do 114°, a mediana do bloco cai para 15,2 A, 48% abaixo dos 29,4 A do mesmo degrau na sessão de referência, e 45% das amostras permanecem em zero, com o motor, quando ligado, drenando mediana de 20,5 A, como se vê na comparação por degrau da Figura 4.8.

Nas retomadas, o padrão é o ciclo de corte por subtensão, com quatro religamentos detectados, com o motor ligado por cerca de 8 s, cortado por cerca de 7 s, até que a terceira retomada, após 0,1 s, não sustenta a partida e a corrente permanece nula até o fim do bloco, a bateria sem carga suficiente para religar o ESC. O colapso é específico das retomadas, e não da sessão, pois degraus vizinhos da mesma corrida executam normalmente, 117° com 28,2 A e 118° com 25,9 A, de forma que o efeito se concentra

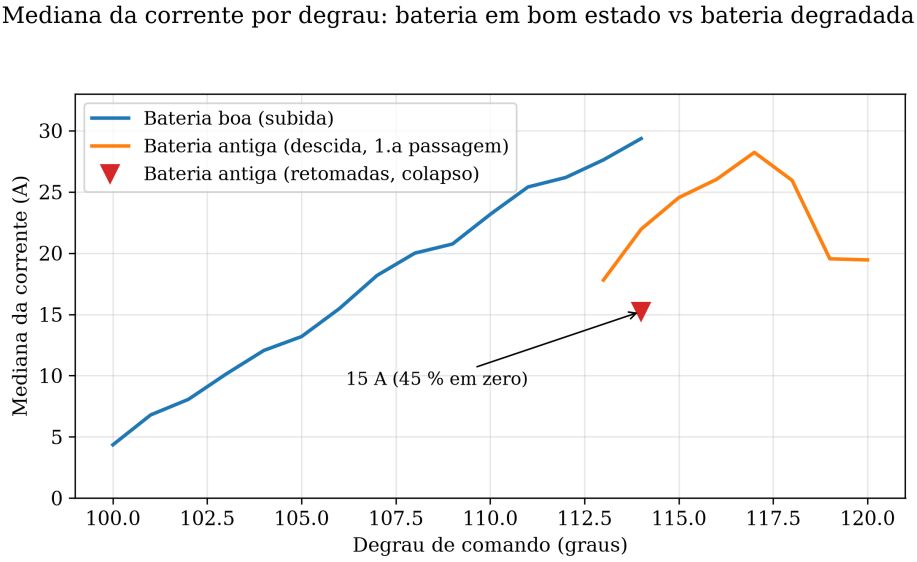


Figura 4.8: Mediana da corrente por degrau de comando: sessão de referência, com a bateria em bom estado, e sessão antiga, com a bateria degradada, destacando o colapso das retomadas de 114°.

Fonte: elaborada pela autora (2026).

onde a exigência de corrente se acumula em blocos repetidos. A interpretação é direta, pois, sob alta corrente, a tensão da bateria degradada cai abaixo do limiar de proteção do ESC, o corte interrompe o motor, a tensão se recupera e o ciclo recomeça, o mesmo mecanismo registrado na região de saturação da caracterização. O ensino prático, que motivou a regra de validação dos consolidados, é que sessões com baterias degradadas executam degraus altos com corrente de degraus menores, e que o estado da bateria deve constar do *checklist* do ensaio e das regras de validação, conforme adotado neste trabalho.

4.7 A Sintonia dos Controladores

A sintonia segue o método de modelo interno, uma síntese direta na qual a lei de controle deriva do modelo do processo, com o parâmetro de ajuste explicitando o compromisso entre desempenho e robustez [16, 11]. O método parte do modelo da planta, especifica o comportamento desejado da malha fechada e resolve o controlador que o entrega, e os passos abaixo definem cada variável empregada.

1. **Modelo da planta.** Cada planta é representada por um modelo de primeira ordem com atraso,

$$\tilde{G}(s) = \frac{K}{\tau\, s + 1}\, e^{-\theta_a s},$$

em que K é o ganho estático da planta, a variação da saída por unidade de entrada em regime, τ é a sua constante de tempo, o tempo para atingir 63,2% do valor final após um degrau, e θ_a é o atraso aparente. A malha escrava emprega a dinâmica identificada no ensaio de tração de corrente (Seção 4.5), $G_2(s) = 111/(0{,}18\,s+1)$, com atraso desprezível, $\theta_a \approx 0$, pois inferior a 0,1 s. Como a curva estática não é uma reta, sendo o ganho do platô crescente de 82 a 126 A/u ao longo da região, emprega-se para o projeto o ganho local, a inclinação dessa curva no ponto de operação, e não a média. Ele é ajustado por uma quadrática sobre os dados da descida, $I_{\mathrm{ss}}(u) = -96{,}4\,u^2 + 175{,}6\,u - 5{,}76$, ajustada por mínimos quadrados de segunda ordem sobre os pontos individuais das repetições da descida, pares de comando normalizado e corrente de regime, pelo mesmo princípio da Seção 3.1 (o script da simulação, listado no Apêndice A.3, ajusta e imprime esses coeficientes), cuja derivada no ponto nominal, o comando de 105° ($u = 0{,}118$), vale $K = 153$ A/u.

2. **Especificação da malha fechada.** Especifica-se que a malha fechada se comporte como sistema de primeira ordem,

$$\frac{Y}{R} = \frac{1}{\lambda\, s + 1},$$

em que λ é a constante de tempo que se deseja exigir dela, o mais lento que se aceita entre desempenho e robustez, e R e Y são a referência e a saída da malha considerada.

3. **Filtro de robustez.** A especificação é filtrada por

$$F(s) = \frac{1}{\lambda\, s + 1},$$

que torna o ajuste robusto a incertezas do modelo. No método [11], o controle clássico realimenta apenas a saída medida, ao passo que o controle por modelo interno compara a saída com a prevista pelo modelo $\tilde{G}$ do processo, e o controlador contém a inversa da parte invertível do modelo e o filtro F.

4. **Controlador equivalente.** O cancelamento do polo da planta e a solução para o controlador resultam, para o PI da Equação (2.11),

$$K_p = \frac{\tau}{K\,(\lambda + \theta_a)}, \qquad T_i = \tau, \tag{4.2}$$

em que K_p é o ganho proporcional do controlador e T_i o seu tempo integral.

5. **Ajuste da malha escrava.** Com a escolha $\lambda_2 = \tau = 0{,}18$ s, que fecha a malha tão rápido quanto a dinâmica permite, e $\theta_a \approx 0$, a Equação (4.2) fornece $T_{i2} = 0{,}18$ s e $K_{p2} = 0{,}18/(153 \times 0{,}18) \approx 6{,}5 \times 10^{-3}$ u/A.

6. **Ajuste da malha mestra.** O mestre vê uma planta composta pela escrava fechada seguida da conversão do estimador, cujo ganho depende do ponto de operação. O primeiro cuidado é a separação de velocidades, pois a regra da cascata manda que o mestre seja de três a dez vezes mais lento que a escrava [5], para que a escrava lhe pareça instantânea, e adota-se $\lambda_1 = 1{,}1$ s, seis vezes λ_2. O segundo é a variação do ganho com o comando, tratada por **ganho programado**, na qual o ganho proporcional do mestre é atualizado com o comando vigente, $K_{p1}(u) = \lambda_2\, u/(K_T\, \lambda_1)$, o que lineariza o caminho da malha externa ao longo da faixa de operação. No ponto nominal, $K_{p1} = 0{,}18 \times 0{,}118/(0{,}028 \times 1{,}1) \approx 0{,}70$ A/(N·m), e $T_{i1} = 0{,}18$ s.

As duas malhas, cada uma com sua planta, seu controlador e seus ganhos, consolidam-se na Tabela 4.5. Os demais elementos da estrutura, o *feedforward*, o seletor de aderência, os limites, o anti-windup, o estimador e as rampas, não são controladores, mas as proteções e as conversões que cercam os dois.

Tabela 4.5: As duas malhas da cascata e seus elementos de sintonia.

	Malha interna (escrava)	Malha externa (mestra)
Variável controlada	Corrente I	Torque estimado $\hat{T}$
Variável manipulada	Comando u	Referência de corrente
Planta	$G_2(s)$, identificada na Seção 4.5	Escrava fechada + estimador
Controlador	PI	PI com ganho programado
Ajuste (IMC)	$\lambda_2 = 0{,}18$ s	$\lambda_1 = 1{,}1$ s
Ganhos	$K_{p2} \approx 6{,}5 \times 10^{-3}$ u/A	$K_{p1}(u) = \lambda_2 u/(K_T \lambda_1)$
	$T_{i2} = 0{,}18$ s	$K_{p1} \approx 0{,}70$ A/(N·m), $T_{i1} = 0{,}18$ s

4.8 O Desempenho da Malha em Simulação

A malha sintonizada na Seção 4.7 tem seu desempenho avaliado por simulação não-linear sobre os modelos identificados neste capítulo, executada em tempo discreto de 10 ms pelo script da simulação (Apêndice A.3), que reproduz a curva estática da corrente, a corrente de perdas, a divisão pelo comando, o seletor e as saturações. O horizonte é de 64 s, e cada degrau só é aplicado após o regime permanente do cenário anterior, com margem de várias constantes de tempo. Quatro cenários cobrem a operação, um

degrau de referência de 1,6 para 3,0 N·m (PSR), uma perturbação de afundamento de tensão de 20% somada a um deslocamento de 0,3 A na corrente de perdas (PRP), um retorno de referência a 1,6 N·m e um degrau para além do limiar do seletor.

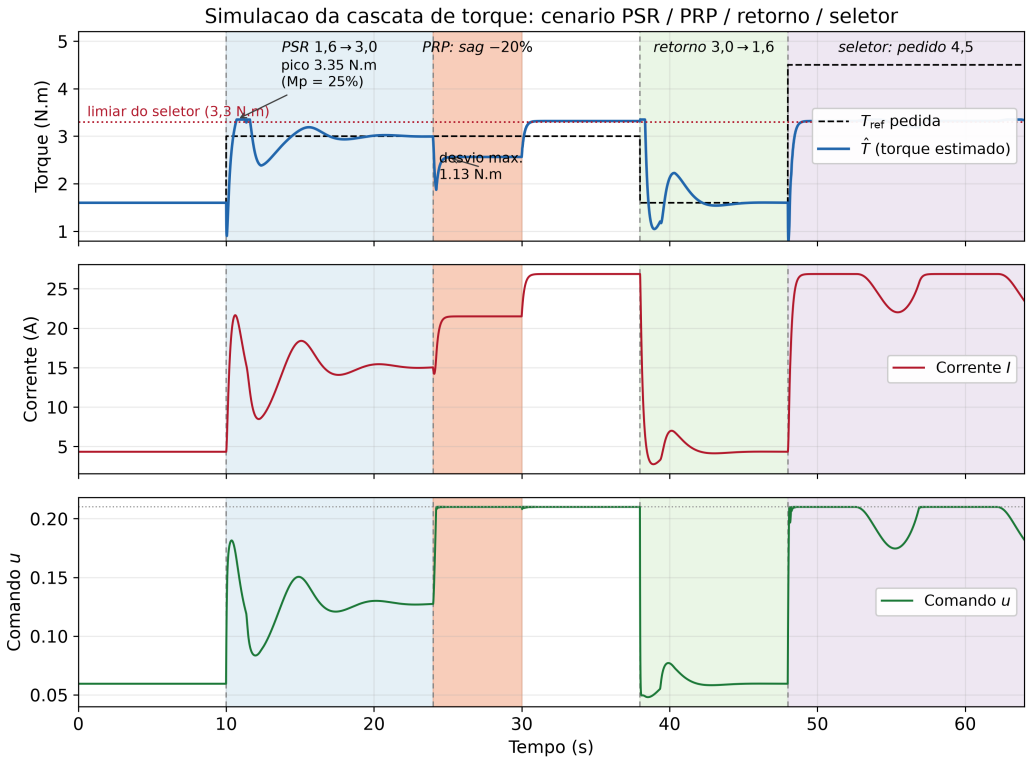


Figura 4.9: Desempenho da malha em simulação: resposta ao degrau de referência (PSR), rejeição da perturbação de afundamento de tensão e deslocamento das perdas (PRP), retorno de referência e atuação do seletor de aderência.
Fonte: elaborada pela autora (2026).

No PSR, o torque acompanha o degrau com pico de 3,35 N·m, sobressinal de 25%, e assenta no valor pedido. Na PRP, a perturbação produz desvio máximo de 1,13 N·m, com retorno ao regime de 3,318 N·m sobre a referência de 3,0 N·m, residual compatível com o deslocamento imposto às perdas. No retorno, o torque acompanha a referência de 1,6 N·m com valor de 1,542 N·m na amostra avalizada. No último cenário, o degrau para além do limiar é cortado pelo seletor, e o torque estabiliza em torno de 3,3 N·m, o limiar do seletor, sem patinagem comandada. A malha, assim, segue a referência dentro da região validada, rejeita a perturbação elétrica e respeita o limiar de aderência.

4.9 Resumo do Capítulo

Este capítulo apresentou os resultados dos ensaios, na ordem em que foram conduzidos. A verificação em bancada quantificou os desvios dos instrumentos, a cadeia

de corrente com 7,42% de leitura alta, corrigível em *software*, e a balança com desvio sistemático de 2%, declarado como incerteza. O ensaio em vazio forneceu a corrente de perdas $I_0(u)$, linear sobre o comando e parâmetro do estimador e do *feedforward*. O ensaio de tração de corrente versus força construiu a curva característica, validada na descida com $G_F = 0{,}372$ N/(A/u) e coeficiente de determinação 0,79, delimitou a região de aderência e o coeficiente de atrito aparente de 0,80, e registrou o corte por subtensão do ESC nos degraus mais altos. O ensaio de tração de corrente identificou a planta $G_2(s) = 111/(0{,}18\,s + 1)$, sem atraso apreciável, e o estudo das sessões antigas quantificou o efeito da bateria degradada, o colapso das retomadas de 114° com mediana de 15,2 A contra 29,4 A na sessão de referência. Sobre esses modelos, a sintonia pelo método de modelo interno fixou os ganhos das duas malhas, e a simulação confirmou o seguimento de referência, a rejeição da perturbação elétrica e o respeito ao limiar do seletor de aderência.

Capítulo 5

Conclusões

5.1 Considerações Finais

Este trabalho caracterizou experimentalmente o sistema de propulsão de um automodelo elétrico 1/8, obtendo sua curva característica torque versus corrente por meio de um ensaio de tração estática de baixo custo, com automodelo ancorado e balança de gancho como padrão de força e sensor de efeito Hall no ramo de alimentação. O modelo de torque foi validado com coeficiente de determinação de 0,79 na descida, com o ajuste direto $F = G_F\,(I - I_0(u))/u$ que dispensa o conhecimento da redução da transmissão, delimitado à região de aderência. A dinâmica entre o comando e a corrente foi identificada com constante de tempo de 0,18 s e atraso desprezível, em confronto com o *prior* clássico do caminho indicando que a dinâmica é do ESC, não do enrolamento. As considerações que seguem se estruturam em dois resultados, a resposta elétrica rápida e a aderência como fronteira de tudo o que se pode controlar.

Aderência e stick-slip. O fator de aderência aparente entre pneu e piso foi de aproximadamente 0,80, tomado sobre o peso total do conjunto, e só é compatível com atrito seco mediante transferência de carga do eixo dianteiro para o traseiro sob tração. A aderência mostrou-se, além disso, uma condição de estado: na subida, com o pneu frio, o limite se manifestou como *stick-slip*, oscilação de relaxamento com a força presa em cerca de 31 N e a corrente oscilando entre repetições, com o tremor do conjunto observado diretamente em bancada; na descida, com o pneu aquecido, a roda travou de forma sustentada e a força atingiu 52 N. A diferença sistemática entre os sentidos (+7% a +45% da região de aderência à fronteira da saturação) é compatível com duas contribuições não separadas neste trabalho, o aquecimento do pneu e o da bateria, e indica que a caracterização de um propulsor de tração deve especificar o histórico térmico do conjunto, não apenas sua massa. Para o controle, a conclusão é direta, pois o torque é controlável apenas dentro da região de aderência, e o limitador de um

controlador deve respeitar o limiar observado, sob pena de comandar patinagem em vez de tração.

O coeficiente de atrito e o piso do ensaio. O limite de aderência que delimita a região validada do modelo é, por construção, um limite de atrito, pois a maior força transferível ao solo vale $F_{\text{máx}} = \mu N = \mu m g$. Com a massa total de 6,640 kg, a carga normal é $N = 6{,}640 \times 9{,}81 \approx 65{,}1$ N, e a maior força observada no ensaio, 52 N na fronteira da saturação, fornece o coeficiente aparente $\mu = 52/65{,}1 \approx 0{,}80$. O valor fecha a cadeia pelo torque, pois 52 N $\times$ 0,0751 m $\approx$ 3,9 N·m é o torque máximo medido, e o seletor de aderência é fixado em 3,3 N·m, abaixo dele e com margem sobre a dispersão, o que confere consistência interna entre a mecânica do contato e a proteção da malha. O valor é fisicamente plausível, pois se situa na faixa usual de atrito entre borracha seca e superfícies rígidas, e apenas se explica com a transferência de carga do eixo dianteiro para o traseiro sob tração, já discutida. Uma caracterização direta do coeficiente seria possível no próprio piso, pelo arrasto de um peso conhecido com a balança do ensaio ou pelo método do plano inclinado, no qual $\mu = \tan\theta$ no ângulo de início do deslizamento, mas nenhuma dessas alternativas em outro piso jamais foi considerada. O modelo é válido onde o veículo opera, e o piso faz parte da planta, de forma que uma caracterização conduzida sobre outra superfície entregaria um limiar com um μ diferente, que invalidaria o modelo exatamente no lugar onde ele deveria valer. Como o coeficiente ainda varia com temperatura, poeira e estado do pneu, a identificação no piso de uso, sem controle sobre essas variáveis, é a única que garante que o limiar protegido pelo seletor é o limiar real. Esses limites, por fim, delimitam o próprio limitador da malha, pois o seletor de aderência e o teto do estimador projetados no Capítulo 2 nascem diretamente das medições aqui consolidadas.

A bateria como elo do experimento. Duas interferências de bateria constituem fontes de erro relevantes para o experimento. A bateria dedicada do Raspberry Pi, quando descarregada, derrubava a tensão do conversor *step-down*, medida em 4,0 V contra 5,3 V esperados, abaixo do mínimo operacional, provocando quedas da rede em plena sessão, com o veículo rodando, sendo o sintoma semelhante a uma falha de comando, que só foi diagnosticado pela medição da tensão de alimentação. As baterias antigas do laboratório, por sua vez, impuseram o problema oposto, o *sag* severo sob carga que conduziu ao corte por subtensão do ESC, quantificado no estudo da Seção 4.6 e responsável por rotular execuções inteiras da escada, o que motivou a regra de validação dos consolidados. A conclusão prática é que, em ensaios com ESC comerciais, o estado da bateria integra a condição de ensaio tanto quanto a massa e o piso, devendo constar do *checklist* e das regras de validação, conforme adotado neste trabalho.

Consequência para o controle. A resposta elétrica do propulsor é rápida, e o atraso aparente das janelas de ensaio pertence ao protocolo de disparo, não à planta, de modo que o projeto da malha não é limitado pela dinâmica elétrica. Com o modelo de torque validado, a dinâmica identificada e a sintonia apresentada na Seção 4.7, a malha proposta, com realimentação de corrente e estimativa de torque, *feedforward* da inversa estática e limitador de aderência, fica completamente especificada pelos parâmetros medidos, restando sua validação experimental como etapa pendente.

5.2 Propostas de Continuidade

Quatro continuidades decorrem diretamente dos resultados. A medição dinâmica da força, com célula de carga e registrador de baixo custo (HX711), permitiria identificar a dinâmica comando–tração e fechar a malha de torque sobre a grandeza de interesse, em vez da corrente. A instrumentação do pneu e da bateria, a temperatura do primeiro e a tensão da segunda durante os degraus, separaria as contribuições da memória térmica e embasaria a modelagem do limite de aderência. O sensor de rotação por efeito Hall (SS41F), previsto desde a revisão de instrumentação, viabilizaria o ensaio em vazio com rotação medida e a validação isolada da constante de torque do motor. E, por fim, a sintonia e a validação da malha de controle proposta, em simulação sobre o modelo identificado e, se possível, em bancada, completariam o caminho da caracterização ao controle.

Referências Bibliográficas

[1] DIGI Electronics. *Stator vs Rotor: Construction, Functions and Maintenance*. n.d. Blog técnico. Disponível em: https://www.digi-electronics.com/en/blogs/stator-vs-rotor-construction-functions-and-maintenance/518.html. Acesso em: 26 out. 2026.

[2] Kyosho Corporation. *Mad Force Kruiser 2.0 VE: Instruction Manual*. Tóquio, n.d. No. 30888-B-3, veículo 1:8 brushless.

[3] RUDRANARAYANG. *Accelerometer MPU-6050 Communication with AVR MCU*. n.d. Instructables. Disponível em: https://www.instructables.com/Accelerometer-MPU-6050-Communication-With-AVR-MCU/. Acesso em: 26 out. 2026.

[4] Honeywell. *Magnetic Position Sensing in Brushless DC Electric Motors*. Charlotte, n.d. Application Note 005849-5-EN. Cobre a comutação eletrônica de motores BLDC com os sensores bipolares das séries SS30 e SS31x/SS41x. Disponível em: https://prod-edam.honeywell.com/content/dam/honeywell-edam/sps/siot/en-us/products/sensors/magnetic-sensors/common/documents/sps-siot-magnetic-position-sensing-brushless-dc-electric-motors-app-note-005849-5-en3-c pdf. Acesso em: 28 out. 2026.

[5] RAFFO, G. V. *Estratégias de Controle: Cascata, Inferencial, Seletivo e Split-range*. 2019. Notas de aula, TCPI, DELT/UFMG.

[6] RAFFO, G. V. *Controle de Seguimento de Referência e Rejeição de Perturbações (PSR/-PRP)*. 2019. Notas de aula, TCPI, DELT/UFMG.

[7] RAFFO, G. V. *Controladores PID Industriais: Estruturas de Controle, Aspectos Práticos e Operacionais*. 2019. Notas de aula, TCPI, DELT/UFMG.

[8] RAFFO, G. V. *Controle de Variáveis Básicas*. 2019. Notas de aula, TCPI, DELT/UFMG.

[9] BRAGA, C. M. P.; MICHEL, H. C. C. *Configurações Generalizadas e Descrições Funcionais de Instrumentos*. 2020. Notas de aula, Instrumentação Industrial, DELT/UFMG.

[10] RAFFO, G. V. *Controle via Síntese Direta: Projeto e Implementação Digital*. 2017. Notas de aula, TCPI, DELT/UFMG.

[11] RAFFO, G. V. *Controle por Modelo Interno (IMC).* 2019. Notas de aula, TCPI, DELT/UFMG.

[12] BRAGA, C. M. P.; MICHEL, H. C. C. *Caracterização Estática de Instrumentos: Calibração.* 2020. Notas de aula, Instrumentação Industrial, DELT/UFMG.

[13] Conselho das Comunidades Europeias. *Regulamento (CEE) n.º 3821/85, de 20 de dezembro de 1985, relativo aos dispositivos de registro no transporte rodoviário: Anexo I, definição de perímetro efetivo dos pneumáticos das rodas.* 1985. Diário Oficial das Comunidades Europeias, L 370, p. 8.

[14] BRAGA, C. M. P.; MICHEL, H. C. C.; BRAGA, A. R. *Caracterização Dinâmica de Instrumentos.* 2023. Notas de aula, Instrumentação Industrial, DELT/UFMG.

[15] BRAGA, C. M. P.; MICHEL, H. C. C. *Incerteza de Medição: Incerteza Padrão Tipo A e Tipo B, Combinada e Expandida.* 2020. Notas de aula, Instrumentação Industrial, DELT/UFMG.

[16] GARCIA, C. E.; MORARI, M. Internal model control. a unifying review and some new results. *Industrial & Engineering Chemistry Process Design and Development*, v. 21, n. 2, p. 308–323, 1982.

Appendices

Apêndice A

Scripts de processamento dos dados

Os scripts a seguir implementam o processamento descrito na Seção 3.1 e geram as figuras de dados do Capítulo 4, a partir dos arquivos brutos dos ensaios. Executam-se em Python 3 e dependem apenas das bibliotecas `numpy`, `openpyxl` e `matplotlib`. Cada script é acompanhado dos arquivos que lê, de modo que a execução reproduz os números e as figuras apresentados no texto. O mapeamento entre ensaios, modelos e scripts é direto: o ensaio em vazio e o ensaio de tração de corrente versus força alimentam o primeiro script (A.1), que produz o modelo estático — a corrente de perdas $I_0(u)$ e o coeficiente G_F da curva característica; o ensaio de tração de corrente alimenta o segundo script (A.2), que produz o modelo dinâmico, a planta $G_2(s)$; o terceiro (A.3) simula a malha de controle sobre esses dois modelos e produz a figura do desempenho; e o quarto (A.4) é a automação do ensaio, que percorre a escala de degraus, dispara os comandos no Raspberry Pi e registra a corrente em CSV. Os três arquivos de dados são a planilha de anotações do ensaio de tração de corrente versus força (`ensaio_tracao_corrente_versus_forca.xlsx`), o registro do ensaio em vazio (`ensaio_vazio_corrente.csv`) e o registro do ensaio de tração de corrente (`ensaio_tracao_corrente.csv`). As dependências estão listadas em `requirements.txt` (`numpy`, `openpyxl`, `matplotlib` e `pyserial`), e o script A.4 reutiliza o módulo de automação do ensaio, `rodar_ensaio.py`, mantido na pasta dos ensaios do projeto.

A análise se divide em dois scripts, A.1 e A.2, não porque haja uma separação natural entre os ensaios, mas porque o controlador consome dois modelos distintos — o estático, do estimador e do *feedforward*, e o dinâmico, da malha escrava —, e cada script entrega um deles, com as figuras respectivas.

A.1 Caracterização estática

```
"""Processamento dos dados da caracterizacao estatica.

Le a planilha de anotacoes do ensaio de tracao de corrente versus forca
('ensaio_tracao_corrente_versus_forca.xlsx') e o ensaio
em vazio ('ensaio_vazio_corrente.csv'), ajusta as retas por minimos quadrados,
imprime os coeficientes com o r2 e gera as figuras i0_vazio.png e
curva_forca_comando.png.

Uso:  python3 analise_estatica.py
"""

#  ---------------------------------------------------------------------------

# MAPA
# 1) Constantes do projeto: conversao kgf->N, normalizacao do comando, raio efetivo (linhas 37-40)
# 2) Funcoes auxiliares: u_de, minimos quadrados (com e sem origem), leitura do vazio e da tracao (linhas 43-115)
# 3) main: ajuste de I0(u) sobre as medias do ensaio em vazio (linhas 118-127)
# 4) main: ganho G_F da curva F = G_F (I - I0(u))/u, ajustado por sentido (linhas 128-135)
# 5) main: medianas de corrente e forca por degrau, para conferencia com a tabela da monografia (linhas 136-151)
# 6) main: repetibilidade Tipo A (sigma_A) por faixa de operacao, para o orcamento de incertezas (linhas 152-171)
# 7) main: tabela do ensaio em vazio (medias por degrau) (linhas 172-176)
# 8) main: figuras i0_vazio.png e curva_forca_comando.png (linhas 177-224)
#  ---------------------------------------------------------------------------


import csv
import statistics
from pathlib import Path

import matplotlib

matplotlib.use("Agg")
import matplotlib.pyplot as plt
```

```
import openpyxl

PASTA_DADOS = Path(__file__).resolve().parent
PASTA_FIGURAS = Path(__file__).resolve().parent

G = 9.81                # conversao de kgf para N (mesmo valor da monografia)
ANGULO_NEUTRO = 95.0    # comando neutro do ESC (graus)
FAIXA_COMANDO = 85.0    # 95 a 180 graus -> u de 0 a 1
RAIO_EFETIVO_M = 0.0751


def u_de(angulo):
    """Comando normalizado u = (angulo - neutro) / faixa."""
    return (angulo - ANGULO_NEUTRO) / FAIXA_COMANDO


# Reta y = a + b x por minimos quadrados. E a versao forcada pela origem
# (y = g x), usada no ganho G_F porque a curva F = G_F (I - I0)/u nao tem
# termo independente.


def minimos_quadrados(x, y):
    """Reta y = a + b x por minimos quadrados. Devolve (a, b, r2)."""
    xm = statistics.mean(x)
    ym = statistics.mean(y)
    b = sum((xi - xm) * (yi - ym) for xi, yi in zip(x, y)) / sum((xi - xm) ** 2 for xi in x)
    a = ym - b * xm
    r2 = 1 - sum((yi - a - b * xi) ** 2 for xi, yi in zip(x, y)) / sum((yi - ym) ** 2 for yi in y)
    return a, b, r2


def minimos_quadrados_origem(x, y):
    """Reta y = g x forcada pela origem. Devolve (g, r2)."""
    g = sum(xi * yi for xi, yi in zip(x, y)) / sum(xi * xi for xi in x)
    ym = statistics.mean(y)
    r2 = 1 - sum((yi - g * xi) ** 2 for xi, yi in zip(x, y)) / sum((yi - ym) ** 2 for yi in y)
    return g, r2

```

```
70
71  def carregar_vazio():
72      """Medias da corrente de perdas por degrau, do ensaio em vazio."""
73      # O ensaio em vazio repete cada degrau varias vezes; aqui as
            leituras sao
74      # agrupadas por angulo e medias, servindo de base para o ajuste de
            I0(u).
75      por_angulo = {}
76      with open(PASTA_DADOS / "ensaio_vazio_corrente.csv") as arquivo:
77          for linha in csv.DictReader(arquivo, delimiter=";"):
78              por_angulo.setdefault(float(linha["angulo"]), []).append(
                    float(linha["corrente_a"]))
79      return {a: sum(v) / len(v) for a, v in por_angulo.items()}
80
81
82  def carregar_tracao():
83      """Pares (corrente_A, forca_N) por sentido e por degrau, das 4
            repeticoes."""
84      # A planilha de anotacoes tem as 4 repeticoes empilhadas em linhas
            ; cada
85      # par de colunas e um degrau. A forca vem em kgf e e convertida
            para N.
86      planilha = openpyxl.load_workbook(PASTA_DADOS / "
            ensaio_tracao_corrente_versus_forca.xlsx", data_only=True)
87      pagina = planilha.worksheets[0]
88      degraus = [int(pagina.cell(row=1, column=c).value) for c in range
            (2, 44, 2)]
89      tracao = {"SUBIDA": {a: [] for a in degraus}, "DESCIDA": {a: []
            for a in degraus}}
90      for linha in pagina.iter_rows(min_row=3, values_only=True):
91          if linha[0] not in tracao:
92              continue
93          for j, degrau in enumerate(degraus):
94              corrente, forca = linha[1 + 2 * j], linha[2 + 2 * j]
95              if corrente is not None and forca is not None:
96                  tracao[linha[0]][degrau].append((float(corrente),
                        float(forca) * G))
97      return tracao
98
99
100 def mediana(valores):
101     return statistics.median(valores)
102
103
104 def ajuste_gf(pares, degraus, i0_a, i0_b):
```

```
105     """G_F da reta F = G_F (I - I0(u))/u sobre os pares dos degraus
            dados."""
106     # Subtrai a corrente de perdas e divide pelo comando para
            linearizar a
107     # caracteristica; so entao a reta por minimos quadrados tem
            sentido.
108     x, y = [], []
109     for degrau in degraus:
110         u = u_de(degrau)
111         for corrente, forca in pares[degrau]:
112             x.append((corrente - (i0_a + i0_b * u)) / u)
113             y.append(forca)
114     return minimos_quadrados_origem(x, y) + (len(x),)
115
116
117 def main():
118     # 1) Corrente de perdas I0(u), ajuste sobre as medias por degrau
            do vazio
119     vazio = carregar_vazio()
120     x = [u_de(a) for a in sorted(vazio)]
121     y = [vazio[a] for a in sorted(vazio)]
122     a0, b0, r2_0 = minimos_quadrados(x, y)
123     print(f"I0(u) = {a0:.4f} + {b0:.4f} u   r2 = {r2_0:.4f}   n = {len
            (x)}")
124
125     def i0(u):
126         return a0 + b0 * u
127
128     # 2) Curva caracteristica F = G_F (I - I0(u))/u, por sentido
129     tracao = carregar_tracao()
130     gf_d, r2_d, n_d = ajuste_gf(tracao["DESCIDA"], range(100, 114), a0
            , b0)
131     gf_s, r2_s, n_s = ajuste_gf(tracao["SUBIDA"], range(100, 111), a0,
             b0)
132     print(f"descida: G_F = {gf_d:.4f} N/(A/u)   r2 = {r2_d:.4f}   n =
            {n_d}")
133     print(f"subida : G_F = {gf_s:.4f} N/(A/u)   r2 = {r2_s:.4f}   n =
            {n_s}")
134     print(f"K_T = G_F * r_ef = {gf_d * RAIO_EFETIVO_M:.4f} N*m/(A/u)")
135
136     # 3) Medianas por degrau (conferencia com a tabela da monografia)
137     # Mediana e desvio-padrao das 4 repeticoes em cada degrau,
            impressos para
138     # confronto com a tabela de resultados da monografia (descida e
            subida).
```

```
    for sentido in ("SUBIDA", "DESCIDA"):
        print(f"--- {sentido} ---")
        for degrau in sorted(tracao[sentido]):
            pares = tracao[sentido][degrau]
            if not pares:
                continue
            correntes = [p[0] for p in pares]
            forcas = [p[1] for p in pares]
            dp = statistics.stdev(correntes) if len(correntes) > 1
                else 0.0
            dpf = statistics.stdev(forcas) if len(forcas) > 1 else 0.0
            print(f"{degrau:>3} gr  I = {mediana(correntes):5.2f} +/-
                {dp:.2f} A   "
                  f"F = {mediana(forcas):5.2f} +/- {dpf:.2f} N")

    # 3a) Repetibilidade Tipo A por faixa de operacao (para o
        orcamento de incertezas)
    # A repetibilidade e avaliada separadamente na zona morta, na
        aderencia e
    # na saturacao: sao regioes com fisicas distintas. O sigma_A de
        cada faixa
    # e a media dos desvios-padrao por degrau e alimenta o orcamento
        de
    # incertezas da monografia (Tabela 4.4).
    FAIXAS = {"zona morta": range(97, 100),
              "aderencia": range(100, 111),
              "saturacao": range(111, 118)}
    for nome, faixa in FAIXAS.items():
        forcas = []
        for degrau in faixa:
            forcas += [f for c, f in tracao["SUBIDA"].get(degrau, [])]
            forcas += [f for c, f in tracao["DESCIDA"].get(degrau, [])
                ]
        dps = [statistics.stdev([f for c, f in tracao[sentido][d]])
               for sentido in ("SUBIDA", "DESCIDA") for d in faixa
               if len(tracao[sentido][d]) > 1]
        sig = sum(dps) / len(dps)
        print(f"sigma_A {nome}: medio dos desvios-padrao = {sig:.2f} N
            "
              f"(mediana da forca da faixa = {mediana(forcas):.1f} N)"
                  )

    # 3b) Tabela do ensaio em vazio (medias por degrau, para a
        monografia)
    print("--- VAZIO: medias por degrau ---")
```

```
174     for angulo in sorted(vazio):
175         print(f"{int(angulo)} gr  u = {u_de(angulo):.3f}   I0 = {vazio
                [angulo]:.2f} A")
176 
177     # 4) Figuras
178     # i0_vazio.png: corrente de perdas versus comando com o ajuste
            linear e a
179     # zona morta destacada. curva_forca_comando.png: mediana da tracao
             por
180     # degrau nos dois sentidos, com as tres faixas de operacao e os
            pontos de
181     # corte do ESC (116-117) ocos, fora da curva.
182     plt.rcParams.update({"font.size": 11.5, "axes.grid": True, "grid.
            alpha": 0.3,
183                          "axes.axisbelow": True, "axes.titlesize":
                             13})
184 
185     fig, eixo = plt.subplots(figsize=(6.4, 4.4))
186     eixo.axvspan(0.0, u_de(99.5), color="0.9")
187     eixo.text(u_de(98), 2.5, "zona morta", ha="center", fontsize=10)
188     eixo.plot(x, y, "o", color="#2166ac", ms=5.5,
189               label="Media por degrau (ensaio em vazio)")
190     u_lin = [0.0, max(x)]
191     eixo.plot(u_lin, [a0 + b0 * u for u in u_lin], "-", color="#b2182b
            ", lw=1.6,
192               label=f"$I_0 = {a0:.2f} + {b0:.2f}\\,u$  ($r^2 = {r2_0
                  :.2f}$)")
193     eixo.set_title("Corrente de perdas $I_0(u)$ no ensaio em vazio")
194     eixo.set_xlabel("Comando normalizado $u$  ($u = (\\alpha - 95)/85$
            )")
195     eixo.set_ylabel("Corrente de perdas $I_0$ (A)")
196     eixo.set_ylim(0, 2.7)
197     eixo.legend(fontsize=10.5, loc="upper left")
198     fig.tight_layout()
199     fig.savefig(PASTA_FIGURAS / "i0_vazio.png", dpi=300)
200     plt.close(fig)
201 
202     fig, eixo = plt.subplots(figsize=(6.8, 4.6))
203     cores = {"SUBIDA": "#d6604d", "DESCIDA": "#2166ac"}
204     for sentido in ("SUBIDA", "DESCIDA"):
205         degs = [a for a in sorted(tracao[sentido]) if tracao[sentido][
                a]]
206         us = [u_de(a) for a in degs]
207         fs = [mediana([p[1] for p in tracao[sentido][a]]) for a in
                degs]
```

```
208         eixo.plot(us, fs, "-", color=cores[sentido], lw=2,
209                   label=sentido.capitalize())
210     eixo.axvspan(0.0235, 0.0529, color="0.93", zorder=0)
211     eixo.axvspan(0.0529, 0.2176, color="0.975", zorder=0)
212     for limite in (0.0529, 0.2176):
213         eixo.axvline(limite, color="0.45", lw=1.1, ls=(0, (4, 3)),
                zorder=1)
214     for centro, nome in ((0.038, "zona morta"), (0.135, "aderencia"),
215                          (0.244, "saturacao")):
216         eixo.text(centro, 56.5, nome, ha="center", va="center",
                fontsize=10.5,
217                   style="italic",
218                   bbox=dict(boxstyle="round,pad=0.28", fc="white", ec=
                      "0.6", lw=0.8))
219     eixo.set_title("Forca de tracao mediana por comando normalizado")
220     eixo.set_xlabel("Comando normalizado $u$")
221     eixo.set_ylabel("Forca de tracao (N)")
222     eixo.set_ylim(0, 60)
223     eixo.set_yticks([0, 10, 20, 30, 40, 50])
224     eixo.margins(x=0.01)
225     eixo.legend(fontsize=10.5)
226     fig.tight_layout()
227     fig.savefig(PASTA_FIGURAS / "curva_forca_comando.png", dpi=300)
228     plt.close(fig)
229     print(f"figuras salvas em {PASTA_FIGURAS}")
230
231
232 if __name__ == "__main__":
233     main()
```

A.2 Dinâmica entre comando e corrente

```
1 """Identificacao da dinamica entre comando e corrente (ensaio de
      tracao de
2 corrente).
3
4 Le 'ensaio_tracao_corrente.csv', isola a sessao de aquisicao rapida
      (100 Hz),
5 ancora cada degrau da regiao de aderencia (100 a 109 graus) no seu
      inicio
6 eletrico, normaliza as respostas, ajusta a constante de tempo por
      minimos
7 quadrados sobre a media das dez bordas e gera as figuras blocos_degrau
      .png
```

```
e resposta_degrau.png.

Uso:  python3 analise_dinamica.py
"""

# ------------------------------------------------------------------------

# MAPA
# 1) Constantes do projeto: normalizacao do comando, faixa de degraus da identificacao, taxa de amostragem (linhas 36-39)
# 2) Leitura do CSV: segmento 100 Hz, quebra em blocos por degrau (linhas 46-70)
# 3) Ancoragem eletrica de cada borda: repouso, plato e inicio eletrico, mais a resposta normalizada (linhas 72-90)
# 4) Ajuste da constante de tempo tau (funcoes e bloco em main) (linhas 92-119)
# 5) main: ganhos de plato de cada borda e ganho medio (linhas 121-127)
# 6) Figuras: blocos_degrau.png (janelas brutas), blocos_degrau_<angulo>.png (janelas diversificadas) e resposta_degrau.png (linhas 130-192)
# ------------------------------------------------------------------------


import csv
import math
import statistics
from pathlib import Path

import matplotlib

matplotlib.use("Agg")
import matplotlib.pyplot as plt

PASTA_DADOS = Path(__file__).resolve().parent
PASTA_FIGURAS = Path(__file__).resolve().parent

ANGULO_NEUTRO = 95.0
FAIXA_COMANDO = 85.0
DEGRAU_MINIMO, DEGRAU_MAXIMO = 100, 109   # bordas usadas na identificacao
AMOSTRAS_POR_SEGUNDO = 100

```

```
41
42 def u_de(angulo):
43     return (angulo - ANGULO_NEUTRO) / FAIXA_COMANDO
44
45
46 def carregar_sessao_rapida():
47     """Segmento 100 Hz do CSV: o mais longo, com um bloco por degrau.
           """
48     # O CSV concatena varias sessoes de aquisicao; o timestamp que
           reseta marca
49     # a fronteira entre elas. A sessao mais longa e a de 100 Hz, e
           nela cada
50     # trecho de comando constante vira um bloco (um degrau).
51     leituras = []
52     with open(PASTA_DADOS / "ensaio_tracao_corrente.csv") as arquivo:
53         for linha in csv.DictReader(arquivo, delimiter=";"):
54             leituras.append((float(linha["t_ensaio_s"]),
55                              float(linha["angulo"]),
56                              float(linha["corrente_a"])))
57     reseta = [0] + [i for i in range(1, len(leituras))
58                     if leituras[i][0] < leituras[i - 1][0]] + [len(
                           leituras)]
59     segmentos = [leituras[reseta[k]:reseta[k + 1]] for k in range(len(
           reseta) - 1)]
60     sessao = max(segmentos, key=len)
61     blocos = []
62     bloco = [sessao[0]]
63     for leitura in sessao[1:]:
64         if leitura[1] != bloco[-1][1]:
65             blocos.append(bloco)
66             bloco = []
67         bloco.append(leitura)
68     blocos.append(bloco)
69     return blocos
70
71
72 def iniciar_em(bloco):
73     """Ancora o bloco no inicio eletrico: primeira amostra acima de
           20% da excursao."""
74     # O inicio eletrico (quando a corrente comeca a subir) nao
           coincide com o
75     # inicio da janela de comando; ancorar na primeira amostra acima
           de 20% da
76     # excursao repouso->plato alinha as dez bordas para a media e o
           ajuste.
```

```
77      correntes = [leitura[2] for leitura in bloco]
78      repouso = statistics.mean(correntes[:150])       # 1,5 s iniciais (neutro)
79      plato = statistics.mean(correntes[-300:])        # 3 s finais (regime)
80      alvo = repouso + 0.2 * (plato - repouso)
81      inicio = next(i for i, c in enumerate(correntes) if c > alvo)
82      return correntes, repouso, plato, inicio
83
84
85  def resposta_normalizada(bloco):
86      """Trecho de 6 s a partir do inicio eletrico, normalizado entre 0 e 1."""
87      correntes, repouso, plato, inicio = iniciar_em(bloco)
88      trecho = correntes[inicio:inicio + 6 * AMOSTRAS_POR_SEGUNDO]
89      return [(c - repouso) / (plato - repouso) for c in trecho], repouso, plato
90
91
92  def ajustar_tau(media):
93      """Minimiza a soma dos quadrados dos residuos de y = 1 - exp(-t/tau)."""
94      # Ajuste direto da resposta de primeira ordem sobre a media das dez bordas,
95      # por varredura de tau; serve de contraste ao metodo grafico de 63,2%.
96      def residuo(tau):
97          return sum((y - (1 - math.exp(-i / AMOSTRAS_POR_SEGUNDO / tau))) ** 2
98                     for i, y in enumerate(media))
99      return min(range(5, 1000, 5), key=lambda t: residuo(t / 1000)) / 1000
100
101
102 def media_movel(serie, janela):
103     return [statistics.mean(serie[max(0, i - janela + 1):i + 1])
104             for i in range(len(serie))]
105
106
107 def main():
108     blocos = carregar_sessao_rapida()
109     usados = [b for b in blocos if DEGRAU_MINIMO <= b[0][1] <= DEGRAU_MAXIMO]
110     print(f"bordas usadas: {[int(b[0][1]) for b in usados]}")
111
```

```
    # constante de tempo sobre a media das respostas normalizadas
    respostas = [resposta_normalizada(b) for b in usados]
    normais = [r[0] for r in respostas]
    n = min(len(r) for r in normais)
    media = [statistics.mean(r[i] for r in normais) for i in range(n)]
    tau = ajustar_tau(media)
    tau_632 = next(i for i, v in enumerate(media) if v >= 0.632) /
        AMOSTRAS_POR_SEGUNDO
    print(f"tau (minimos quadrados) = {tau:.2f} s | tau (63,2%) = {
        tau_632:.2f} s")

    # ganho de cada borda: plato por unidade de comando
    ganhos = []
    for bloco, (normalizada, repouso, plato) in zip(usados, respostas)
        :
        angulo = int(bloco[0][1])
        ganho = plato / u_de(angulo)
        ganhos.append(ganho)
        print(f"{angulo} gr: plato = {plato:.2f} A | ganho = {ganho:.0
            f} A/u")
    print(f"ganho medio das dez bordas = {statistics.mean(ganhos):.1f}
        A/u")

    # figura 1: janelas brutas de dois degraus, com media movel
    plt.rcParams.update({"font.size": 11, "axes.grid": True, "grid.
        alpha": 0.3,
                         "axes.axisbelow": True, "axes.titlesize":
                            12})
    fig, eixos = plt.subplots(1, 2, figsize=(9.4, 3.8), sharey=True)
    for eixo, bloco in zip(eixos, (usados[0], usados[6])):
        correntes, _, _, _ = iniciar_em(bloco)
        tempo = [i / AMOSTRAS_POR_SEGUNDO for i in range(len(correntes
            ))]
        eixo.plot(tempo, correntes, color="0.75", lw=0.8, label="
            Corrente bruta (100 Hz)")
        eixo.plot(tempo, media_movel(correntes, 50), color="#b2182b",
            lw=1.6,
                  label="Media movel (0,5 s)")
        eixo.set_xlabel("Tempo desde o inicio da janela (s)")
        eixo.set_title(f"Degrau de comando {int(bloco[0][1])} gr",
            fontsize=12)
        eixo.set_xlim(0, 8)
    eixos[0].set_ylabel("Corrente (A)")
    eixos[0].legend(fontsize=10)
```

```
145     fig.suptitle("Janelas brutas de corrente nos degraus de comando (
            ensaio a 100 Hz)",
146                  fontsize=13)
147     fig.tight_layout(rect=(0, 0, 1, 0.93))
148     fig.savefig(PASTA_FIGURAS / "blocos_degrau.png", dpi=300)
149     plt.close(fig)
150 
151     # figura 2: respostas alinhadas, media e ajuste de primeira ordem
152     fig, eixo = plt.subplots(figsize=(6.4, 4.4))
153     tempo = [i / AMOSTRAS_POR_SEGUNDO for i in range(n)]
154     for normalizada, _, _ in respostas:
155         eixo.plot(tempo[:len(normalizada)], normalizada, color="0.8",
                lw=0.9)
156     eixo.plot(tempo, media, color="#2166ac", lw=2.0, label="Media das
            dez bordas")
157     ajuste = [1 - math.exp(-t / tau) for t in tempo]
158     eixo.plot(tempo, ajuste, "--", color="#b2182b", lw=1.8,
159               label=f"Ajuste de 1.a ordem: $1-e^{{-t/{tau:.2f}}}$")
160     eixo.plot(tau, 1 - math.exp(-1), "o", color="#b2182b")
161     eixo.annotate("$\\tau$", (tau, 0.632), textcoords="offset points",
162                   xytext=(8, -4), fontsize=12)
163     eixo.set_title("Resposta normalizada das dez bordas: media e
            ajuste de 1.a ordem")
164     eixo.set_xlabel("Tempo desde o inicio eletrico (s)")
165     eixo.set_ylabel("Resposta normalizada")
166     eixo.set_xlim(0, 2.5)
167     eixo.legend(fontsize=10.5)
168     fig.tight_layout()
169     # figuras 1b e 1c: janela bruta de um degrau no inicio e no fim da
            regiao
170     for indice in (3, 9):
171         bloco = usados[indice]
172         angulo = int(bloco[0][1])
173         correntes, _, _, _ = iniciar_em(bloco)
174         tempo = [i / AMOSTRAS_POR_SEGUNDO for i in range(len(correntes
                ))]
175         fig, eixo = plt.subplots(figsize=(6.4, 3.6))
176         eixo.plot(tempo, correntes, color="0.75", lw=0.8, label="
                Corrente bruta (100 Hz)")
177         eixo.plot(tempo, media_movel(correntes, 50), color="#b2182b",
                lw=1.6,
178                   label="Media movel (0,5 s)")
179         eixo.set_xlabel("Tempo desde o inicio da janela (s)")
180         eixo.set_ylabel("Corrente (A)")
181         eixo.set_xlim(0, 8)
```

```
182         eixo.set_title(f"Janela bruta do degrau de comando {angulo}
                graus (ensaio a 100 Hz)",
183                        fontsize=12)
184         eixo.legend(fontsize=10)
185         fig.tight_layout()
186         fig.savefig(PASTA_FIGURAS / f"blocos_degrau_{angulo}.png", dpi
                =300)
187         plt.close(fig)
188
189     fig.savefig(PASTA_FIGURAS / "resposta_degrau.png", dpi=300)
190     plt.close(fig)
191     print(f"figuras salvas em {PASTA_FIGURAS}")
192
193
194 if __name__ == "__main__":
195     main()
```

A.3 Simulação da malha de controle

```
1  """Simulacao da cascata de controle de torque do automodelo (PSR / PRP
       / seletor).
2
3  Le a caracterizacao estatica (ensaio_tracao_corrente_versus_forca.xlsx
       (pasta reprodutibilidade)), ajusta a planta nao-linear
4  da corrente e o ganho de torque diretamente dos dados (sem reducao i,
       sem K_t),
5  executa a simulacao da cascata mestre (torque estimado) / escrava (
       corrente) com
6  todas as nao-linearidades e salvaguardas do projeto, e gera a figura
7  PFC_monografia/Figuras/simulacao_cascata.png.
8
9  Uso:  python3 simulacao_cascata.py
10 Saida: figura PNG + metricas PSR/PRP impressas no terminal.
11 """
12
13 #
       ------------------------------------------------------------------------
14 # MAPA
15 # 1) Constantes do projeto: ajuste da planta, sintese IMC,
       salvaguardas e cenario de simulacao (linhas 30-74)
16 # 2) Leitura e ajuste dos dados de ensaio: repeticoes da descida, I0(u
       ), planta I_ss(u), ganho de torque, comando equivalente (linhas
       76-160)
```

```
# 3) Simulacao da cascata: ganhos IMC, cenario PSR/PRP/seletor, estimador, seletor, mestre, caminho direto com feedforward, PI escravo com clamping, planta (linhas 163-261)
# 4) Metricas PSR/PRP/retorno/seletor e figura simulacao_cascata.png (linhas 264-372)
# ------------------------------------------------------------------------------


from pathlib import Path

import matplotlib

matplotlib.use("Agg")
import matplotlib.pyplot as plt
import numpy as np
import openpyxl

# ------------------------------------------------------------------------------
# Constantes do projeto (identificadas nos ensaios)
# ------------------------------------------------------------------------------
PASTA_ENSAIOS = Path(__file__).resolve().parent
PASTA_FIGURAS = Path(__file__).resolve().parent
# (na pasta reprodutibilidade, as figuras saem junto aos scripts)

PLANILHA_DADOS = PASTA_ENSAIOS / "ensaio_tracao_corrente_versus_forca.xlsx"
ABA_PLANILHA = "P\xe1gina1"

DEGRAU_MINIMO = 100          # faixa de ajuste: regiao de aderencia (descida)
DEGRAU_MAXIMO = 113
ANGULO_NEUTRO_GRAUS = 95.0   # u = (angulo - neutro) / faixa_total
FAIXA_COMANDO = 85.0
RAIO_EFETIVO_M = 0.0751      # roll-out (metodologia)

CONSTANTE_TEMPO_PLANTA_S = 0.18   # dinamica comando->corrente (ensaio de tracao de corrente)

# Controladores (sintese IMC, conforme planejamento do controle)
```

```
49 LAMBDA_ESCRAVA_S = 0.18            # malha de corrente: tao rapida
       quanto a planta
50 LAMBDA_MESTRE_S = 1.1              # regra da cascata: 3-10x mais lenta
       (6x)
51 TEMPO_INTEGRAL_ESCRAVA_S = CONSTANTE_TEMPO_PLANTA_S
52 TEMPO_INTEGRAL_MESTRE_S = 0.18
53 TEMPO_AMOSTRAGEM_S = 0.01          # firmware rapido (<= Ti/10)
54
55 # Salvaguardas (cada uma justificada por um modo de falha, ver
       planejamento)
56 COMANDO_MIN = 0.045                # limite inferior: saida da zona
       morta
57 COMANDO_MAX = 0.21                 # fronteira da regiao de aderencia
       validada
58 TORQUE_MAXIMO_SELETOR_NM = 3.3     # limiar de aderencia (forca max. x
       r_ef)
59 TORQUE_MAXIMO_ESTIMADOR_NM = 3.35 # estimador valido so onde foi
       caracterizado
60 RAMPA_CAMINHO_DIRETO_U_POR_S = 0.05
61 RAMPA_REFERENCIA_MESTRE_NM_POR_S = 0.6
62 CONSTANTE_FILTRO_ESTIMADOR_S = 0.6
63
64 # Cenario de simulacao
65 TEMPO_FINAL_S = 64.0
66 TORQUE_INICIAL_NM = 1.6
67 TORQUE_PSR_NM = 3.0
68 TORQUE_SELETOR_NM = 4.5
69 INSTANTE_PSR_S, INSTANTE_PRP_S = 10.0, 24.0
70 DURACAO_PRP_S = 6.0                 # sag + deslocamento de I0 entre 24
       e 30 s
71 INSTANTE_RETORNO_S, INSTANTE_SELETOR_S = 38.0, 48.0
72 QUEDA_GANHO_PRP = 0.8              # sag de 20% na planta
73 DESLOCAMENTO_I0_PRP_A = 0.3
74
75
76 #
       ------------------------------------------------------------------------
       #
77 # Leitura e ajuste dos dados de ensaio
78 #
       ------------------------------------------------------------------------
       #
79 def carregar_repeticoes_descida(caminho_planilha):
80     """Retorna dicionario {degrau: (correntes_A, forcas_kgf)} das
           repeticoes."""
```

```
81      # So a descida e usada: e o sentido em que a caracteristica foi
            validada
82      # ate a saturacao, sem os pontos de corte do ESC que contaminam a
            subida.
83      planilha = openpyxl.load_workbook(caminho_planilha, data_only=True
            )
84      pagina = planilha[ABA_PLANILHA]
85      linhas = list(pagina.iter_rows(values_only=True))
86      cabecalho = linhas[0]
87      degraus = [int(cabecalho[i]) for i in range(1, 44, 2) if cabecalho
            [i] is not None]
88      repeticoes = []
89      for linha in linhas[2:]:
90          if linha[0] != "DESCIDA":
91              continue
92          pares = {}
93          for j, degrau in enumerate(degraus):
94              corrente, forca = linha[1 + 2 * j], linha[2 + 2 * j]
95              if corrente is not None and forca is not None:
96                  pares[degrau] = (float(corrente), float(forca))
97          repeticoes.append(pares)
98      return repeticoes
99
100
101 def comando_normalizado(degrau):
102     """u = (angulo - neutro) / faixa, conforme a metodologia."""
103     return (degrau - ANGULO_NEUTRO_GRAUS) / FAIXA_COMANDO
104
105
106 # Corrente de perdas I0(u) = I0_A + I0_B u, ajustada no ensaio em
        vazio
107 # (Apendice A, analise_estatica.py)
108 I0_A = 0.53
109 I0_B = 6.70
110
111
112 def corrente_perdas_a(u):
113     """I0(u) ajustado no ensaio em vazio: I0 = 0,53 + 6,70 u."""
114     return I0_A + I0_B * u
115
116
117 def ajustar_planta_corrente(repeticoes):
118     """Planta estatica I_ss(u): quadratica sobre a descida, regiao de
            aderencia."""
```

```
119     # A corrente de regime nao e linear no comando (ganho cai na
            saturacao);
120     # um ajuste quadratico sobre a regiao de aderencia captura essa
            curvatura
121     # com os dados brutos das repeticoes, sem reducao intermediaria.
122     comodos, correntes = [], []
123     for rep in repeticoes:
124         for degrau in range(DEGRAU_MINIMO, DEGRAU_MAXIMO + 1):
125             if degrau in rep:
126                 comodos.append(comando_normalizado(degrau))
127                 correntes.append(rep[degrau][0])
128     coeficientes = np.polyfit(np.array(comodos), np.array(correntes),
            2)
129     return coeficientes  # I_ss(u) = c2 u^2 + c1 u + c0
130
131
132 def ajustar_ganho_torque(repeticoes):
133     """K_T da relacao F = G_F (I - I0(u))/u, ajustada pela origem (sem
            i, sem K_t).
134
135     Retorna K_T = G_F * r_ef, em N*m por (A/u).
136     """
137     eixo_x, eixo_y = [], []
138     for rep in repeticoes:
139         for degrau in range(DEGRAU_MINIMO, DEGRAU_MAXIMO + 1):
140             if degrau in rep:
141                 u = comando_normalizado(degrau)
142                 corrente, forca_kgf = rep[degrau]
143                 eixo_x.append((corrente - corrente_perdas_a(u)) / u)
144                 eixo_y.append(forca_kgf * 9.81)
145     eixo_x, eixo_y = np.array(eixo_x), np.array(eixo_y)
146     ganho_forca = float((eixo_x * eixo_y).sum() / (eixo_x * eixo_x).
            sum())
147     return ganho_forca * RAIO_EFETIVO_M
148
149
150 def comando_equivalente(torque_desejado, coef_planta, ganho_torque):
151     """Inverte a curva estatica: u tal que K_T (I_ss(u) - I0(u))/u =
            torque."""
152     # A inversao analitica (raiz do polinomio) e o caminho direto do
153     # feedforward: posiciona o comando no ponto de equilibrio do
            torque
154     # pedido, deixando ao PI escravo apenas o desvio.
155     c2, c1, c0 = coef_planta
```

```
156     coeficientes = [c2, (c1 - I0_B) - torque_desejado / ganho_torque,
            c0 - I0_A]
157     raizes = np.roots(coeficientes)
158     validas = [r.real for r in raizes
159                if np.isreal(r) and COMANDO_MIN - 0.015 < r.real < 0.5]
160     return min(validas) if validas else COMANDO_MIN
161
162
163 #
        ----------------------------------------------------------------------
        #
164 # Simulacao da cascata
165 #
        ----------------------------------------------------------------------
        #
166 # Passo a passo de 10 ms, com o cenario PSR/PRP/seletor injetado como
        funcoes
167 # do tempo e todas as salvaguardas do projeto ativas. A ordem de cada
        passo e:
168 # estimador, seletor+rampa, mestre, caminho direto com feedforward,
        escravo com
169 # clamping e, por fim, a planta.
170
171
172 def simular_cascata(coef_planta, ganho_torque):
173     c2, c1, c0 = coef_planta
174     ganho_local = 2 * c2 * comando_normalizado(105) + c1   # dI/du no
            nominal
175     kp_escrava = CONSTANTE_TEMPO_PLANTA_S / (ganho_local *
            LAMBDA_ESCRAVA_S)
176     kp_mestre = lambda u: (CONSTANTE_TEMPO_PLANTA_S * u
177                            / (ganho_torque * LAMBDA_MESTRE_S))
178
179     n_passos = int(TEMPO_FINAL_S / TEMPO_AMOSTRAGEM_S)
180     tempos = np.arange(n_passos) * TEMPO_AMOSTRAGEM_S
181
182     def referencia_pedida(tempo):
183         if tempo < INSTANTE_PSR_S:
184             return TORQUE_INICIAL_NM
185         if tempo < INSTANTE_RETORNO_S:
186             return TORQUE_PSR_NM
187         if tempo < INSTANTE_SELETOR_S:
188             return TORQUE_INICIAL_NM
189         return TORQUE_SELETOR_NM
190
```

```
191     def perturbacao_presente(tempo):
192         em_prp = INSTANTE_PRP_S <= tempo < INSTANTE_PRP_S +
                DURACAO_PRP_S
193         return (QUEDA_GANHO_PRP if em_prp else 1.0,
194                 DESLOCAMENTO_I0_PRP_A if em_prp else 0.0)
195
196     # Estado inicial consistente (ponto de equilibrio do torque
            inicial)
197     comando = comando_equivalente(TORQUE_INICIAL_NM, coef_planta,
            ganho_torque)
198     corrente = float(np.polyval(coef_planta, comando))
199     integral_mestre, integral_escrava = 0.0, 0.0
200     comando_direto = comando
201     torque_filtrado = TORQUE_INICIAL_NM
202     referencia_mestre = TORQUE_INICIAL_NM
203
204     historico = {chave: np.zeros(n_passos) for chave in
205                  ("torque", "corrente", "comando", "referencia", "
                         referencia_mestre")}
206
207     for passo in range(n_passos):
208         tempo = tempos[passo]
209         comando_efetivo = max(comando, COMANDO_MIN)
210         fator_ganho, deslocamento_i0 = perturbacao_presente(tempo)
211
212         # estimador de torque (quase-estatico, limitado a faixa
                validada)
213         torque_bruto = ganho_torque * (corrente - corrente_perdas_a(
                comando_efetivo)
214                                        - deslocamento_i0) /
                                           comando_efetivo
215         torque_estimado = min(max(torque_bruto, 0.0),
                TORQUE_MAXIMO_ESTIMADOR_NM)
216         torque_filtrado += ((torque_estimado - torque_filtrado)
217                             * TEMPO_AMOSTRAGEM_S /
                                CONSTANTE_FILTRO_ESTIMADOR_S)
218
219         # seletor de aderencia + filtro de referencia do mestre
220         referencia = min(referencia_pedida(tempo),
                TORQUE_MAXIMO_SELETOR_NM)
221         passo_rampa = RAMPA_REFERENCIA_MESTRE_NM_POR_S *
                TEMPO_AMOSTRAGEM_S
222         referencia_mestre += min(max(referencia - referencia_mestre, -
                passo_rampa),
223                                  passo_rampa)
```

```
224
225         # controlador mestre (torque -> correcao de corrente)
226         erro_mestre = referencia_mestre - torque_filtrado
227         correcao_corrente = kp_mestre(comando_efetivo) * (erro_mestre
                + integral_mestre)
228
229         # caminho direto: comando equivalente (com rampa) +
                feedforward de corrente
230         passo_direto = RAMPA_CAMINHO_DIRETO_U_POR_S *
                TEMPO_AMOSTRAGEM_S
231         alvo_direto = comando_equivalente(referencia, coef_planta,
                ganho_torque)
232         comando_direto += min(max(alvo_direto - comando_direto, -
                passo_direto),
233                               passo_direto)
234         corrente_referencia = (corrente_perdas_a(comando_efetivo)
235                                + comando_efetivo * referencia /
                                    ganho_torque
236                                + correcao_corrente)
237
238         # controlador escravo (corrente -> comando), com anti-windup
                por clamping
239         erro_escrava = corrente_referencia - corrente
240         comando_nao_limitado = comando_direto + kp_escrava * (
                erro_escrava + integral_escrava)
241         comando = min(max(comando_nao_limitado, COMANDO_MIN),
                COMANDO_MAX)
242         saturado = abs(comando - comando_nao_limitado) > 1e-9
243         if saturado:
244             # clamping: ajusta o integral para a saida ficar
                    exatamente no limite
245             integral_escrava = (comando - comando_direto) / kp_escrava
                    - erro_escrava
246         else:
247             integral_escrava += erro_escrava * TEMPO_AMOSTRAGEM_S /
                    TEMPO_INTEGRAL_ESCRAVA_S
248             integral_mestre += erro_mestre * TEMPO_AMOSTRAGEM_S /
                    TEMPO_INTEGRAL_MESTRE_S
249
250         # planta nao-linear da corrente (curva estatica + 1a ordem)
251         corrente_estatica = max(float(np.polyval(coef_planta,
                comando_efetivo)), 0.0)
252         corrente += ((corrente_estatica * fator_ganho - corrente)
253                      / CONSTANTE_TEMPO_PLANTA_S) * TEMPO_AMOSTRAGEM_S
254
```

```
255         historico["torque"][passo] = torque_estimado
256         historico["corrente"][passo] = corrente
257         historico["comando"][passo] = comando
258         historico["referencia"][passo] = referencia_pedida(tempo)
259         historico["referencia_mestre"][passo] = referencia_mestre
260
261     return tempos, historico
262
263
264 # ------------------------------------------------------------------------
    #
265 # Metricas e figura
266 # ------------------------------------------------------------------------
    #
267 # As metricas leem o torque estimado nos trechos de cada transicao (
    media de
268 # 0,5 s antes do instante indicado) e alimentam os resultados PSR/PRP
    da
269 # monografia.
270
271
272 def amostra_media(vetor, tempo, alvo_s, janela_s=0.5):
273     fim = int(alvo_s / TEMPO_AMOSTRAGEM_S)
274     inicio = fim - int(janela_s / TEMPO_AMOSTRAGEM_S)
275     return vetor[inicio:fim].mean()
276
277
278 def imprimir_metricas(tempos, historico):
279     torque = historico["torque"]
280     i_psr = int(INSTANTE_PSR_S / TEMPO_AMOSTRAGEM_S)
281     i_prp = int(INSTANTE_PRP_S / TEMPO_AMOSTRAGEM_S)
282     i_fim_prp = int((INSTANTE_PRP_S + DURACAO_PRP_S) /
            TEMPO_AMOSTRAGEM_S)
283
284     trecho_psr = torque[i_psr:i_prp]
285     amplitude = TORQUE_PSR_NM - TORQUE_INICIAL_NM
286     pico = trecho_psr.max()
287     print(f"PSR {TORQUE_INICIAL_NM}->{TORQUE_PSR_NM} N.m: "
288           f"pico={pico:.2f} N.m (Mp={max((pico - TORQUE_PSR_NM) /
                amplitude * 100, 0):.0f}%) | "
289           f"T({INSTANTE_PSR_S + 5.5:.1f}s)={amostra_media(torque,
                tempos, INSTANTE_PSR_S + 5.5):.3f} N.m")
290     print(f"PRP (sag 20% + I0 +0,3 A): "
```

```
291             f"desvio max={abs(torque[i_prp:i_fim_prp].min() -
                    TORQUE_PSR_NM):.2f} N.m | "
292             f"regime={amostra_media(torque, tempos, INSTANTE_PRP_S +
                    DURACAO_PRP_S + 1.5):.3f} N.m (ref {TORQUE_PSR_NM})")
293     print(f"retorno: T({INSTANTE_RETORNO_S + 5.5:.1f}s)={amostra_media
            (torque, tempos, INSTANTE_RETORNO_S + 5.5):.3f} N.m "
294             f"(ref {TORQUE_INICIAL_NM})")
295     print(f"seletor: T({INSTANTE_SELETOR_S + 13.0:.1f}s)={
            amostra_media(torque, tempos, INSTANTE_SELETOR_S + 13.0):.2f}
            N.m "
296             f"(limite {TORQUE_MAXIMO_SELETOR_NM})")
297
298
299 def gerar_figura(tempos, historico, caminho_saida):
300     # Tres eixos empilhados (torque, corrente, comando) ao longo de 64
            s, com as
301     # janelas de cada cenario sombreadas: cada degrau so e aplicado
            apos o regime
302     # permanente do cenario anterior.
303     plt.rcParams.update({"font.size": 12, "axes.grid": True,
304                          "grid.alpha": 0.25, "axes.axisbelow": True,
305                          "axes.titlesize": 14, "legend.framealpha":
                                0.95})
306     fig, eixos = plt.subplots(3, 1, figsize=(10.2, 7.6), sharex=True)
307
308     janelas = [
309         (INSTANTE_PSR_S, INSTANTE_PRP_S, "#d1e5f0", "PSR $1{,}6 \\
                rightarrow 3{,}0$"),
310         (INSTANTE_PRP_S, INSTANTE_PRP_S + DURACAO_PRP_S, "#f4a582",
311          "PRP: sag $-20\\%$"),
312         (INSTANTE_RETORNO_S, INSTANTE_SELETOR_S, "#d9f0d3",
313          "retorno $3{,}0 \\rightarrow 1{,}6$"),
314         (INSTANTE_SELETOR_S, TEMPO_FINAL_S, "#e0d4e8",
315          "seletor: pedido $4{,}5$"),
316     ]
317     for eixo in eixos:
318         for inicio_j, fim_j, cor, _ in janelas:
319             eixo.axvspan(inicio_j, fim_j, color=cor, alpha=0.55,
                    zorder=0)
320
321     eixos[0].plot(tempos, historico["referencia"], "k--", lw=1.2,
322                   label="$T_\\mathrm{ref}$ pedida")
323     eixos[0].plot(tempos, historico["torque"], color="#2166ac", lw
            =1.8,
324                   label="$\\hat{T}$ (torque estimado)")
```

```
325     eixos[0].axhline(TORQUE_MAXIMO_SELETOR_NM, color="#b2182b", ls=":"
            , lw=1.2)
326     eixos[0].text(0.6, TORQUE_MAXIMO_SELETOR_NM + 0.12,
327                   "limiar do seletor (3,3 N.m)", fontsize=9.5, color="
                       #b2182b")
328     eixos[0].set_ylabel("Torque (N.m)")
329     eixos[0].set_title("Simulacao da cascata de torque: cenario PSR /
            PRP / retorno / seletor")
330     eixos[0].set_ylim(0.8, 5.2)
331     eixos[0].legend(fontsize=10.5, loc="center right")
332
333     i_pico = int(np.argmax(historico["torque"][:int(INSTANTE_PRP_S /
            TEMPO_AMOSTRAGEM_S)]))
334     eixos[0].annotate(f"pico {historico['torque'][i_pico]:.2f} N.m\n(
            Mp = 25%)",
335                       xy=(tempos[i_pico], historico["torque"][i_pico])
                              ,
336                       xytext=(tempos[i_pico] + 3.0, 4.05),
337                       fontsize=9.5, ha="left",
338                       arrowprops=dict(arrowstyle="->", lw=0.9, color="
                           0.25"))
339     i_prp = int((INSTANTE_PRP_S + 1.0) / TEMPO_AMOSTRAGEM_S)
340     desvio = abs(historico["torque"][int(INSTANTE_PRP_S /
            TEMPO_AMOSTRAGEM_S)
341                                      :int((INSTANTE_PRP_S +
                                             DURACAO_PRP_S)
342                                               / TEMPO_AMOSTRAGEM_S)].min() -
                                                   TORQUE_PSR_NM)
343     eixos[0].annotate(f"desvio max.\n{desvio:.2f} N.m",
344                       xy=(tempos[i_prp], historico["torque"][i_prp]),
345                       xytext=(INSTANTE_PRP_S + 0.4, 2.1),
346                       fontsize=9.5, ha="left",
347                       arrowprops=dict(arrowstyle="->", lw=0.9, color="
                           0.25"))
348
349     eixos[1].plot(tempos, historico["corrente"], color="#b2182b", lw
            =1.4,
350                   label="Corrente $I$")
351     eixos[1].set_ylabel("Corrente (A)")
352     eixos[1].legend(fontsize=10.5, loc="center right")
353
354     eixos[2].plot(tempos, historico["comando"], color="#1b7837", lw
            =1.4,
355                   label="Comando $u$")
356     eixos[2].axhline(COMANDO_MAX, color="0.6", ls=":", lw=0.9)
```

```
357     eixos[2].set_ylabel("Comando $u$")
358     eixos[2].set_xlabel("Tempo (s)")
359     eixos[2].set_xlim(0, TEMPO_FINAL_S)
360     eixos[2].legend(fontsize=10.5, loc="center right")
361 
362     for inicio_j, fim_j, _, nome in janelas:
363         eixos[0].text((inicio_j + fim_j) / 2, 5.0, nome, ha="center",
                va="top",
364                       fontsize=10, style="italic")
365     for eixo in eixos:
366         for inicio_j, fim_j, _, _ in janelas:
367             eixo.axvline(inicio_j, color="0.5", lw=0.9, ls=(0, (4, 3))
                    , zorder=1)
368 
369     fig.tight_layout()
370     caminho_saida.parent.mkdir(parents=True, exist_ok=True)
371     fig.savefig(caminho_saida, dpi=300)
372     plt.close(fig)
373     print(f"figura salva em {caminho_saida}")
374 
375 
376 def main():
377     # Pipeline: ajusta planta e ganho a partir das repeticoes da
            descida,
378     # simula a cascata, imprime as metricas e gera a figura.
379     repeticoes = carregar_repeticoes_descida(PLANILHA_DADOS)
380     coef_planta = ajustar_planta_corrente(repeticoes)
381     ganho_torque = ajustar_ganho_torque(repeticoes)
382     print(f"planta I_ss(u) = {coef_planta[0]:.1f} u^2 + {coef_planta
            [1]:.1f} u "
383           f"{coef_planta[2]:.2f} | K_T = {ganho_torque:.4f} N.m/(A/u)"
                )
384 
385     tempos, historico = simular_cascata(coef_planta, ganho_torque)
386     imprimir_metricas(tempos, historico)
387     gerar_figura(tempos, historico, PASTA_FIGURAS / "simulacao_cascata
            .png")
388 
389 
390 if __name__ == "__main__":
391     main()
```

A.4 Automação do ensaio

```
"""Modo robusto/arcaico: dispara o throttle SEM confirmacao de rede e detecta a
resposta ELETRICAMENTE, na corrente da serial do Arduino.

Para quando a rede do Pi cai nos degraus altos (EMI/brown-out): o unico canal
confiavel e a serial local. A janela de cada degrau comeca no instante em que a
corrente responde, nao quando o Pi confirma. A rede, quando viva, so serve para
limpar processos orfaos (pkill) antes de cada disparo.

Uso: python3 rodar_ensaio_cego.py

Gera os dados do ensaio de tracao de corrente (sem anotacao de forca).
"""

# ---------------------------------------------------------------------------

# MAPA
# 1) Configuracao do ensaio: faixa de degraus, direcao, limiares eletricos, CSV de saida (linhas 35-45)
# 2) le_corrente e aguardar_repouso: leitura da serial e sincronizacao eletrica entre degraus (linhas 48-64)
# 3) disparar_e_medir: disparo fogo-e-esqueca, deteccao da resposta na corrente e gravacao da janela (linhas 66-153)
# 4) main: escada de degraus, lock de instancia unica, abertura da serial e loop do ensaio (linhas 156-190)
# ---------------------------------------------------------------------------


import csv
import os
import time
from collections import deque
from serial import Serial

from rodar_ensaio import (BAUD, DURACAO_S, INICIO_JANELA_S,
                          LOCK_PATH, MM_SEGUNDOS, NEUTRO_GRAUS, PORTA_SERIAL,
                          enviar_comando, extrair_corrente, garantir_neutro,
```

```
31                          instancia_unica, limpar_pi, log, media,
32                          medir_latencia, rede_acordada, resumo_degrau
                                )
33
34  # ------------------------------------------------------------------
        config
35  ANGULO_INICIAL = 98
36  ANGULO_FINAL = 114
37  DIRECAO = "descida"             # "subida" | "descida"
38  NOME_CSV = "ensaio_cego_descida.csv"
39  LIMIAR_RESPOSTA_A = 0.3         # I acima disso = throttle respondeu
        (96-98 da
40                                  # ~0,5-1,5 A; ruido de repouso e +/-0,15
                                        A)
41  LIMIAR_REPOUSO_A = 0.5          # |I| abaixo disso = repouso entre
        degraus
42  TENTATIVAS_CEGAS = 8            # disparos por degrau (cobre reboot ~60
        s do Pi)
43  ESPERA_RESPOSTA_S = 10          # janela para a corrente responder apos
        cada disparo
44  PAUSA_ENTRE_DEGRAUS_S = 5
45  CABECALHO_CSV = "t_ensaio_s;t_degrau_s;angulo;corrente_a"
46
47
48  def le_corrente(ser):
49      # Uma linha da serial do Arduino = uma leitura de corrente; a
            decodificacao
50      # tolerante (errors="replace") aguenta bytes quebrados quando a
            rede cai.
51      return extrair_corrente(ser.readline().decode(errors="replace"))
52
53
54  def aguardar_repouso(ser, timeout_s=20):
55      """Espera a corrente ficar abaixo do limiar (fim do degrau
            anterior)."""
56      # A sincronizacao entre degraus e eletrica, nao de rede: so se
            dispara o
57      # proximo angulo quando o motor voltou ao repouso.
58      fim = time.time() + timeout_s
59      while time.time() < fim:
60          i = le_corrente(ser)
61          if i is not None and abs(i) < LIMIAR_REPOUSO_A:
62              return True
63      return False
64
```

```
65
66 def disparar_e_medir(ser, csv_writer, angulo, t_ensaio0,
       gravar_por_segundo=False):
67     """Dispara (fogo-e-esqueca) ate a corrente responder; mede a
           janela a
68     partir da resposta. Retorna True se o degrau foi executado.
69
70     gravar_por_segundo=False: CSV recebe TODA leitura (~100 Hz).
71     gravar_por_segundo=True : CSV recebe so a media de cada segundo
72                               (15 pontos por degrau; a aquisicao
                                      continua
73                               a 100 Hz e o resumo usa todas as
                                      leituras)."""
74     # Como nao ha confirmacao de rede, o degrau so "aconteceu" quando
           a
75     # corrente passa do limiar; ate la, re-dispara ate
           TENTATIVAS_CEGAS vezes
76     # (o Pi pode estar reiniciando por brown-out). A janela comeca na
           resposta,
77     # nao no disparo, e o resumo por segundo (com media movel) vai
           para o
78     # terminal em tempo real.
79     ser.reset_input_buffer()
80     t0 = primeira = None
81     probar = True                  # so sonda a rede ate ela falhar;
           depois dispara direto
82     for tentativa in range(1, TENTATIVAS_CEGAS + 1):
83         if probar:
84             if medir_latencia() is not None:
85                 limpar_pi()        # rede viva: garante que so ha este
                       throttle
86             else:
87                 log.warning("rede morta; disparando no escuro ate ela
                       voltar")
88                 probar = False
89         enviar_comando(angulo, DURACAO_S)
90         limite = time.time() + ESPERA_RESPOSTA_S
91         while time.time() < limite:
92             i = le_corrente(ser)
93             if i is not None and i >= LIMIAR_RESPOSTA_A:
94                 t0, primeira = time.time(), i
95                 break
96         if t0 is not None:
97             break
```

```
98          log.warning("degrau %d sem resposta na corrente; re-disparando
                (%d/%d)",
99                      angulo, tentativa, TENTATIVAS_CEGAS)
100         probar = True             # nova tentativa reavalia a rede
101         time.sleep(3)
102
103     if t0 is None:
104         log.error("DEGRAU %d NAO executado apos %d disparos; pulado",
105                   angulo, TENTATIVAS_CEGAS)
106         return False
107
108     log.info(">> DEGRAU %d: resposta em %.1f s (I=%.2f A); medindo %d
            s",
109              angulo, t0 - t_ensaio0, primeira, DURACAO_S)
110     leituras = [(0.0, primeira)]
111     if not gravar_por_segundo:
112         csv_writer.writerow([f"{t0 - t_ensaio0:.1f}", "0.0", angulo,
113                              f"{primeira:.2f}"])
114     print(f"  t=0s | I={primeira:.2f} A (resposta)", flush=True)
115
116     fim = t0 + DURACAO_S
117     seg_ant, soma_seg, n_seg = 0, 0.0, 0
118     mm_seg = deque(maxlen=MM_SEGUNDOS)
119     while time.time() < fim:
120         i = le_corrente(ser)
121         if i is None:
122             continue
123         t = time.time() - t0
124         if not gravar_por_segundo:
125             csv_writer.writerow([f"{time.time() - t_ensaio0:.1f}", f"{
                    t:.1f}",
126                                  angulo, f"{i:.2f}"])
127         leituras.append((t, i))
128         soma_seg += i
129         n_seg += 1
130         seg = int(t)
131         if seg > seg_ant:
132             m_seg = soma_seg / n_seg
133             mm_seg.append(m_seg)
134             if gravar_por_segundo:
135                 csv_writer.writerow([f"{time.time() - t_ensaio0:.1f}",
136                                      f"{float(seg_ant):.1f}", angulo,
137                                      f"{m_seg:.2f}"])
138             print(f"  t={seg_ant}s | I={m_seg:.2f} A | MM={media(
                    mm_seg):.2f} A",
```

```
139                     flush=True)
140             seg_ant, soma_seg, n_seg = seg, 0.0, 0
141     if n_seg:
142         m_seg = soma_seg / n_seg
143         mm_seg.append(m_seg)
144         if gravar_por_segundo:
145             csv_writer.writerow([f"{time.time() - t_ensaio0:.1f}",
146                                  f"{float(seg_ant):.1f}", angulo,
147                                  f"{m_seg:.2f}"])
148         print(f"  t={seg_ant}s | I={m_seg:.2f} A | MM={media(mm_seg)
                :.2f} A",
149               flush=True)
150 
151     garantir_neutro() if medir_latencia() is not None else
            enviar_comando(NEUTRO_GRAUS, 2)
152     resumo_degrau(angulo, leituras)
153     return True
154 
155 
156 def main(direcao=DIRECAO, nome_csv=NOME_CSV,
157          angulo_inicial=ANGULO_INICIAL, angulo_final=ANGULO_FINAL):
158     # Monta a escada de degraus na direcao pedida, fixa o cwd junto ao
            script
159     # (o CSV cai sempre na pasta do ensaio), trava a instancia unica e
            percorre
160     # a escada com pausa e repouso entre degraus.
161     if direcao == "descida":
162         escada = range(angulo_final, angulo_inicial - 1, -1)
163     else:
164         escada = range(angulo_inicial, angulo_final + 1)
165 
166     os.chdir(os.path.dirname(os.path.abspath(__file__)))
167     with instancia_unica(LOCK_PATH), rede_acordada():
168         with Serial(PORTA_SERIAL, BAUD, timeout=3) as ser:
169             time.sleep(2)
170             ser.reset_input_buffer()
171 
172             t_ensaio0 = time.time()
173             existe = os.path.exists(nome_csv) and os.path.getsize(
                    nome_csv) > 0
174             modo = "a" if existe else "w"
175             with open(nome_csv, modo, buffering=1, newline="") as arq:
176                 csv_writer = csv.writer(arq, delimiter=";")
177                 if not existe:
178                     csv_writer.writerow(CABECALHO_CSV.split(";"))
```

```
179                 try:
180                     for angulo in escada:
181                         aguardar_repouso(ser)
182                         disparar_e_medir(ser, csv_writer, angulo,
                               t_ensaio0)
183                         time.sleep(PAUSA_ENTRE_DEGRAUS_S)
184                 except KeyboardInterrupt:
185                     log.warning("Interrompido; enviando neutro e
                           fechando CSV.")
186                     garantir_neutro()
187                     return False
188     log.info("Ensaio completo. CSV salvo em %s", nome_csv)
189     return True
190
191
192 if __name__ == "__main__":
193     main()
```

A.5 Efeito da bateria nos ensaios

```
1  """Analise do efeito de bateria degradada (sag de tensao e corte do
       ESC).
2
3  Durante os ensaios de degraus de comando, uma das sessoes de descida
       foi
4  executada com baterias antigas/degradadas do laboratorio. Sob alta
       corrente,
5  a tensao da bateria caia (sag severo) e o corte de subtensao do ESC
6  interrompia o motor, gerando um comportamento ciclico de aceleracao e
       parada.
7  Esta rotina compara essa sessao com a sessao de referencia (bateria
       boa,
8  ensaio de subida) e gera a figura 'bateria_sag.png' para a monografia.
9
10 LE:
11 - ensaios/ensaio_cego_descida.csv (sessao com bateria antiga; colunas
12   t_ensaio_s, t_degrau_s, angulo, corrente_a, amostragem ~100 Hz)
13 - reprodutibilidade/ensaio_tracao_corrente.csv (sessao de referencia
       com
14   bateria boa, mesmo formato; usa a corrida principal de 100 a 114
         graus)
15
16 GERA:
```

```
- Figuras/bateria_sag.png (300 dpi): painel (a) com a mediana de corrente
  por degrau das duas sessoes, evidenciando o colapso nas retomadas do
  degrau de 114 graus da sessao degradada; painel (b) com a forma de onda
  a 100 Hz do degrau colapsado ao lado do degrau saudavel de referencia.

Uso:  python3 analise_bateria.py
"""

# ---------------------------------------------------------------------------

# MAPA
# 1) Constantes e leitura dos CSVs (linhas 40-60)
# 2) Separacao dos blocos de degrau e estatisticas por bloco (linhas 63-90)
# 3) Selecao das corridas: boa (subida principal) e antiga (descida) (linhas 93-120)
# 4) Deteccao dos ciclos de corte do ESC no bloco colapsado (linhas 123-140)
# 5) Figura bateria_sag.png (linhas 143-210)
# 6) Impressao dos numeros-chave para o texto da monografia (linhas 213-245)
# ---------------------------------------------------------------------------


import csv
from pathlib import Path

import matplotlib

matplotlib.use("Agg")
import matplotlib.pyplot as plt
import numpy as np

PASTA_RAIZ = Path(__file__).resolve().parents[2]
PASTA_DADOS = PASTA_RAIZ / "ensaios"
PASTA_REPRO = Path(__file__).resolve().parent
PASTA_FIGURAS = PASTA_REPRO.parent / "Figuras"

TAXA_HZ = 100.0          # amostragem da corrente nos ensaios (~1500 amostras/15 s)
```

```
50 LIMIAR_CORTE_A = 5.0      # abaixo disso o motor esta cortado (corrente
       de repouso ~0 A)
51 TAXA_REF = PASTA_REPRO / "ensaio_tracao_corrente.csv"
52 TAXA_ANTIGA = PASTA_DADOS / "ensaio_cego_descida.csv"
53
54
55 def ler_csv(caminho):
56     """Le o CSV do ensaio (separador ';') e retorna matriz [t_ensaio,
           t_degrau, angulo, corrente]."""
57     with open(caminho) as arquivo:
58         linhas = list(csv.reader(arquivo, delimiter=";"))
59     return np.array([[float(v) for v in linha] for linha in linhas[1:]
           if len(linha) == 4])
60
61
62 def separar_blocos(dados):
63     """Separa a serie em blocos contiguos de mesmo angulo de comando.
           """
64     angulos = dados[:, 2]
65     blocos = []
66     inicio = 0
67     for i in range(1, len(dados) + 1):
68         if i == len(dados) or angulos[i] != angulos[inicio]:
69             blocos.append(dados[inicio:i])
70             inicio = i
71     return blocos
72
73
74 def estatistica_bloco(bloco):
75     """Mediana, percentil 95 e fracao de amostras abaixo de 0,5 A de
           um bloco."""
76     corrente = bloco[:, 3]
77     return {
78         "angulo": bloco[0, 2],
79         "n": len(bloco),
80         "mediana": float(np.median(corrente)),
81         "p95": float(np.percentile(corrente, 95)),
82         "frac_zero": float((np.abs(corrente) < 0.5).mean()),
83         "t_degrau": bloco[:, 1],
84         "corrente": corrente,
85     }
86
87
88 def main():
```

```
89      # --- 1) leitura
            ----------------------------------------------------------
90      referencia = ler_csv(TAXA_REF)
91      antiga = ler_csv(TAXA_ANTIGA)
92
93      blocos_ref = [estatistica_bloco(b) for b in separar_blocos(
            referencia)]
94      blocos_ant = [estatistica_bloco(b) for b in separar_blocos(antiga)
            ]
95
96      # --- 2) selecao das corridas
            ------------------------------------------
97      # Referencia (bateria boa): corrida principal da subida, blocos de
            ~15 s
98      # (~1500 amostras) de 100 a 114 graus, com corrente crescente
            normal.
99      # Blocos de 1 Hz (n pequeno) e blocos com picos > 100 A (
            corrompidos,
100     # ver ensaio_consolidado.csv) sao descartados.
101     boa = [
102         b for b in blocos_ref
103         if 100 <= b["angulo"] <= 114 and b["n"] > 1000 and b["p95"] <
                100
104     ]
105     boa.sort(key=lambda b: b["angulo"])
106     # a corrida principal sobe de 100 a 114 sem repetir angulo;
            retomadas
107     # posteriores do mesmo degrau (ultima corrida do arquivo) sao
            descartadas
108     vistos = set()
109     boa = [b for b in boa if not (b["angulo"] in vistos or vistos.add(
            b["angulo"]))]
110
111     # Sessao com bateria antiga (descida): primeira passagem 120 ->
            113,
112     # blocos saudaveis de ~15 s; as retomadas do degrau de 114 graus
            sao o
113     # bloco final colapsado (3 tentativas, corrente ciclando ate zerar
            ).
114     ant_saudavel = [
115         b for b in blocos_ant
116         if b["angulo"] >= 113 and b["n"] > 1000 and b["frac_zero"] <
                0.3
117     ]
118     ant_colapso = max(blocos_ant, key=lambda b: b["frac_zero"])
```

```
119
120     # --- 3) deteccao dos ciclos de corte no bloco colapsado
            ---------------
121     ligado = ant_colapso["corrente"] > LIMIAR_CORTE_A
122     bordas = np.diff(ligado.astype(int))
123     religamentos = np.where(bordas == 1)[0]
124     desligamentos = np.where(bordas == -1)[0]
125     corrente_ligada = ant_colapso["corrente"][ligado]
126     mediana_ligada = float(np.median(corrente_ligada)) if len(
            corrente_ligada) else float("nan")
127
128     # --- 4) numeros-chave
            ------------------------------------------------
129     boa_por_angulo = {b["angulo"]: b for b in boa}
130     ref_114 = boa_por_angulo[114.0]
131     print("=== Sessao de referencia (bateria boa, subida) - mediana
            por degrau ===")
132     for b in boa:
133         print(f"  {b['angulo']:5.0f} graus: mediana = {b['mediana
                ']:5.1f} A "
134               f"(p95 = {b['p95']:5.1f} A, zeros = {100*b['frac_zero
                    ']:.0f} %)")
135     print("\n=== Sessao com bateria antiga (descida) - primeira
            passagem ===")
136     for b in sorted(ant_saudavel, key=lambda b: -b["angulo"]):
137         print(f"  {b['angulo']:5.0f} graus: mediana = {b['mediana
                ']:5.1f} A "
138               f"(p95 = {b['p95']:5.1f} A, zeros = {100*b['frac_zero
                    ']:.0f} %)")
139     print("\n=== Colapso nas retomadas do degrau de 114 graus (bateria
            antiga) ===")
140     print(f"  mediana do bloco: {ant_colapso['mediana']:.1f} A "
141           f"(vs {ref_114['mediana']:.1f} A no degrau de 114 graus da
                sessao boa)")
142     print(f"  amostras ~zero: {100*ant_colapso['frac_zero']:.0f} %")
143     print(f"  corrente com o motor ligado: mediana = {mediana_ligada
            :.1f} A "
144           f"(p95 = {np.percentile(corrente_ligada, 95):.1f} A)")
145     print(f"  religamentos do motor detectados: {len(religamentos)} "
146           f"(ciclos de corte e retomada do ESC)")
147
148     # --- 5) figura
            ------------------------------------------------------------
149     plt.rcParams.update({
150         "font.size": 11, "axes.grid": True, "grid.alpha": 0.3,
```

```
        "font.family": "serif", "axes.titlesize": 12,
    })

    fig, ax = plt.subplots(figsize=(7.4, 4.6))
    fig.suptitle("Mediana da corrente por degrau: bateria em bom estado vs bateria degradada",
                 fontsize=13)
    ax.plot([b["angulo"] for b in boa], [b["mediana"] for b in boa],
            "-", color="tab:blue", lw=2, label="Bateria boa (subida)")
    ax.plot([b["angulo"] for b in ant_saudavel], [b["mediana"] for b in ant_saudavel],
            "-", color="tab:orange", lw=2, label="Bateria antiga (descida, 1.a passagem)")
    ax.plot(ant_colapso["angulo"], ant_colapso["mediana"], "v", color="tab:red",
            markersize=10, label="Bateria antiga (retomadas, colapso)")
    ax.annotate(f"{ant_colapso['mediana']:.0f} A ({100*ant_colapso['frac_zero']:.0f} % em zero)",
                xy=(ant_colapso["angulo"], ant_colapso["mediana"]),
                xytext=(ant_colapso["angulo"] - 3, ant_colapso["mediana"] - 6),
                fontsize=10, ha="right", arrowprops=dict(arrowstyle="->", lw=0.9))
    ax.set_xlabel("Degrau de comando (graus)")
    ax.set_ylabel("Mediana da corrente (A)")
    ax.set_ylim(0, 33)
    ax.legend(fontsize=10.5, loc="upper left")
    fig.tight_layout(rect=(0, 0, 1, 0.94))
    fig.savefig(PASTA_FIGURAS / "bateria_sag.png", dpi=300, bbox_inches="tight")
    plt.close(fig)
    print(f"\nFigura salva em {PASTA_FIGURAS / 'bateria_sag.png'}")


if __name__ == "__main__":
    main()
```